

IBS: Bezpečnost a počítačové sítě

Zabezpečení finančního styku

Brno University of Technology, Faculty of Information Technology

Božetěchova 1/2, 612 66 Brno - Královo Pole

Vladimír Veselý, veselyv@fit.vut.cz

Matěj Grégr, mgregr@fit.vut.cz



14th April 2025

- *Why Bitcoin matters to IT?*
 - P2P network (Bitcoin Protocol)
 - Language theory (Bitcoin Script)
 - Asymmetrical cryptography (ECDSA)
 - Hashing algorithms (SHA-256, RIPEMD-160)
 - Distributed system (Consensus)
 - Game theory (Mining)

1) History

2) Basics

3) Blockchain

4) Bitcoin Script

5) Protocol

6) Mining



- 28th October 2008
- <https://bitcoin.org/bitcoin.pdf>
 - Open and distributed consensus (with incentives and no trusted parties)
 - Peer-to-peer (equal participants)
 - Cheap, anonymous/private, instant, and censorship-free payments
- **Idea first, implementation next!**

Bitcoin: A Peer-to-Peer Electronic Cash System

Satoshi Nakamoto
satoshin@gmx.com
www.bitcoin.org

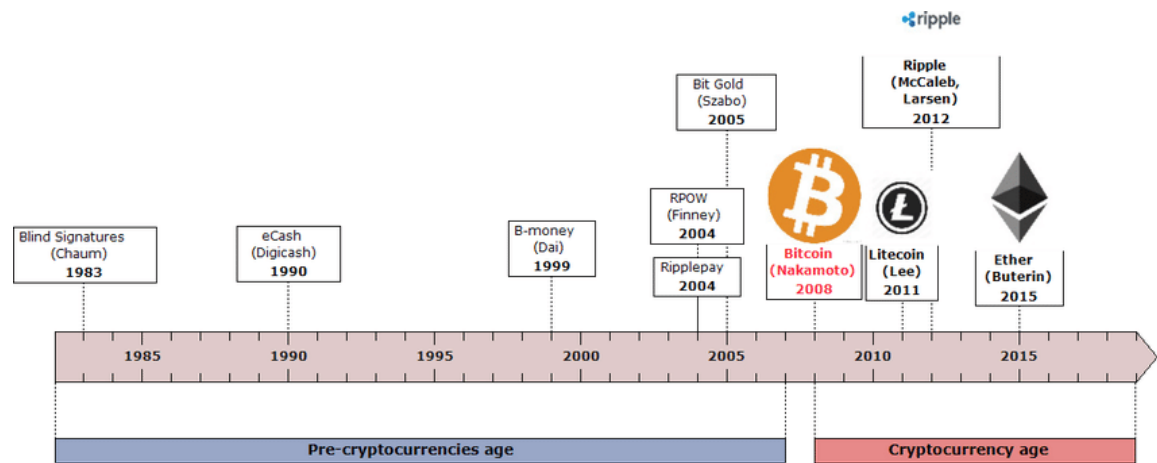
Abstract. A purely peer-to-peer version of electronic cash would allow online payments to be sent directly from one party to another without going through a financial institution. Digital signatures provide part of the solution, but the main benefits are lost if a trusted third party is still required to prevent double-spending. We propose a solution to the double-spending problem using a peer-to-peer network. The network timestamps transactions by hashing them into an ongoing chain of hash-based proof-of-work, forming a record that cannot be changed without redoing the proof-of-work. The longest chain not only serves as proof of the sequence of events witnessed, but proof that it came from the largest pool of CPU power. As long as a majority of CPU power is controlled by nodes that are not cooperating to attack the network, they'll generate the longest chain and outpace attackers. The network itself requires minimal structure. Messages are broadcast on a best effort basis, and nodes can leave and rejoin the network at will, accepting the longest proof-of-work chain as proof of what happened while they were gone.

1. Introduction

Commerce on the Internet has come to rely almost exclusively on financial institutions serving as trusted third parties to process electronic payments. While the system works well enough for most transactions, it still suffers from the inherent weaknesses of the trust based model. Completely non-reversible transactions are not really possible, since financial institutions cannot avoid mediating disputes. The cost of mediation increases transaction costs, limiting the minimum practical transaction size and cutting off the possibility for small casual transactions, and there is a broader cost in the loss of ability to make non-reversible payments for non-reversible services. With the possibility of reversal, the need for trust spreads. Merchants must be wary of their customers, hassling them for more information than they would otherwise need. A certain percentage of fraud is accepted as unavoidable. These costs and payment uncertainties can be avoided in person by using physical currency, but no mechanism exists to make payments over a communications channel without a trusted party.

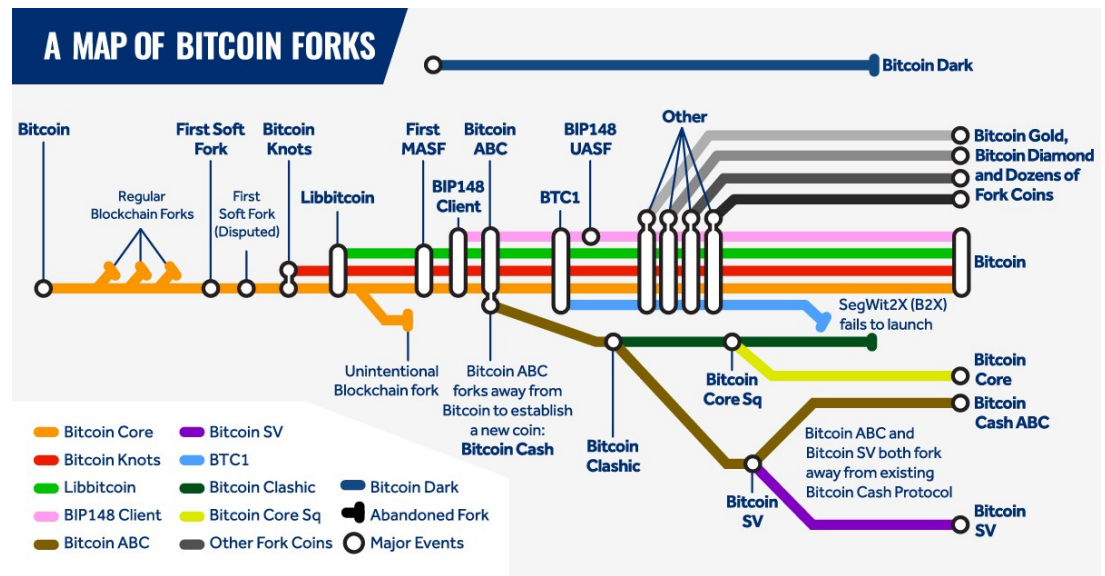
What is needed is an electronic payment system based on cryptographic proof instead of trust, allowing any two willing parties to transact directly with each other without the need for a trusted third party. Transactions that are computationally impractical to reverse would protect sellers from fraud, and routine escrow mechanisms could easily be implemented to protect buyers. In this paper, we propose a solution to the double-spending problem using a peer-to-peer distributed timestamp server to generate computational proof of the chronological order of transactions. The system is secure as long as honest nodes collectively control more CPU power than any cooperating group of attacker nodes.

- Dai's b-money
- Szabo's bit gold
- Chaum's Ecash



- Bitcoin Gold, Bitcoin Cash, Bitcoin SV, ...

- Litecoin
- Dogecoin
- Dash
- Monero



- Unknown persona
- Officially disappeared in 2011
- Candidates are/were



Nick
Szabo



Dorian
Nakamoto



Hal
Finney



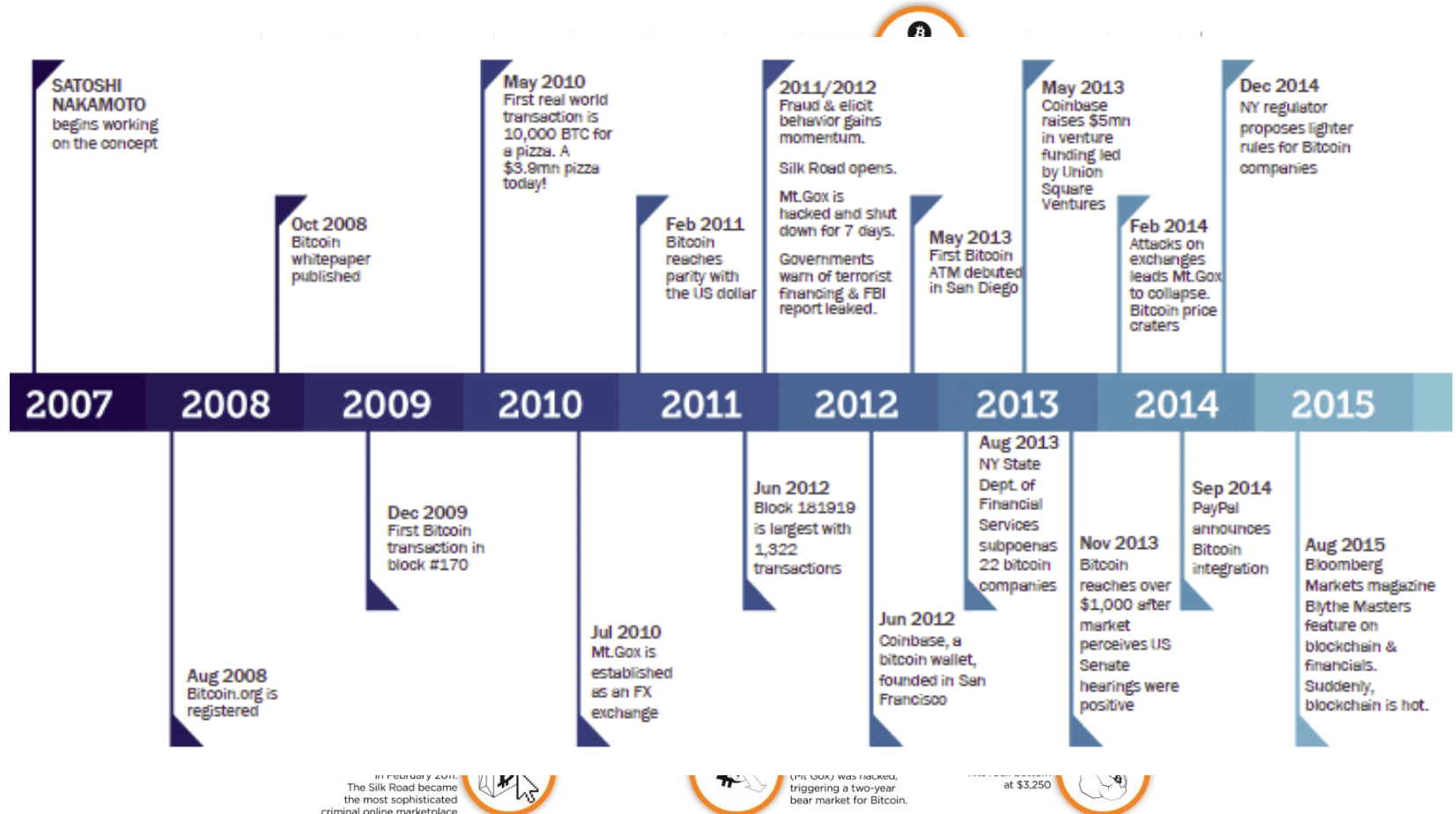
Craig
S. Wright



Len
Sassaman

Timeline of Bitcoin's Major Events

Market Cap (in Billions of \$) 2008 - Today



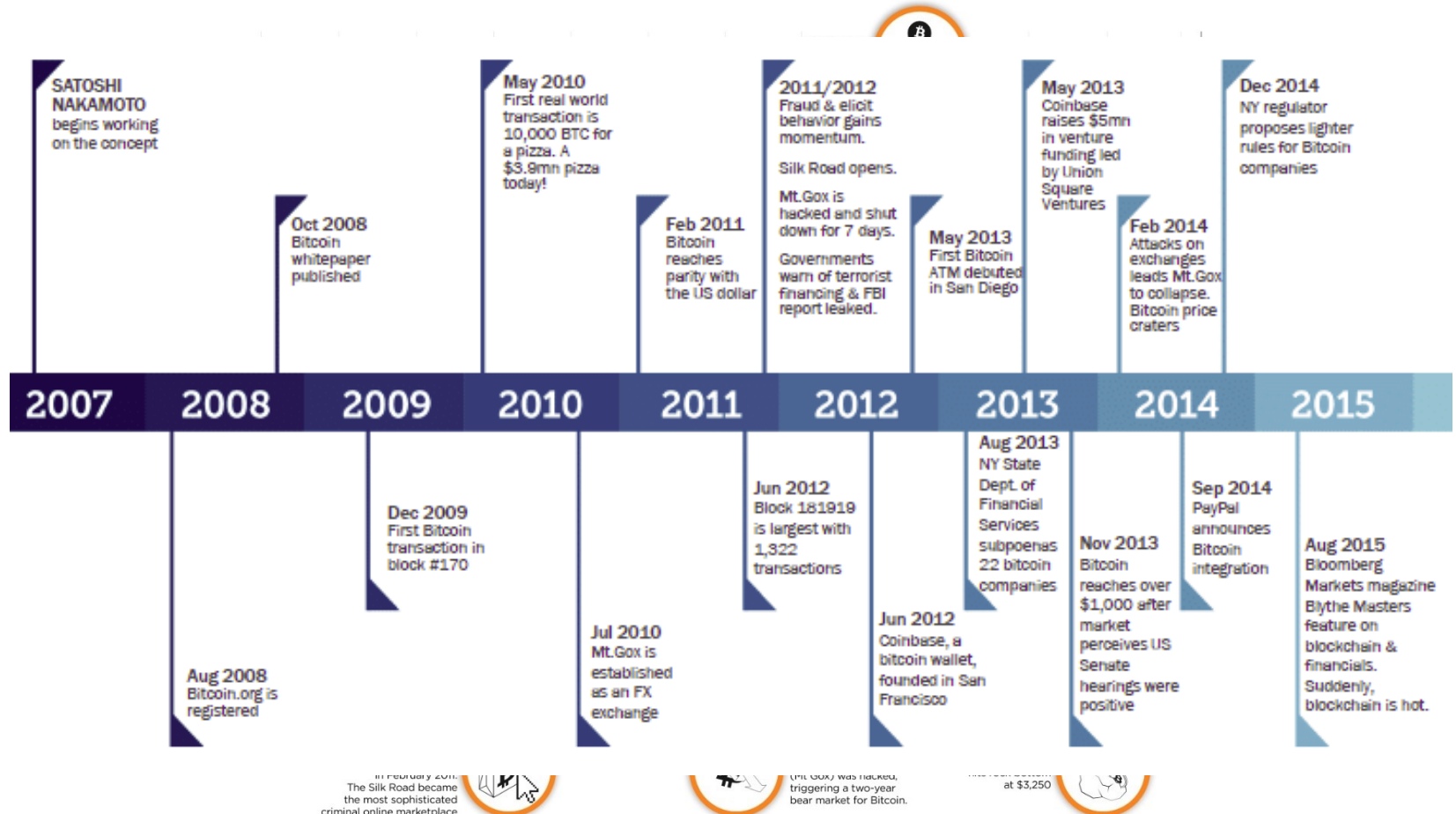
Note: Market Cap & Price as of 03/05/2020

Article & Sources:
<https://howmuch.net/articles/timeline-bitcoin-major-events>
[Bitcoinity.org](https://data.bitcoinity.org) - <https://data.bitcoinity.org>

howmuch.net

Timeline of Bitcoin's Major Events

Market Cap (in Billions of \$) 2008 - Today



Note: Market Cap & Price as of 03/05/2020

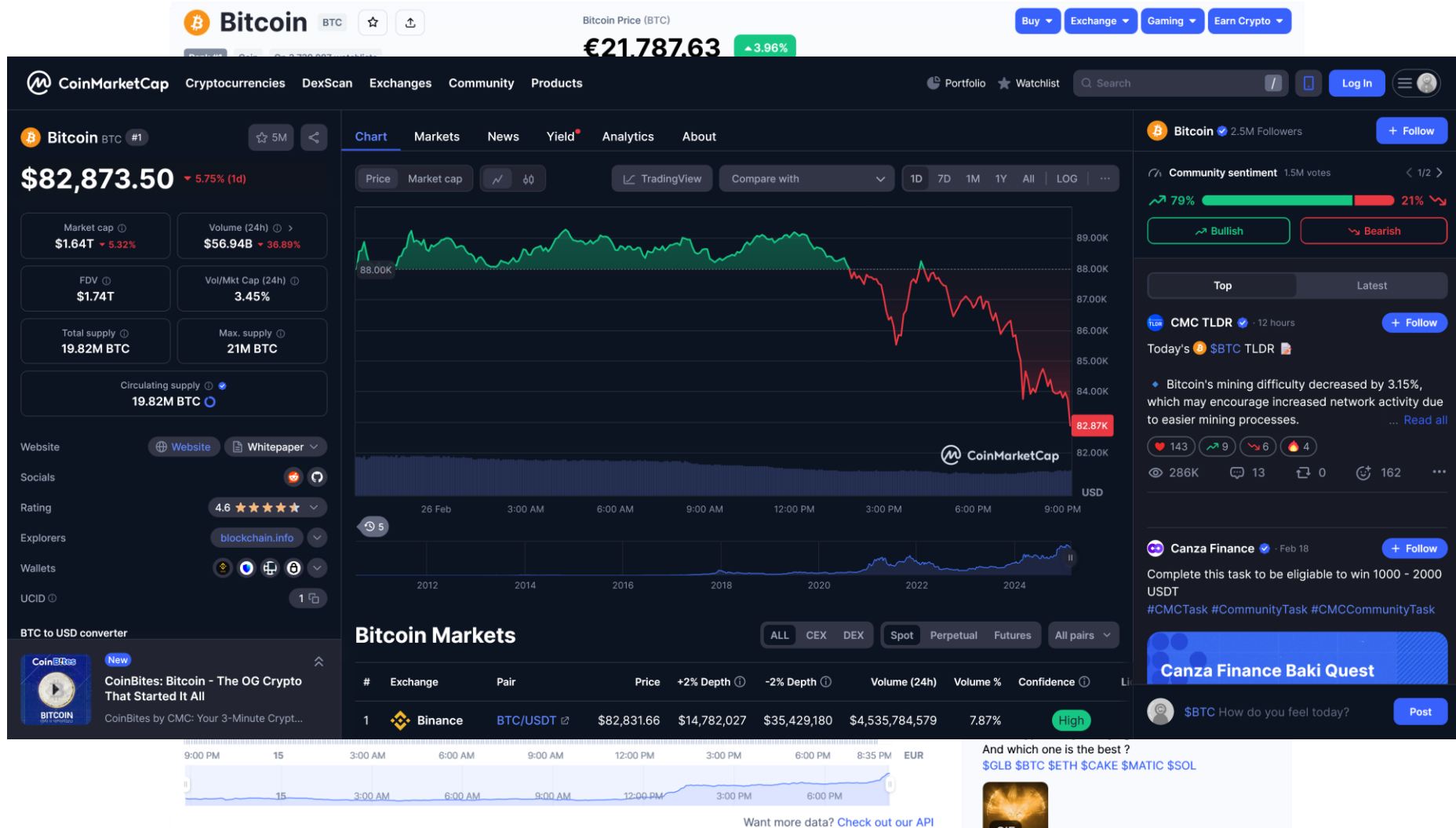
Article & Sources:
<https://howmuch.net/articles/timeline-bitcoin-major-events>
[Bitcoinity.org](https://data.bitcoinity.org) - <https://data.bitcoinity.org>

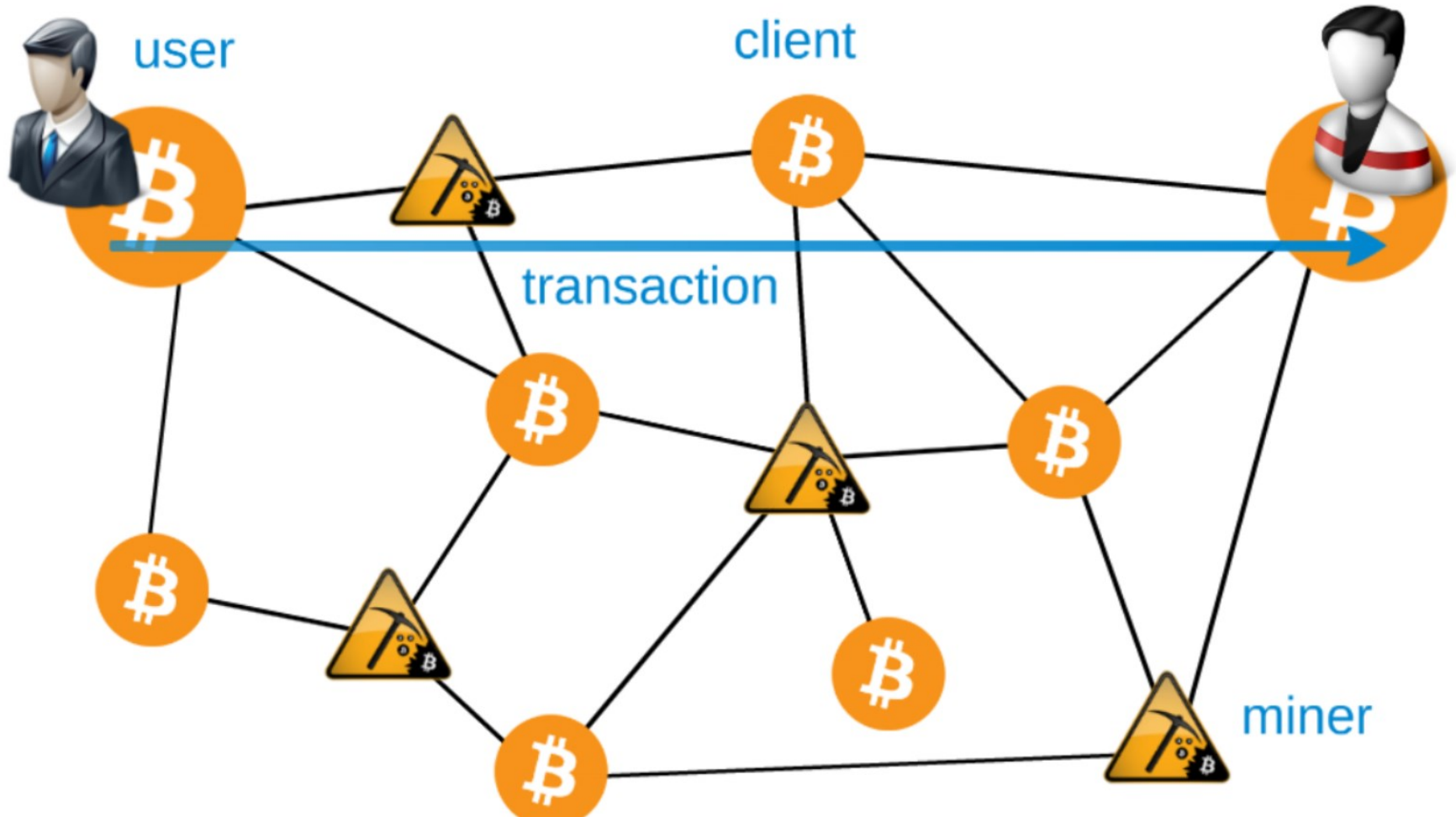
howmuch.net

BASICS

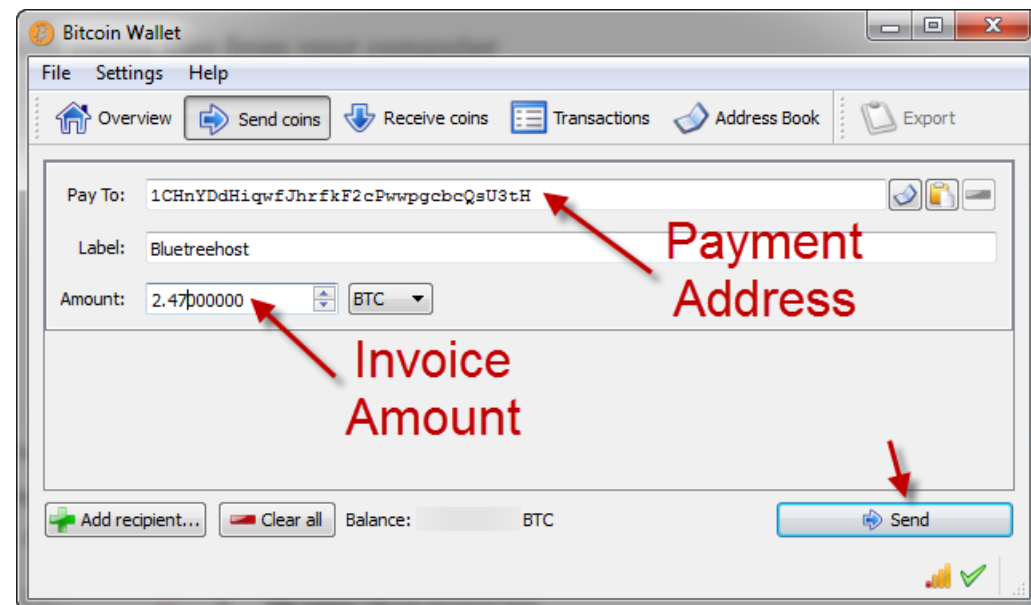
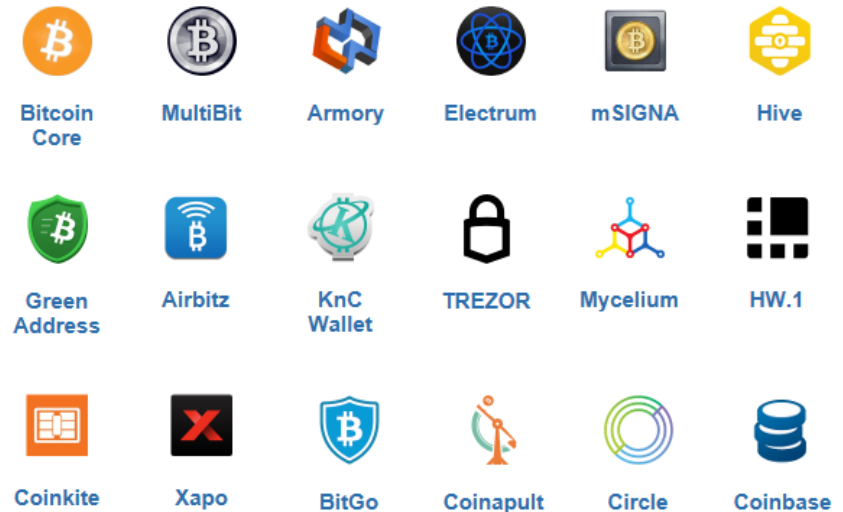
- Maximum supply = 21 mil. BTC
 - <https://github.com/bitcoin/bitcoin/blob/master/src/consensus/amount.h#L26>
- Maximum block size = 4 MB
 - <https://github.com/bitcoin/bitcoin/blob/master/src/consensus/consensus.h#L13>
- PoW
 - <https://github.com/bitcoin/bitcoin/blob/master/src/pow.cpp>
- Validation
 - <https://github.com/bitcoin/bitcoin/blob/master/src/validation.cpp>

- <https://coinmarketcap.com/currencies/bitcoin/>



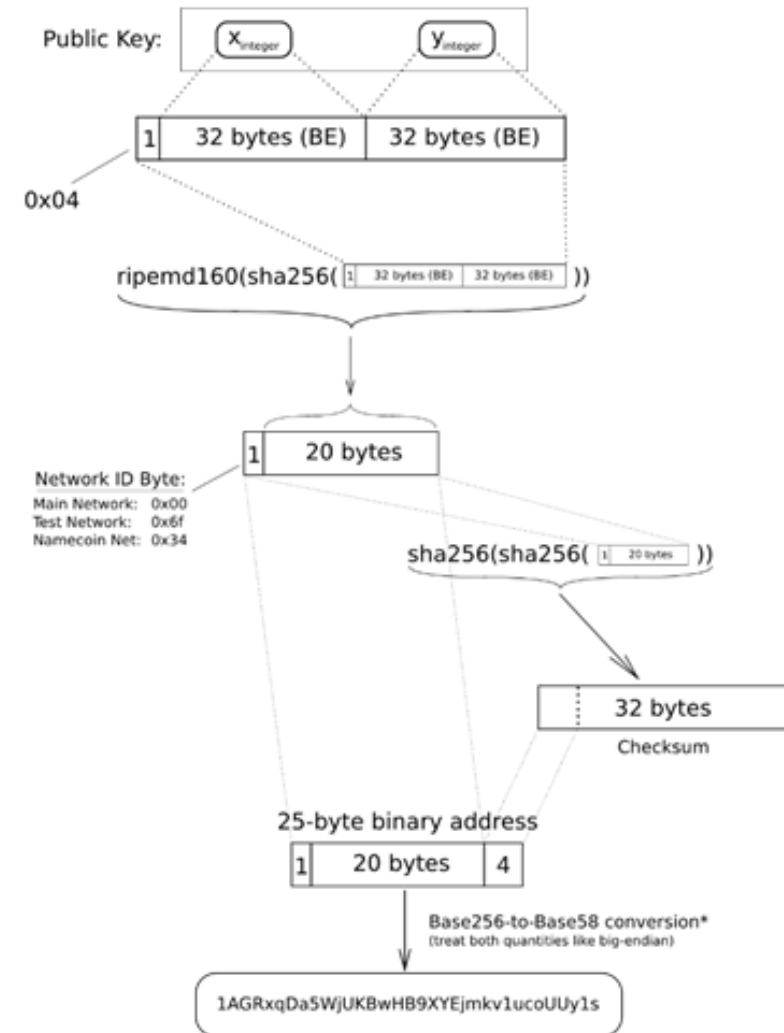


- communication with other nodes
 - maintains blockchain
 - issues transaction
 - manages addresses
-
- Online vs. Offline
 - SW vs. HW



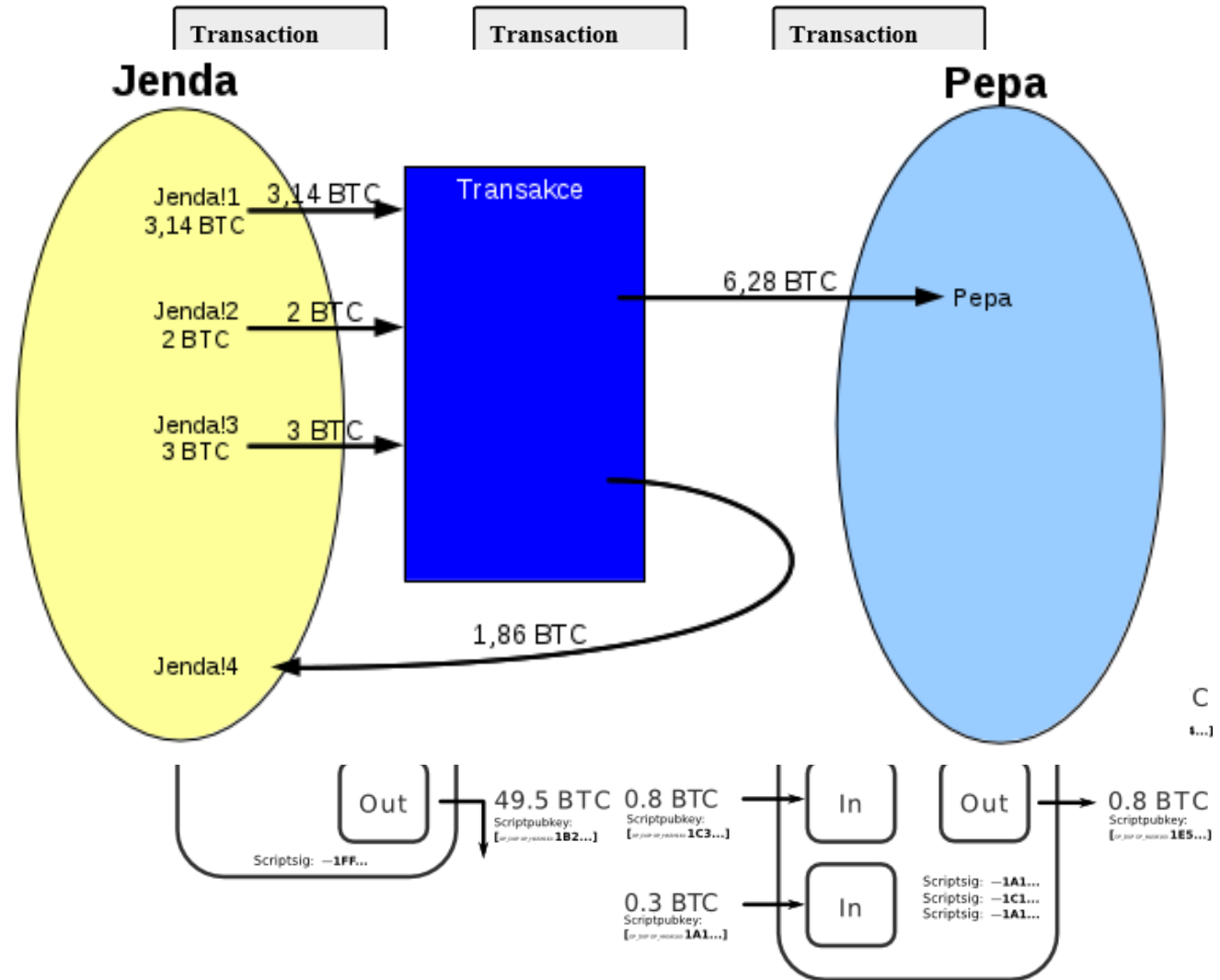
Elliptic-Curve Public Key to BTC Address conversion

- ECDSA private-public key pair
- Deterministic
 - (non)-BIP39
 - Seed phrase



- <http://blog.bitcoin.cz/2013/03/bitcoin-pod-kapotou/>

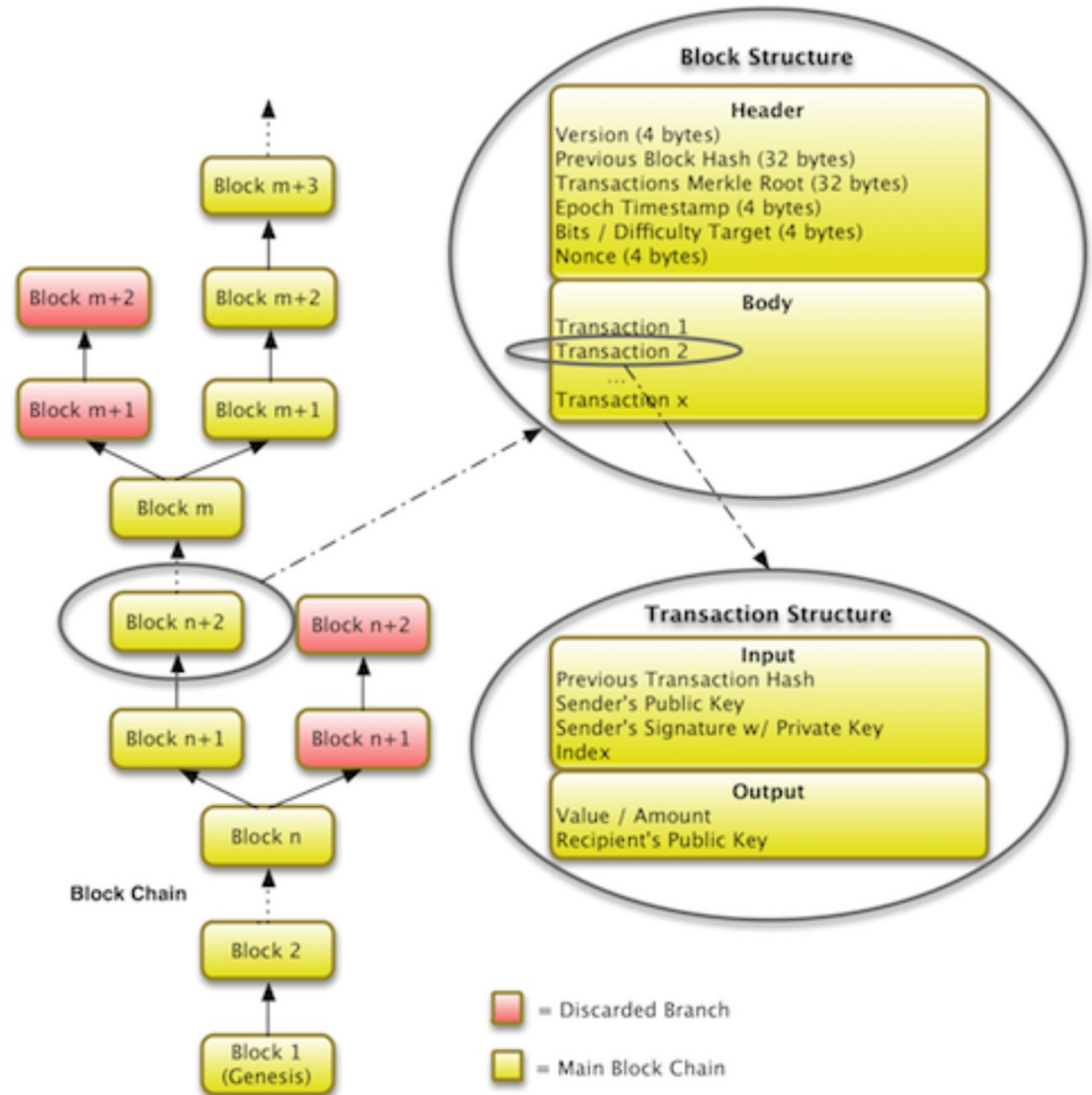
- M inputs
- N outputs



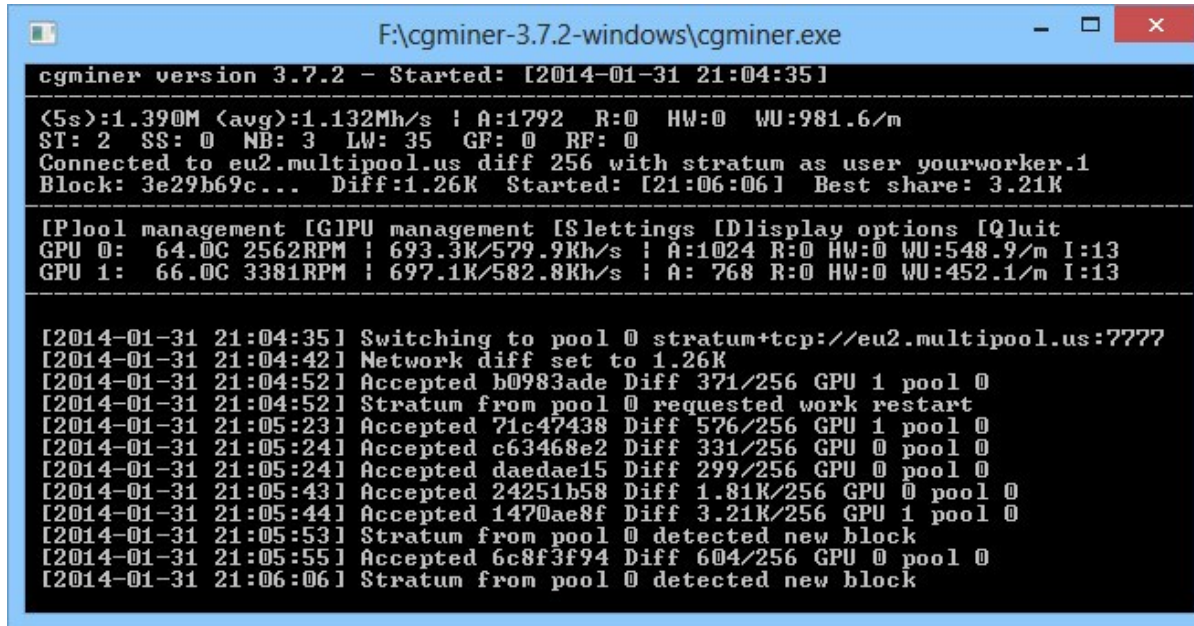
- Ethereum
 - 1 input
 - 1 output

- [illegible]

- 1 block every 10 minutes
 - <https://bitinfoch arts.com/comparison/bitcoin-confirmationtime.html#3y>
- “all” transactions during last block
 - <https://www.blockchain.com/charts/n-transactions-per-block>



- Verify transactions with special SW
 - bitcoind, CGMiner, BFGMiner

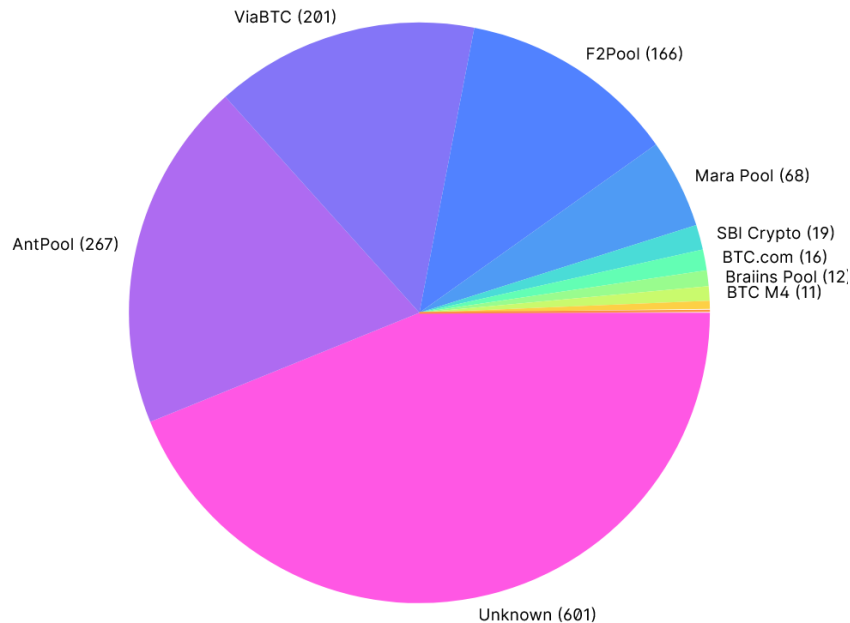


```
F:\cgminer-3.7.2-windows\cgminer.exe
cgminer version 3.7.2 - Started: [2014-01-31 21:04:35]
-----
<5s>:1.390M <avg>:1.132Mh/s | A:1792 R:0 HW:0 WU:981.6/m
ST: 2 SS: 0 NB: 3 LW: 35 GF: 0 RF: 0
Connected to eu2.multipool.us diff 256 with stratum as user yourworker.1
Block: 3e29b69c... Diff:1.26K Started: [21:06:06] Best share: 3.21K
-----
[P]ool management [G]PU management [S]ettings [D]isplay options [Q]uit
GPU 0: 64.0C 2562RPM | 693.3K/579.9Kh/s | A:1024 R:0 HW:0 WU:548.9/m I:13
GPU 1: 66.0C 3381RPM | 697.1K/582.8Kh/s | A: 768 R:0 HW:0 WU:452.1/m I:13
-----
[2014-01-31 21:04:35] Switching to pool 0 stratum+tcp://eu2.multipool.us:7777
[2014-01-31 21:04:42] Network diff set to 1.26K
[2014-01-31 21:04:52] Accepted b0983ade Diff 371/256 GPU 1 pool 0
[2014-01-31 21:04:52] Stratum from pool 0 requested work restart
[2014-01-31 21:05:23] Accepted 71c47438 Diff 576/256 GPU 1 pool 0
[2014-01-31 21:05:24] Accepted c63468e2 Diff 331/256 GPU 0 pool 0
[2014-01-31 21:05:24] Accepted daedae15 Diff 299/256 GPU 0 pool 0
[2014-01-31 21:05:43] Accepted 24251b58 Diff 1.81K/256 GPU 0 pool 0
[2014-01-31 21:05:44] Accepted 1470ae8f Diff 3.21K/256 GPU 1 pool 0
[2014-01-31 21:05:53] Stratum from pool 0 detected new block
[2014-01-31 21:05:55] Accepted 6c8f3f94 Diff 604/256 GPU 0 pool 0
[2014-01-31 21:06:06] Stratum from pool 0 detected new block
```

- Compete for block reward
 - <https://www.bitcoinblockhalf.com/>

- Solo-mining vs. Pool-mining

- <https://www.blockchain.com/explorer/charts/pools>
- https://btc.com/stats/pool?pool_mode=day

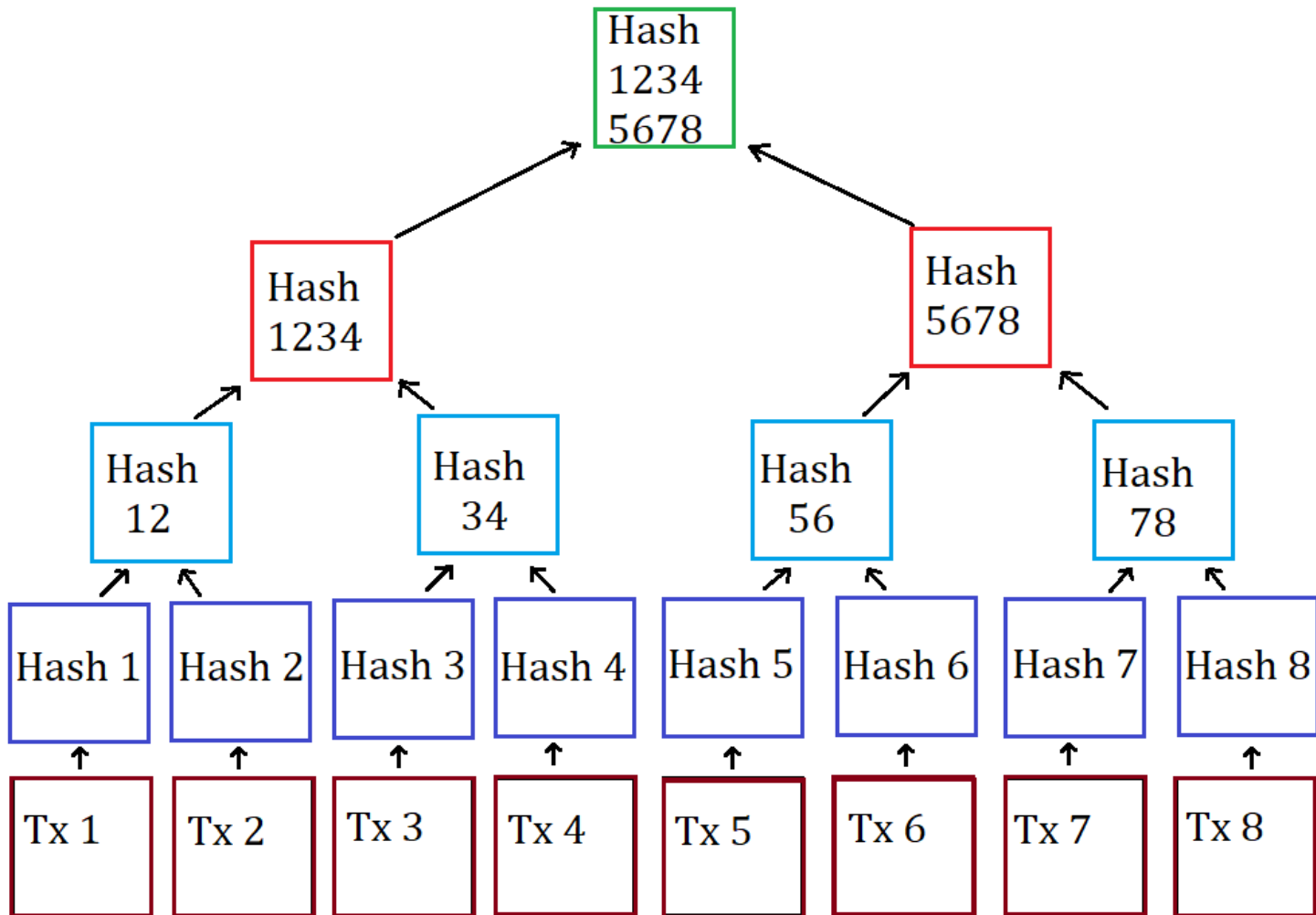


Summary of Mined Blocks

Miner / Pool	Percent	Blocks Mined
Unknown	43.869%	601
AntPool	19.489%	267
ViaBTC	14.672%	201
F2Pool	12.117%	166
Mara Pool	4.964%	68
SBI Crypto	1.387%	19
BTC.com	1.168%	16
Braiiins Pool	0.876%	12
BTC M4	0.803%	11
Ultimus	0.438%	6
Poolin	0.219%	3

DIVING DEEPER INTO BLOCKCHAIN

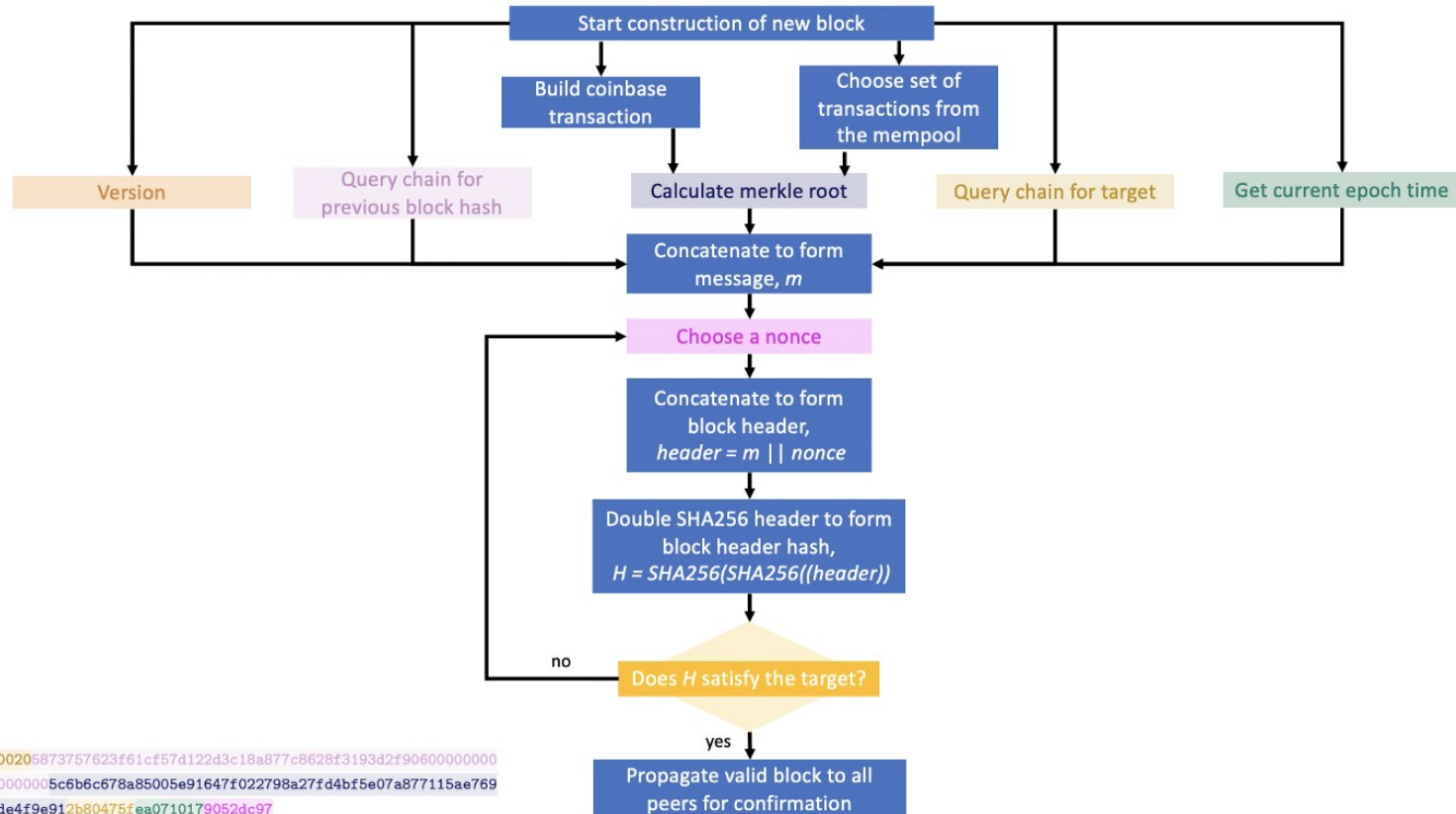
- Block has Merkel Tree structure



- <https://www.blockchain.com/btc/block/100000>

Preamble	magic number			4 bytes, fixed value
	block size			4 bytes
Block	Block Header (used to calculate hash)	version		4 bytes
		pre block hash		32 bytes
		merkle root		32 bytes
		time		4 bytes
		bits		4 bytes
		nonce		4 bytes
		number of transaction		variable integer
	Transaction (used to calculate hash)	version		4 bytes
		number of input		variable integer
		Transaction Input	pre tx hash	32 bytes
			pre tx out index	4 bytes, Signed Integer
			script length	variable integer
			script	specified by script length
			sequence	4 bytes
		more input ...		
		number of output		variable integer
		Transaction Output	value	8 bytes
			script length	variable integer
			script	specified by script length
		more output ...		
		lock time		4 bytes
	more transactions ...			

- Hash of block header < target (proves that a lot of PoW was invested)
- Security (to misbehave a significant mining power is required)
- Fast verification



- <https://medium.com/fcats-blockchain-incubator/understanding-the-bitcoin-blockchain-header-a2b0db06b515>

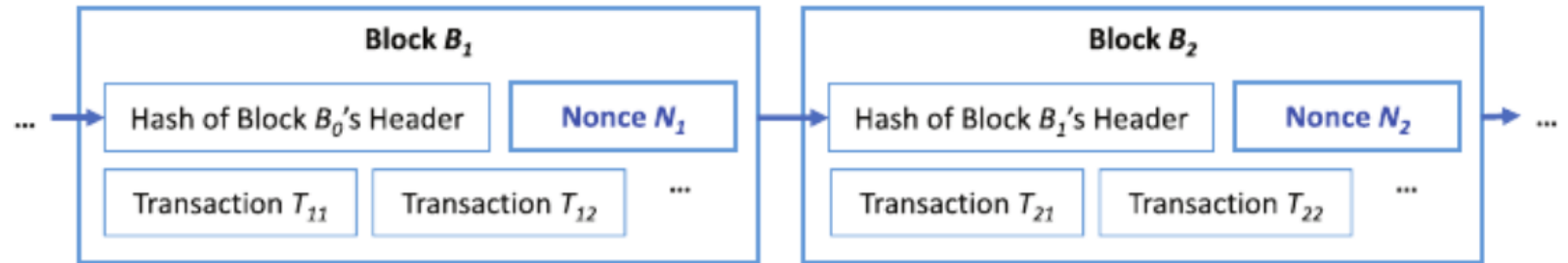
```
import hashlib, struct

ver = 2
prev_block = "000000000000000117c80378b8da0e33559b5997f2ad55e2f7d18ec1975b9717"
mrkl_root = "871714dcbae6c8193a2bb9b2a69felc0440399f38d94b3a0f1b447275a29978a"
time_ = 0x53058b35 # 2014-02-20 04:57:25
bits = 0x19015f53

exp = bits >> 24
mant = bits & 0xffffffff
target_hexstr = '%064x' % (mant * (1<<(8*(exp - 3))))
target_str = target_hexstr.decode('hex')

nonce = 0
while nonce < 0x100000000:
    header = ( struct.pack("<L", ver) + prev_block.decode('hex')[::-1] +
               mrkl_root.decode('hex')[::-1] + struct.pack("<LLL", time_, bits, nonce))
    hash = hashlib.sha256(hashlib.sha256(header).digest()).digest()
    print nonce, hash[::-1].encode('hex')
    if hash[::-1] < target_str:
        print 'success'
        break
    nonce += 1
```

- <http://www.righto.com/2014/02/bitcoin-mining-hard-way-algorithms.html>
- Paper <https://www.youtube.com/watch?v=y3dqhixzGVo>
- Excel <https://www.youtube.com/watch?v=UZZBZPOEVyJA>



Every node starts proof-of-work

Node 1 creates new block B_2

Node 1

...
 Nonce N_2 = "7C 4D DB 29" → Hash of B_2 's header = "2D F8 8E 32 ... 10 9A FE 1C", Failed (time = 10:14:20)
 Nonce N_2 = "7C 4D DB 30" → Hash of B_2 's header = "41 2A B3 DC ... 94 29 AB B5", Failed (time = 10:14:25)
 Nonce N_2 = "7C 4D DB 31" → Hash of B_2 's header = "00 00 4F 65 ... 2F ED 31 09", Succeeded (time = 10:14:30)

Node 2

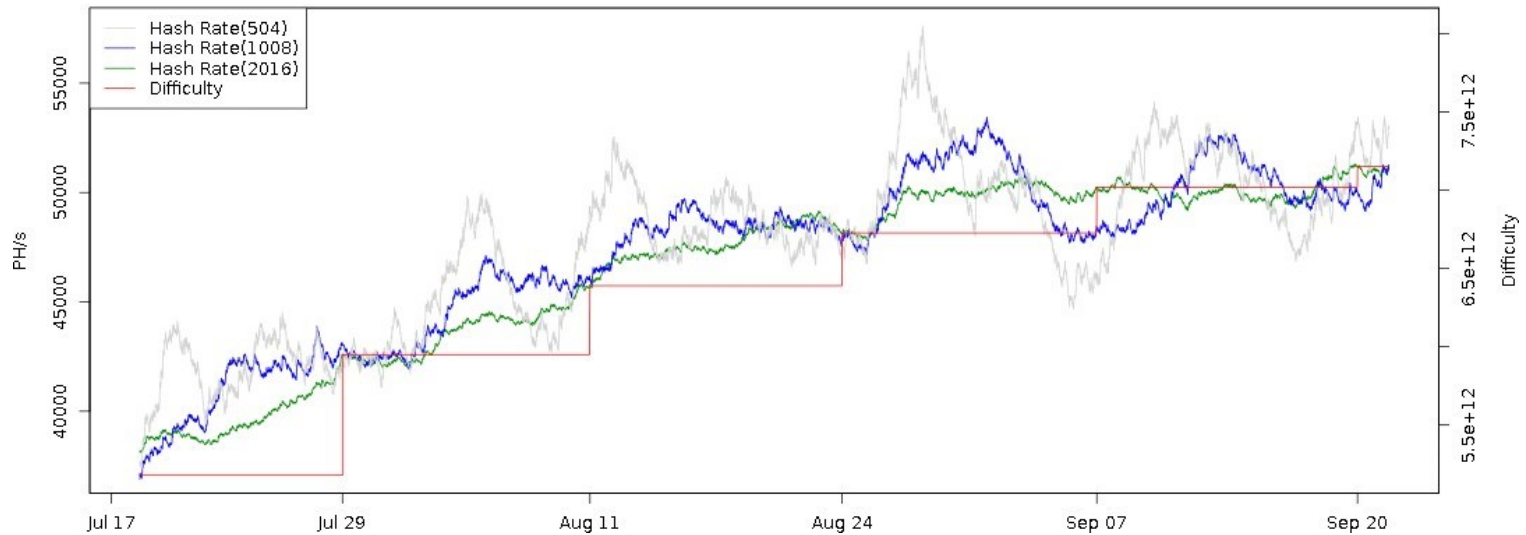
...
 Nonce N_2 = "61 0A 3F 3A" → Hash of B_2 's header = "A8 C7 08 C9 ... 3D F1 A2 F9", Failed (time = 10:14:23)
 Nonce N_2 = "61 0A 3F 3B" → Hash of B_2 's header = "2A E9 84 66 ... 91 B4 58 CE", Failed (time = 10:14:28)
 Stopped after identifying that Node 1 has completed proof-of-work at time = 10:14:30

Node 3

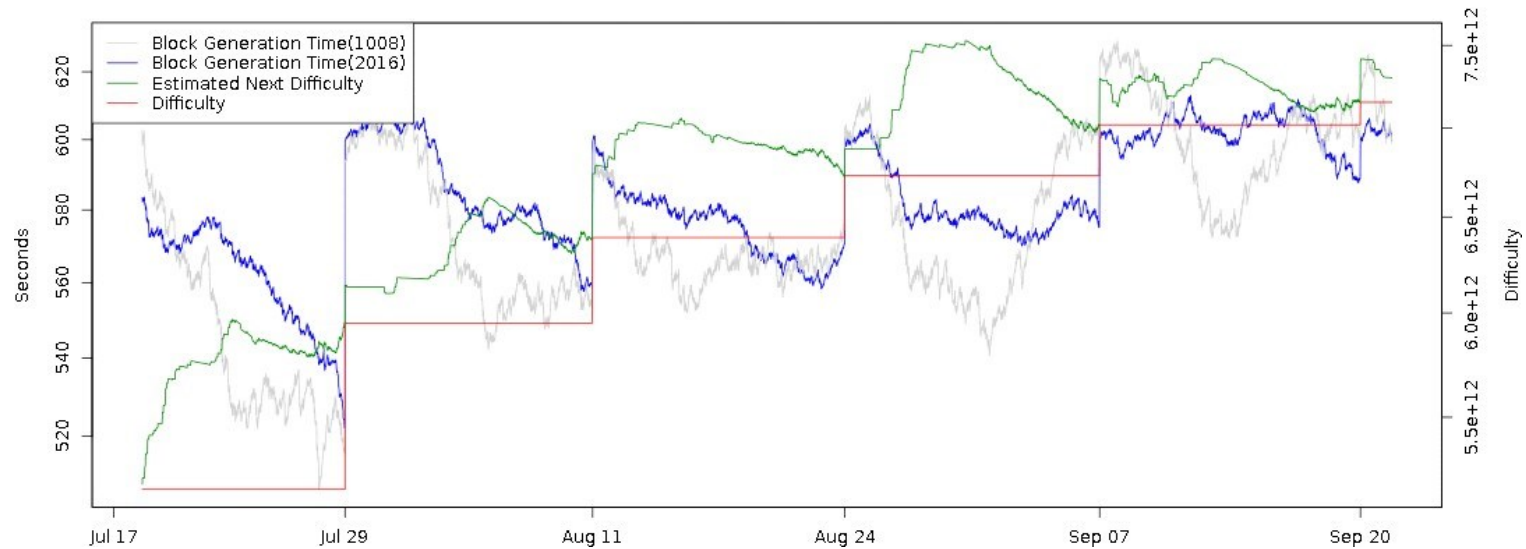
...
 Nonce N_2 = "99 06 10 13" → Hash of B_2 's header = "FB 2F 26 D9 ... 39 F5 C1 0B", Failed (time = 10:14:21)
 Nonce N_2 = "99 06 10 14" → Hash of B_2 's header = "E2 1C 09 05 ... 25 3E AA CF", Failed (time = 10:14:26)
 Stopped after identifying that Node 1 has completed proof-of-work at time = 10:14:30

- Difficulty adjusts block production based on available hashrate
 - <https://www.blockchain.com/charts/difficulty>
 - <https://en.bitcoin.it/wiki/Difficulty>
- Retarget every 2016 blocks ~ 2 weeks
 - $\Delta = B(i + 1) \times 2016_timestamp - B_i \times 2016_timestamp$
 - If ($\Delta < \text{two weeks}$) then increase difficulty proportionally (max 300%)
 - If ($\Delta > \text{two weeks}$) then decrease difficulty proportionally (max 75%)

Bitcoin Hash Rate vs Difficulty (2 Months)



Bitcoin Block Generation Time vs Difficulty



- <https://www.blockchain.com/explorer/charts/hash-rate>

Total Hash Rate (TH/s)

The estimated number of terahashes per second the bitcoin network is performing in the last 24 hours.

Scales
Linear

Average

Type
Line

Colors
Blue Black



1M

3M

6M

1Y

3Y

All



- [illegible]

Name	Type	Bytes	Description
version	int32.t	4	Version number
witness_marker	x	1	Witness marker
witness_flag	x	1	Witness flag
num_inputs	VarInt	1 - 9	Number of inputs
prev_hash	x	32	32 0-byte placeholder
prev_hash_index	x	32	0xffffffff place holder
unlocking_script	x	Script size	Unlocking script
n_sequence	VarInt	1 - 9	Sequence
num_outputs	VarInt	1 - 9	Number of outputs
amount	uint64.t	8	UTXO amount being created in sats
locking_script	x	Script size	Locking script
n_locktime	int32.t	4	Lock time

28

- 3rd January 2009 at 18:15 UTC

00000000	01 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000010	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000020	00 00 00 00 3B A3 ED FD	7A 7B 12 B2 7A C7 2C 3E	;fíýz{.²zÇ,>
00000030	67 76 8F 61 7F C8 1B C3	88 8A 51 32 3A 9F B8 AA	gv.a.È.Ã~ŠQ2:Ÿ_a
00000040	4B 1E 5E 4A 29 AB 5F 49	FF FF 00 1D 1D AC 2B 7C	K.^J)«_IŸŸ...¬+
00000050	01 01 00 00 00 01 00 00	00 00 00 00 00 00 00 00	
00000060	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00000070	00 00 00 00 00 00 FF FF	FF FF 4D 04 FF FF 00 1D	ŸŸŸŸM.ŸŸ..
00000080	01 04 45 54 68 65 20 54	69 6D 65 73 20 30 33 2F	..EThe Times 03/
00000090	4A 61 6E 2F 32 30 30 39	20 43 68 61 6E 63 65 6C	Jan/2009 Chancel
000000A0	6C 6F 72 20 6F 6E 20 62	72 69 6E 6B 20 6F 66 20	lor on brink of
000000B0	73 65 63 6F 6E 64 20 62	61 69 6C 6F 75 74 20 66	second bailout f
000000C0	6F 72 20 62 61 6E 6B 73	FF FF FF FF 01 00 F2 05	or banksŸŸŸŸ..ð.
000000D0	2A 01 00 00 00 43 41 04	67 8A FD B0 FE 55 48 27	*....CA.gŠŸ°pUH'
000000E0	19 67 F1 A6 71 30 B7 10	5C D6 A8 28 E0 39 09 A6	.gñ q0•.\ð"(à9.
000000F0	79 62 E0 EA 1F 61 DE B6	49 F6 BC 3F 4C EF 38 C4	ybàè.ab¶IÖ¿?Lİ8Ä
00000100	F3 55 04 E5 1E C1 12 DE	5C 38 4D F7 BA 0B 8D 57	ÖU.Å.Å.ð\8M+q...W
00000110	8A 4C 70 2B 6B F1 1D 5F	AC 00 00 00 00	šLp+kñ._¬....

THE TIMES

Max SC, min -5C

Saturday January 3 2009 timesonline.co.uk No 69523

£1.50

Eat Out from £5

More than 900 great restaurants, including four Gordon Ramsay favourites from £15

Start collecting tokens today! Pullout inside

Israel prepares to send tanks and troops into Gaza

Israel allowed foreigners to flee the Gaza Strip as it prepared for a ground offensive. At least 430 Palestinians were killed in a week of airstrikes. *News, page 3*

Chancellor on brink of second bailout for banks

Billions may be needed as lending squeeze tightens

Francis Elliott *Osaka Finance Editor*
Gary Duncan *Economics Editor*

Alastair Darling has been forced to consider a second bailout for banks as the lending drought worsens.

The Chancellor will decide within weeks whether to pump billions more into the economy as evidence mounts that the £200bn part-nationalisation last year has failed to keep credit flowing. Options include cash injections, offering banks cheaper state guarantees to raise money privately or helping up "zombie" banks.

The Bank of England revealed yesterday that, despite intense pressure, the banks refused lending in the final quarter of last year and plus even tighter restrictions in the coming months. Its findings will alarm the Treasury.

The Bank is expected to take yet more aggressive action this week by cutting the base rate from its current level of 5 per cent. Doing so would reduce the cost of borrowing but have little effect on the availability of loans.

Whitaker warned that that ministers planned to "pump the banks on the back" but accepted that they need more help to restore lending levels. Famously, the Treasury plans to back

on state-backed guarantees to encourage private finance, but a number of institutions are on the table, including further injections of taxpayer cash.

Under one option, a "bad bank" would be created to dispose of bad debts. The Treasury would take bad loans off the hands of troubled banks, perhaps swapping them for government bonds. The toxic assets, blamed for poisoning the financial system, would be parked in a state vehicle or "bad bank" that would manage them and attempt to dispose of them while "detoxifying" the mainstream banking system.

The idea would mirror the initial proposal by Henry Paulson, the US Treasury Secretary, to underpin the American banking system by buying

debts. The Treasury would take bad loans off the hands of troubled banks, perhaps swapping them for government bonds. The toxic assets, blamed for poisoning the financial system, would be parked in a state vehicle or "bad bank" that would manage them and attempt to dispose of them while "detoxifying" the mainstream banking system.

The idea would mirror the initial proposal by Henry Paulson, the US Treasury Secretary, to underpin the American banking system by buying

Michael Sheen Frost, Nixon and me
Magazine

Working mums So that's how she does it
Body&Soul

Detox in style The best spas on the planet
Travel

Salmon Rushdie I Won't Marry Again
Pages 22, 13

Giant Killing? Guide to the FA Cup Third Round
Sport

[illegible]

- <https://bitcoinstrings.com>
- <https://btc1.trezor.io/tx/057954bb28527ff9c7701c6fd2b7f770163718ded09745da56cc95e7606afe99>

1GoDxxMEvikCb7JLAuqGWNeh75E48AgN6

0.004 321 BTC →

1BibLEafdYxCvioFhuemkbiY9r6TT5iNBn

0.004 321 BTC →

OP_RETURN (Do not be overcome by evil, but overcome evil with good - Romans 12:21)

0 BTC x

- **Full**
 - state machine over replicated shared state (i.e., blockchain)
 - set of unspent coins
 - apply operations to modify it over time (i.e., account transactions)
 - receive, validate, and store all blocks (~ 518 GB)
- **Simple Payment Verification (SPV)**
 - copy of the block headers of the longest chain
 - spv client get the Merkle branch linking the transaction to its block
 - linking the transaction to a place in the active chain demonstrates that a network node has accepted it
- **Miners**
 - are full nodes running the consensus protocol
 - propagate new blocks

- A new block is accepted **iff**
 - $\text{size} \leq \text{MAX_BLOCK_SIZE}$ (very controversial topic)
 - $\text{hash of its header} < \text{current target}$
 - the previous block's hash is correct
 - transactions form a hash tree whose root is in the header
 - all its transactions are correct
 - the timestamps does not deviate from the network time
- https://www.reddit.com/r/Bitcoin/comments/449f57/the_circuit_breaker_and_satoshi_or_why_the_one/

UTXO MODEL

- Transaction
 - Sender (coins to be spent)
 - Receiver (how to spend them later)
 - Fee (incentivize miners)
- One or more transaction per block
- Transactions are identified by their hashes
 - TXID = H(transaction)

Transaction

b4b56ad726d68d1c86eac2e0bcfec32a8ea3664ac96cdc11e5168e6be0e62e4a

Summary

Mined Time	Sat, 07 Jul 2018 22:49:51 UTC
In Block	0000000000000000000917907b6a0218119fde6dbb88d7a4a1c37f50e281e358
In Block Height	530951
Total Input	0.06 BTC
Total Output	0.0598 BTC
Fees	0.0002 BTC

Details

b4b56ad726d68d1c86eac2e0bcfec32a8ea3664ac96cdc11e5168e6be0e62e4a

mined Sat, 07 Jul 2018 22:49:51 UTC

← 12runTC15Zi5TVWBGrejwYdsV7y4hc5Kh

0.06 BTC

1QJehzp7B4xX3CxvcePKV3zmHmXP5fiPL2

0.05334068 BTC →

1BVvxMwHPmXrP6GbPVD2qP1ABeXZzFQjbR

0.00645932 BTC →

Fee: 0.0002 BTC

193275 Confirmations

0.0598 BTC

- Each transaction spends the coins previously received in one or more earlier transactions
 - the input of one transaction is the output of a previous transaction
- A single transaction can create multiple outputs
 - each output can only be used as an input once
- **Transaction validity**
 - must only use UTXOs as inputs
 - value of its outputs $\leq$ inputs
 - the difference (inputs - outputs) can be claimed as a fee by the miner who creates the block containing this transaction

- Fees
 - Price per byte
 - Prioritize transactions
 - Change outputs
 - https://en.bitcoin.it/wiki/Miner_fees

Setting	Default Value (units)
txconfirmtarget	2 (blocks)
paytxfee	0 (BTC/kB)
mintxfee	0.00001 (BTC/kB)
limitfreerelay	15 (thousand bytes per minute)
minrelaytxfee	0.00001 (BTC/kB)
blockmaxsize	750000 (bytes)
blockminsize	0 (bytes)
blockprioritysize	0 (bytes)

- No change output specified
<https://btc1.trezor.io/tx/258478e8b7a3b78301661e78b4f93a792af878b545442498065ab272eaacf035>

258478e8b7a3b78301661e78b4f93a792af878b545442498065ab272eaacf035

mined Tue, 17 Sep 2013 21:23:26 UTC

→ 1LtjWsKsrr2RweDLAmv75oGL7tjVF4wx7W

80.98 BTC

➤

1CfsAiYaVfk12dnZpZALcRSP9jjWDk26FX

0.01252199 BTC →

→ 1CfsAiYaVfk12dnZpZALcRSP9jjWDk26FX

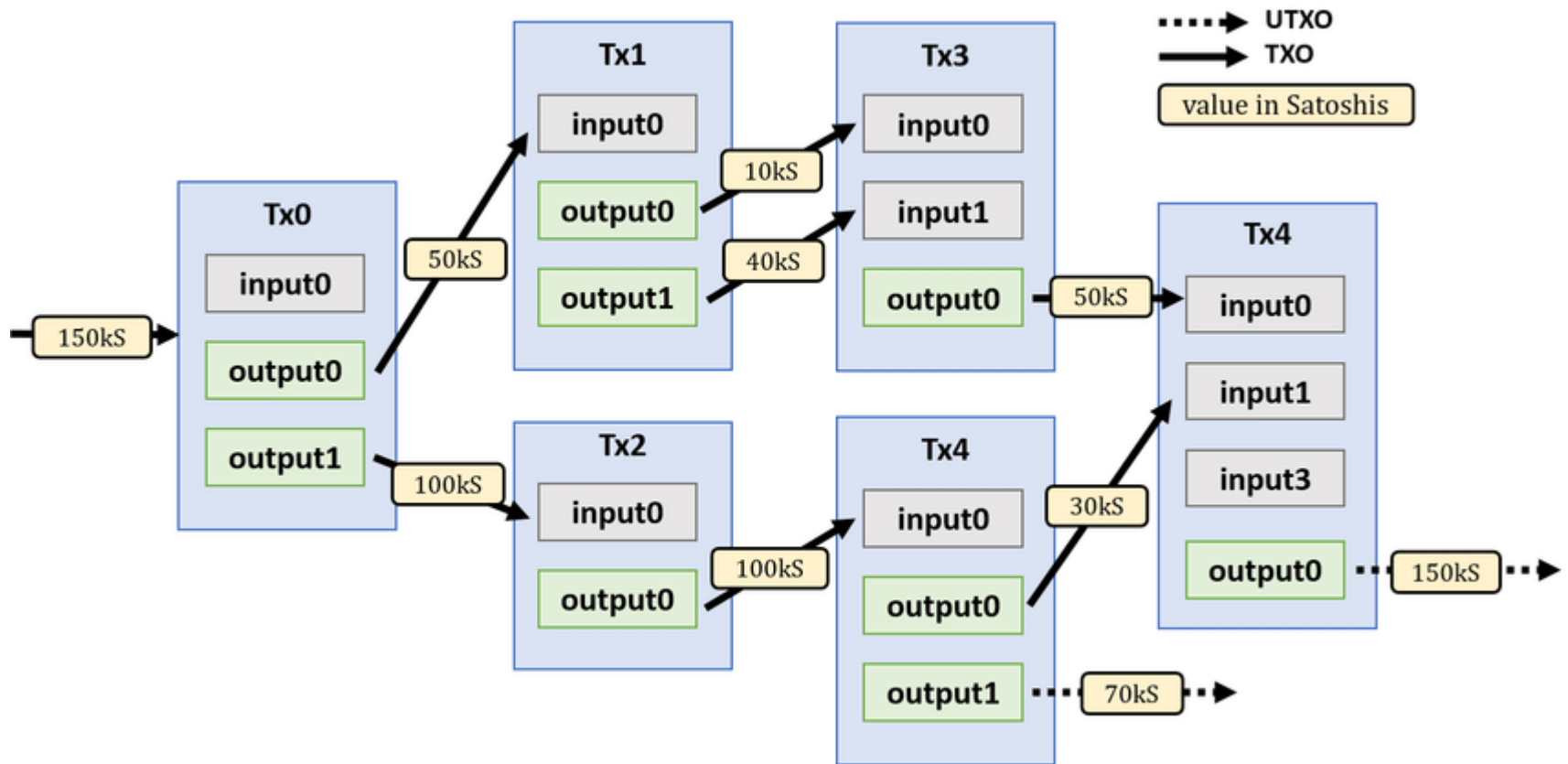
0.01252199 BTC

Fee: 80.98 BTC

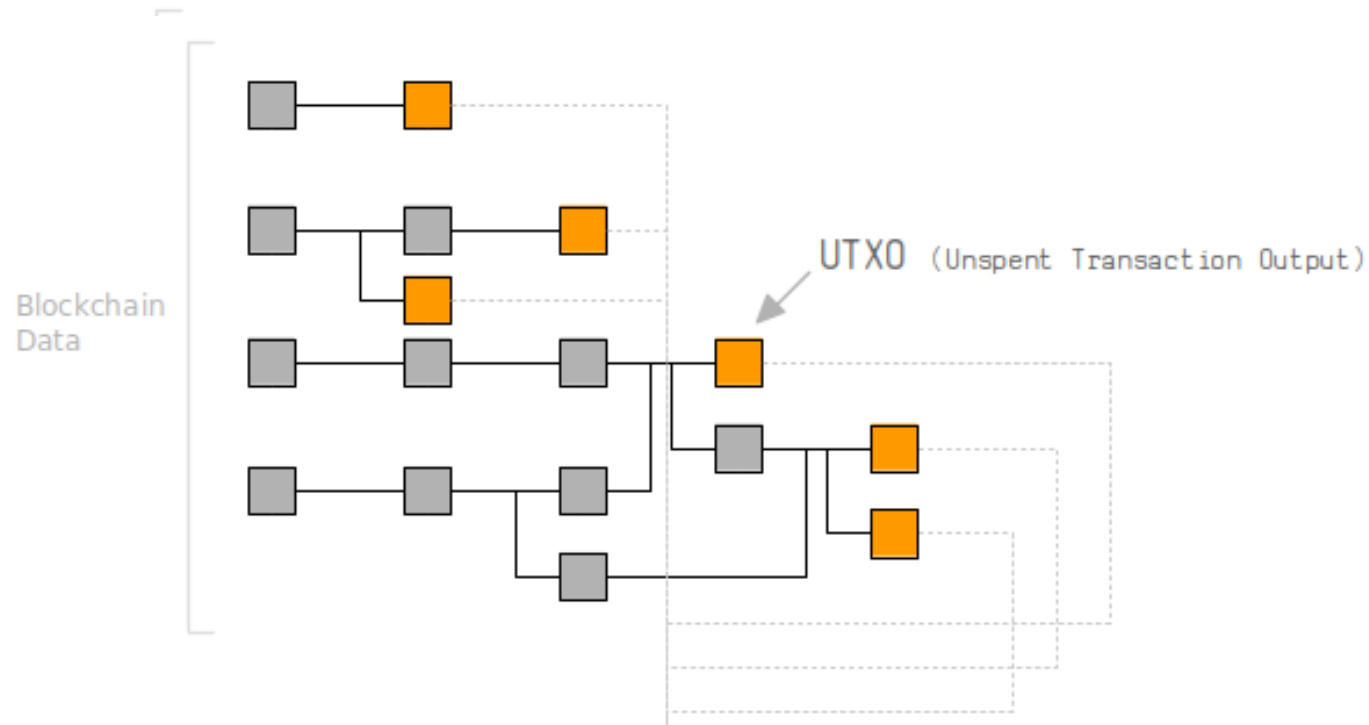
465686 Confirmations

0.01252199 BTC

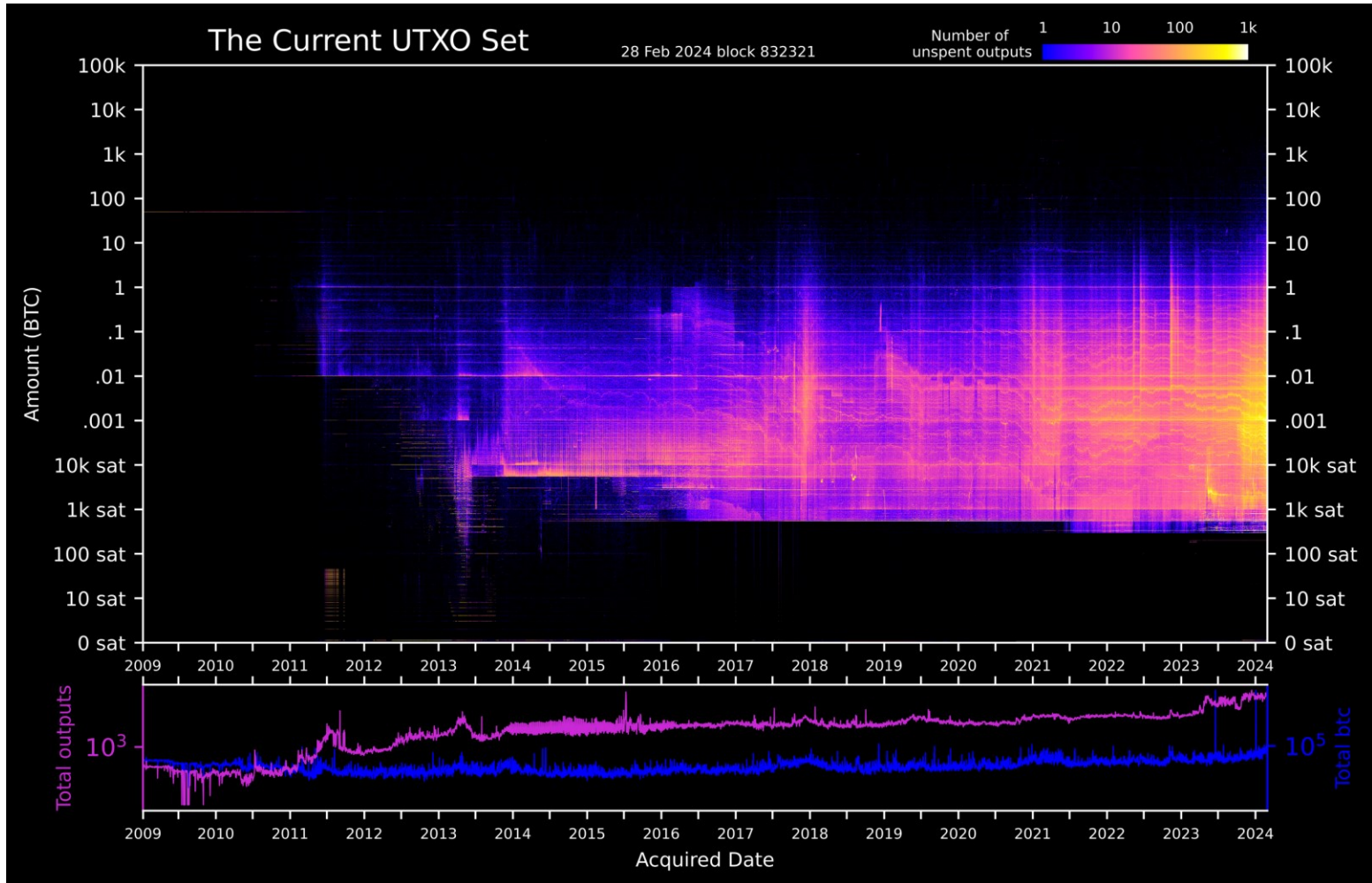




- <https://github.com/in3rsha/bitcoin-utxo-dump>
- <https://btc1.trezor.io/api/v2/utxo/3L8Ck6bm3sve1vJGKo6Ht2k167YKSKi8TZ>

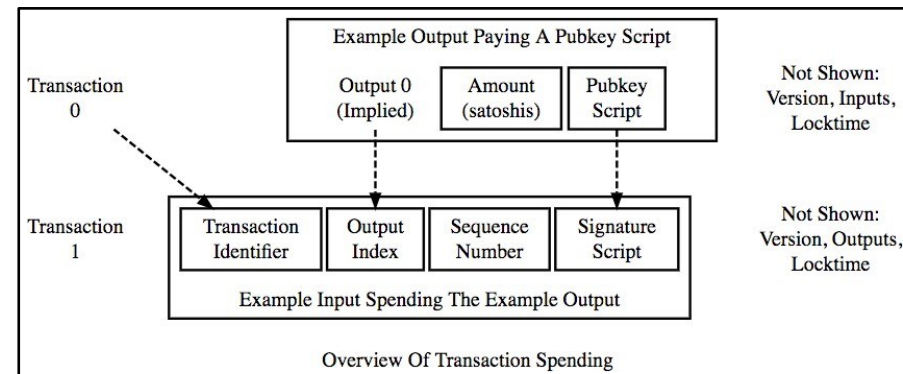
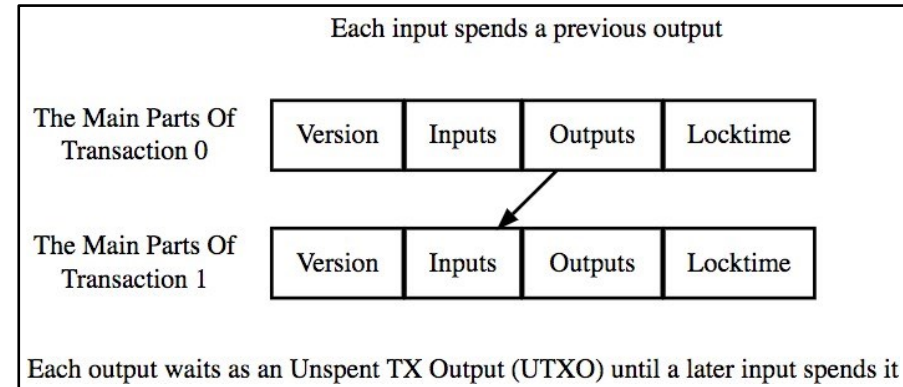


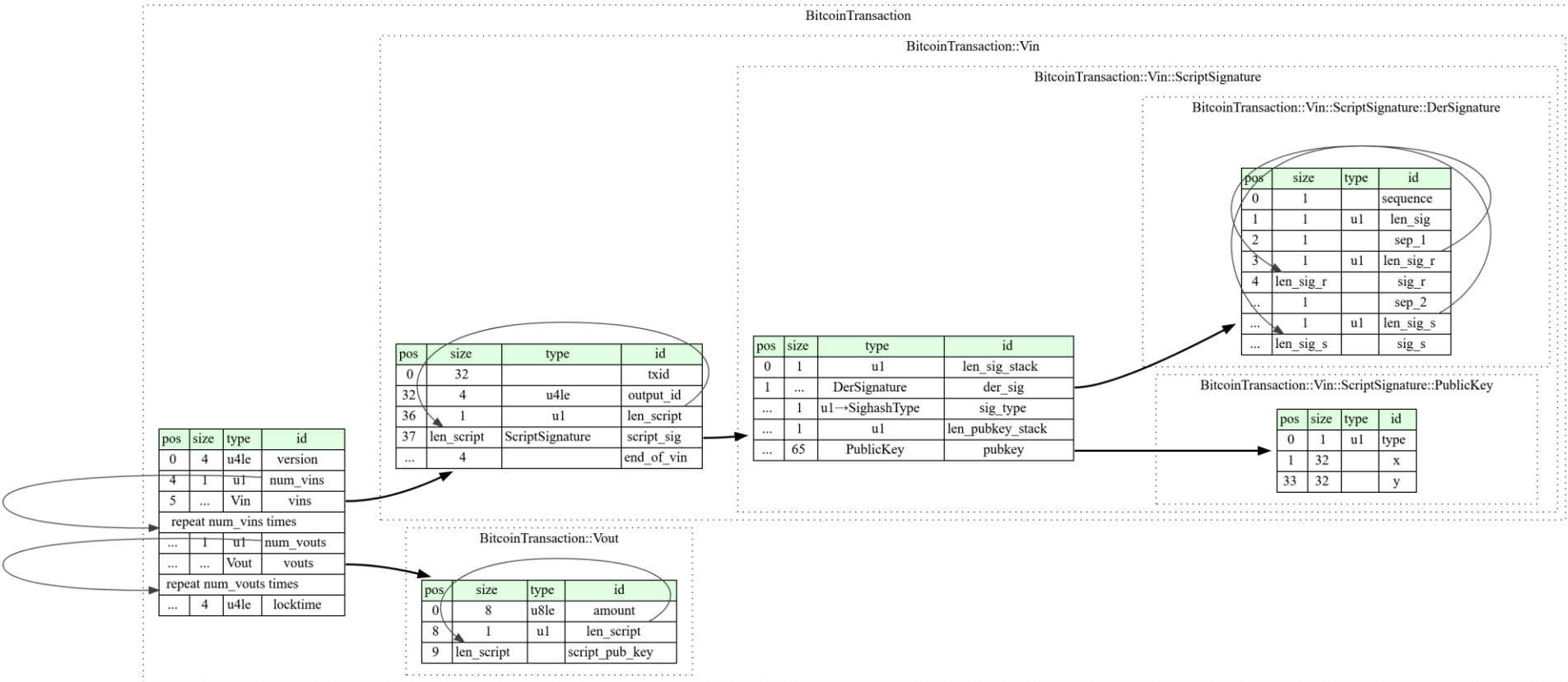
- <https://utxo.live/>



- = A transaction that uses the same input as an already broadcast transaction.
 - The attempt of duplication, deceit, or conversion, will be adjudicated when only one of the transactions is recorded in the blockchain.
- For our purposes, the earliest transaction is the one that counts, so we don't care about later attempts to double-spend.
 - The only way to confirm the absence of a transaction is to be aware of all transactions.
 - To accomplish this without a trusted party, transactions must be publicly announced, and we need a system for participants to agree on a single history of the order in which they were received.
 - The payee needs proof that at the time of each transaction, the majority of nodes agreed it was the first received.

- **Version** determines set of rules to apply
- An input uses
 - TXID
 - an output index number to identify a particular output to be spent
 - It also has a **signature script** which allows it to provide data parameters that satisfy the conditions to spend the output
- **Locktime**
 - The earliest time a transaction can be added to the blockchain
 - New non-locked transaction can invalidate it (use the same outputs)





- https://formats.kaitai.io/bitcoin_transaction/bitcoin_transaction.svg

<https://btc1.trezor.io/tx/b4b56ad726d68d1c86eac2e0bfc32a8ea3664ac96cdc11e5168e6be0e62e4a>

Details

b4b56ad726d68d1c86eac2e0bfc32a8ea3664ac96cdc11e5168e6be0e62e4a

mined Sat, 07 Jul 2018 22:49:51 UTC

→ 12runTC15ZI5TVWBGrejwYdsV7y4hc5Kh

0.06 BTC

1QJehzp7B4xX3CxcvePKV3zmHmXP5fiPL2

0.05334068 BTC

1BVvxMwHPmXrP6GbPVD2qP1ABeXZzFQjbR

0.00645932 BTC

Fee: 0.0002 BTC

193273 Confirmations

0.0598 BTC

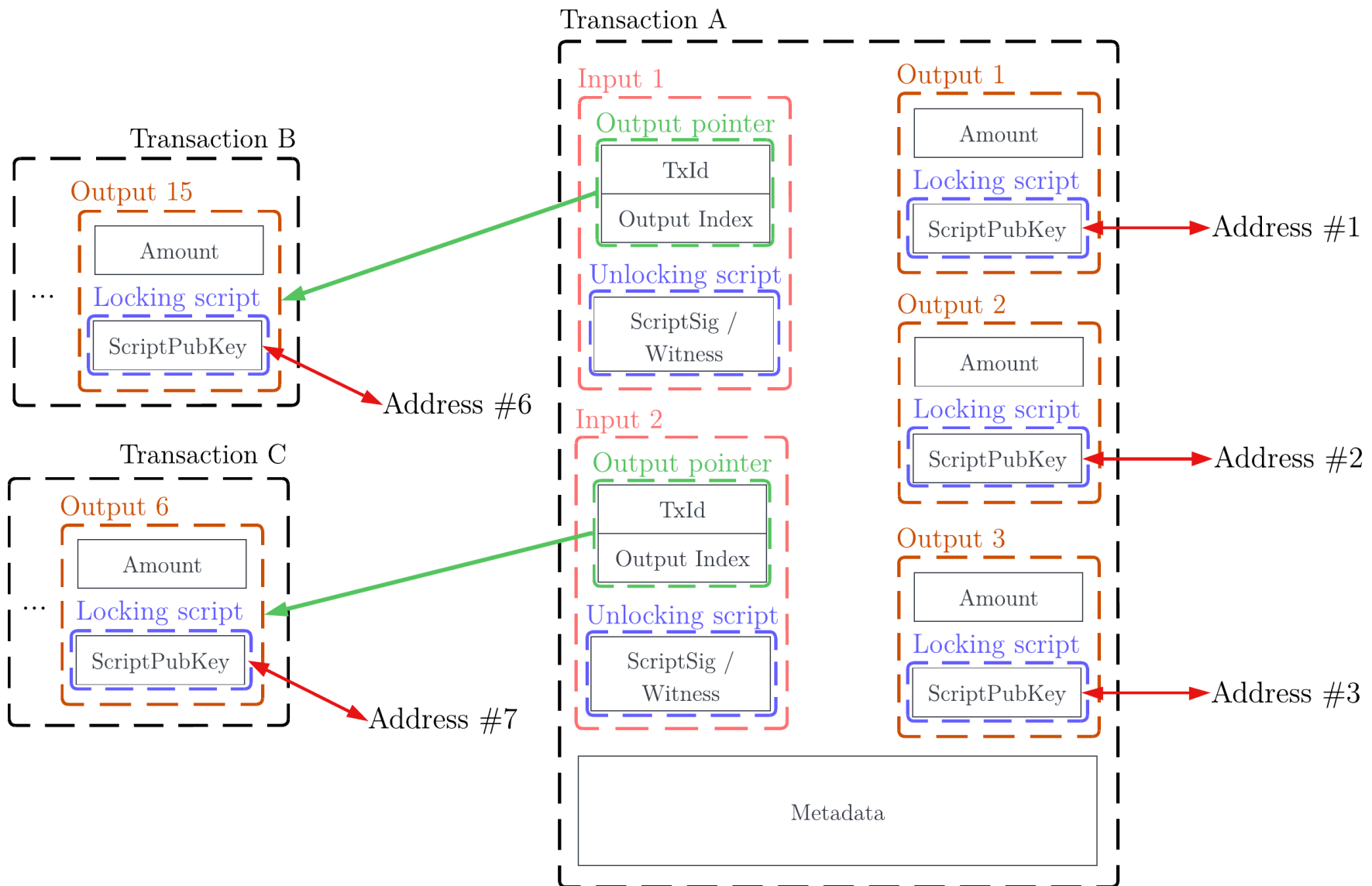
Raw Transaction

```
{
  "txid": "b4b56ad726d68d1c86eac2e0bfc32a8ea3664ac96cdc11e5168e6be0e62e4a",
  "hash": "b4b56ad726d68d1c86eac2e0bfc32a8ea3664ac96cdc11e5168e6be0e62e4a",
  "version": 2,
  "size": 225,
  "vsize": 225,
  "weight": 900,
  "locktime": 530950,
  "vin": [
    {
      "txid": "48021b6105ecf3d0421537eb460126f894185507caf06b990f5d010fab3bb5",
      "vout": 21,
      "scriptSig": {
        "asm": "3044022019add287b4b41bf17013e264f3eede065ce6878a306910f2ba5d3a55a747666022008eac2d7eff31b73ee9495544b26b7f1a4d28da8ef1c7e6ac95dc97dab3e2c2d[ALL] 033be0b2a433433e129391734f854e923f5",
        "hex": "473044022019add287b4b41bf17013e264f3eede065ce6878a306910f2ba5d3a55a747666022008eac2d7eff31b73ee9495544b26b7f1a4d28da8ef1c7e6ac95dc97dab3e2c2d0121033be0b2a433433e129391734f854e923f5",
        "sequence": 4294967294
      }
    }
  ],
  "vout": [
    {
      "value": 0.05334068,
      "n": 0,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 ffa170c22bb78b6454bb298f29a6723b617b6d65 OP_EQUALVERIFY OP_CHECKSIG",
        "hex": "76a914ffa170c22bb78b6454bb298f29a6723b617b6d6588ac",
        "address": "1QJehzp7B4xX3CxcvePKV3zmHmXP5fiPL2",
        "type": "pubkeyhash"
      }
    },
    {
      "value": 0.00645932,
      "n": 1,
      "scriptPubKey": {
        "asm": "OP_DUP OP_HASH160 7329f1975c00ae424b23af309cd6c49c789597ce OP_EQUALVERIFY OP_CHECKSIG",
        "hex": "76a9147329f1975c00ae424b23af309cd6c49c789597ce88ac",
        "address": "1BVvxMwHPmXrP6GbPVD2qP1ABeXZzFQjbR",
        "type": "pubkeyhash"
      }
    }
  ],
  "blockhash": "0200000001b5bbcfa18d0f590b906fcca07551894f8260146eb371542d0f3ec05611b0248150000006a473044022019add287b4b41bf17013e264f3eede065ce6878a306910f2ba5d3a55a747666022008eac2d7eff31b73ee94955441",
  "confirmations": 193273,
  "time": 1531003791,
  "blocktime": 1531003791
}
```

BITCOIN SCRIPT

- How to send/receive Bitcoin?
 - How to lock BTC?
 - How to unlock BTC?
- Handled by Script
- Sender pays to a "fingerprint" of a secret
- Recipient uses the secret itself to claim the output
- Secret usually involves asymmetric cryptography
 - ECDSA or Schnorr
- **Public key**-based locking/unlocking methods
- **Script hash**-based locking/unlocking methods
- Combination of both - **Taproot**

- Transaction consists of inputs and outputs
- *ScriptPubKey* = script that **LOCKS** the output
 - Provided with the **output**
- *ScriptSig* = script that **UNLOCKS** the output
 - Provided with the **input**
- How is unlocking verified? => **Script**
 - Mini programming language
 - Full script = (**unlocking + locking**) - in that order!
 - Top of stack is **TRUE** => Output is unlocked!



- Every transaction output is associated with script, which specifies condition upon which it may be spent
 - *transfer BTC to address*
 - *unlock only during Mondays*
 - *pay BTC when I am dead*
 - precursor for smart contracts
- Features
 - Similar to Forth
 - Turing incomplete (no loops nor recursion)
 - Stack-like processing
 - Data and a set of instructions
 - https://www.researchgate.net/publication/335146032_An_Analysis_of_Non-standard_Transactions

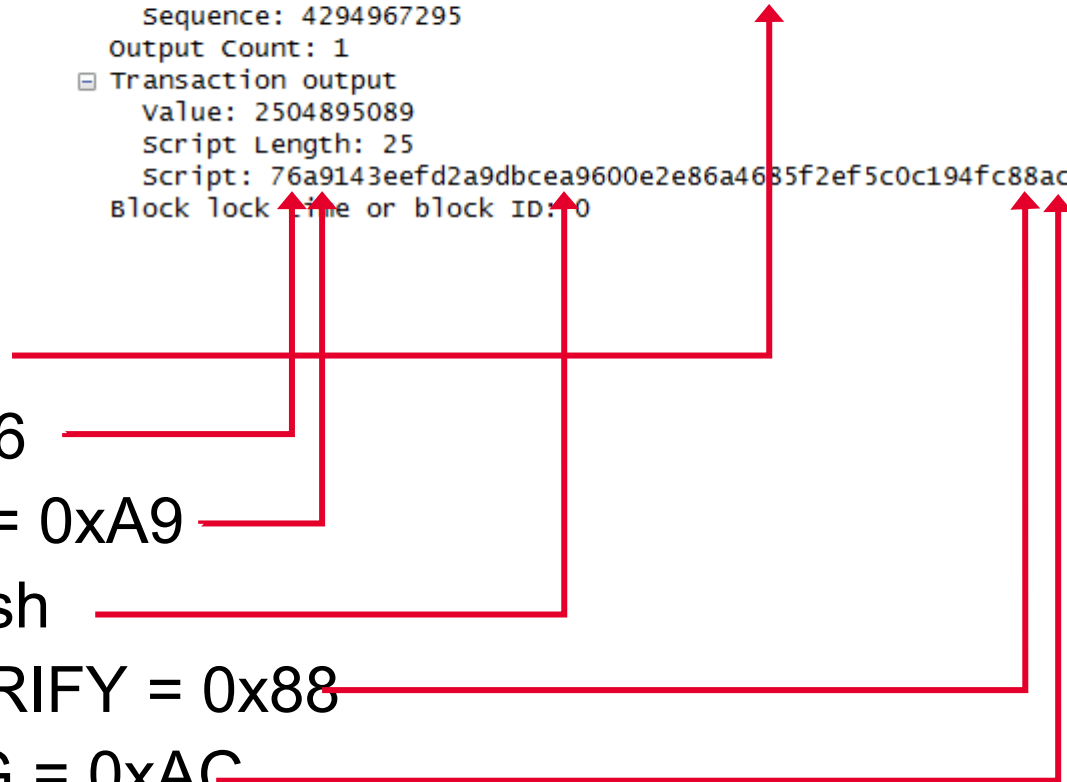
- Version
- In-counter
- List of inputs
- Out-counter
- List of outputs
- Lock time

```

Tx message [ 1 ]
  Transaction version: 1
  Input Count: 1
  Transaction input
    Previous output
      Hash: 0000000000000000000000000000000000000000000000000000000...
      Index: 4294967295
      Script Length: 64
      Signature script: 03c84a050762706c54fd34effabe6d6d36d98a4155d6c590...
      Sequence: 4294967295
    Output Count: 1
  Transaction output
    Value: 2504895089
    Script Length: 25
    Script: 76a9143eefd2a9dbcea9600e2e86a4685f2ef5c0c194fc88ac
    Block lock time or block ID: 0
  
```

• Bitcoin script

- sig, pubKey
- OP_DUP = 0x76
- OP_HASH160 = 0xA9
- 20B pubKeyHash
- OP_EQUALVERIFY = 0x88
- OP_CHECKSIG = 0xAC



```
enum opcodetype
{
    // push value
    OP_0 = 0x00,
    OP_FALSE = OP_0,
    OP_PUSHDATA1 = 0x4c,
    OP_PUSHDATA2 = 0x4d,
    OP_PUSHDATA4 = 0x4e,
    OP_1NEGATE = 0x4f,
    OP_RESERVED = 0x50,
    OP_1 = 0x51,
    OP_TRUE=OP_1,
    OP_2 = 0x52,
    OP_3 = 0x53,
    OP_4 = 0x54,
    OP_5 = 0x55,
    OP_6 = 0x56,
    OP_7 = 0x57,
    OP_8 = 0x58,
    OP_9 = 0x59,
    OP_10 = 0x5a,
    OP_11 = 0x5b,
    OP_12 = 0x5c,
    OP_13 = 0x5d,
    OP_14 = 0x5e,
    OP_15 = 0x5f,
    OP_16 = 0x60,

    // control
    OP_NOP = 0x61,
    OP_VER = 0x62,
    OP_IF = 0x63,
    OP_NOTIF = 0x64,
    OP_VERIF = 0x65,
    OP_VERNOTIF = 0x66,
    OP_ELSE = 0x67,
    OP_ENDIF = 0x68,
    OP_VERIFY = 0x69,
    OP_RETURN = 0x6a,

    // stack ops
    OP_TOALTSTACK = 0x6b,
    OP_FROMALTSTACK = 0x6c,
    OP_2DROP = 0x6d,
    OP_2DUP = 0x6e,
    OP_3DUP = 0x6f,
    OP_2OVER = 0x70,
    OP_2ROT = 0x71,
    OP_2SWAP = 0x72,
    OP_IFDUP = 0x73,
    OP_DEPTH = 0x74,
    OP_DROP = 0x75,
    OP_DUP = 0x76,
    OP_NIP = 0x77,
    OP_OVER = 0x78,
    OP_PICK = 0x79,
    OP_ROLL = 0x7a,
    OP_ROT = 0x7b,
    OP_SWAP = 0x7c,
    OP_TUCK = 0x7d,

    // splice ops
    OP_CAT = 0x7e,
    OP_SUBSTR = 0x7f,
    OP_LEFT = 0x80,
    OP_RIGHT = 0x81,
    OP_SIZE = 0x82,

    // bit logic
    OP_INVERT = 0x83,
    OP_AND = 0x84,
    OP_OR = 0x85,
    OP_XOR = 0x86,
    OP_EQUAL = 0x87,
    OP_EQUALVERIFY = 0x88,
    OP_RESERVED1 = 0x89,
    OP_RESERVED2 = 0x8a,

    // numeric
    OP_1ADD = 0x8b,
    OP_1SUB = 0x8c,
    OP_2MUL = 0x8d,
    OP_2DIV = 0x8e,
    OP_NEGATE = 0x8f,
    OP_ABS = 0x90,
    OP_NOT = 0x91,
    OP_0NOTEQUAL = 0x92,

    OP_ADD = 0x93,
    OP_SUB = 0x94,
    OP_MUL = 0x95,
    OP_DIV = 0x96,
    OP_MOD = 0x97,
    OP_LSHIFT = 0x98,
    OP_RSHIFT = 0x99,

    OP_BOOLAND = 0x9a,
    OP_BOOLOR = 0x9b,
    OP_NUMEQUAL = 0x9c,
    OP_NUMEQUALVERIFY = 0x9d,
    OP_NUMNOTEQUAL = 0x9e,
    OP_LESSTHAN = 0x9f,
    OP_GREATERTHAN = 0xa0,
    OP_LESSTHANOREQUAL = 0xa1,
    OP_GREATERTHANOREQUAL = 0xa2,
    OP_MIN = 0xa3,
    OP_MAX = 0xa4,

    OP_WITHIN = 0xa5,

    // crypto
    OP_RIPEMD160 = 0xa6,
    OP_SHA1 = 0xa7,
    OP_SHA256 = 0xa8,
    OP_HASH160 = 0xa9,
    OP_HASH256 = 0xaa,
    OP_CODESEPARATOR = 0xab,

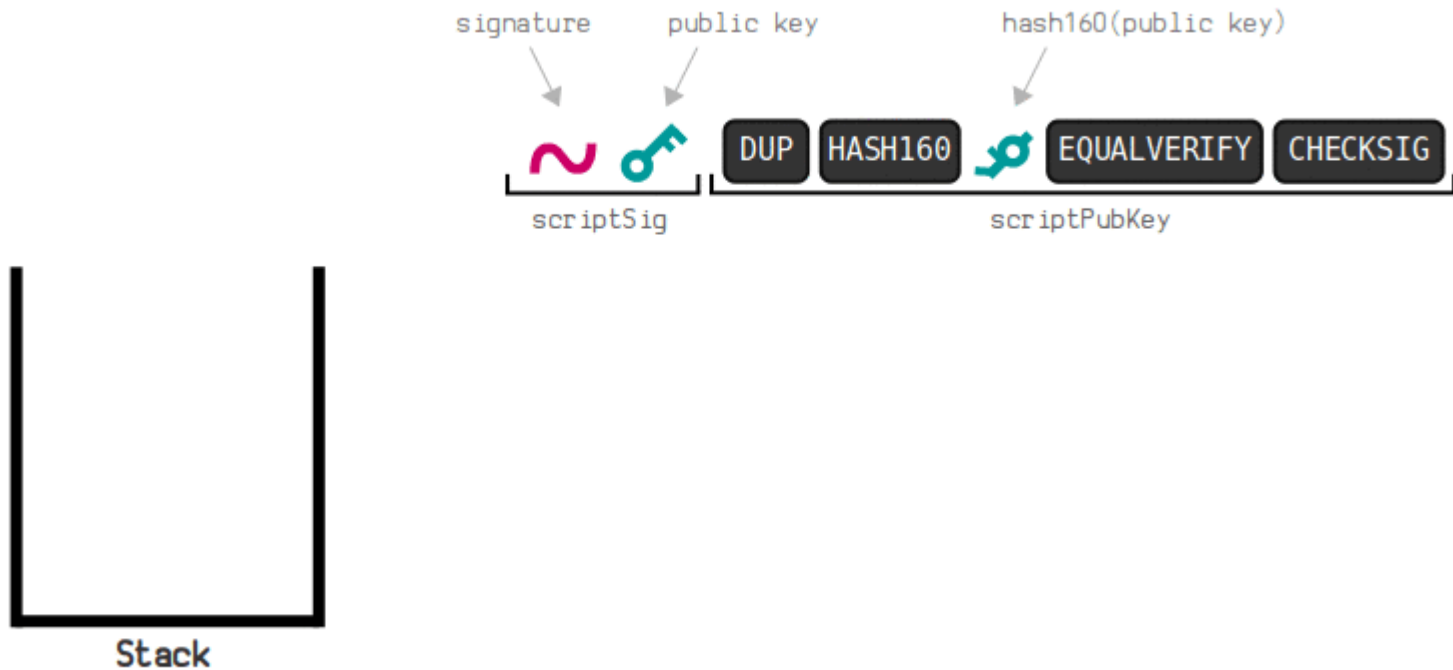
    OP_CHECKSIG = 0xac,
    OP_CHECKSIGVERIFY = 0xad,
    OP_CHECKMULTISIG = 0xae,
    OP_CHECKMULTISIGVERIFY = 0xaf,

    // expansion
    OP_NOP1 = 0xb0,
    OP_NOP2 = 0xb1,
    OP_NOP3 = 0xb2,
    OP_NOP4 = 0xb3,
    OP_NOP5 = 0xb4,
    OP_NOP6 = 0xb5,
    OP_NOP7 = 0xb6,
    OP_NOP8 = 0xb7,
    OP_NOP9 = 0xb8,
    OP_NOP10 = 0xb9,

    // template matching params
    OP_SMALLDATA = 0xf9,
    OP_SMALLINTEGER = 0xfa,
    OP_PUBKEYS = 0xfb,
    OP_PUBKEYHASH = 0xfd,
    OP_PUBKEY = 0xfe,

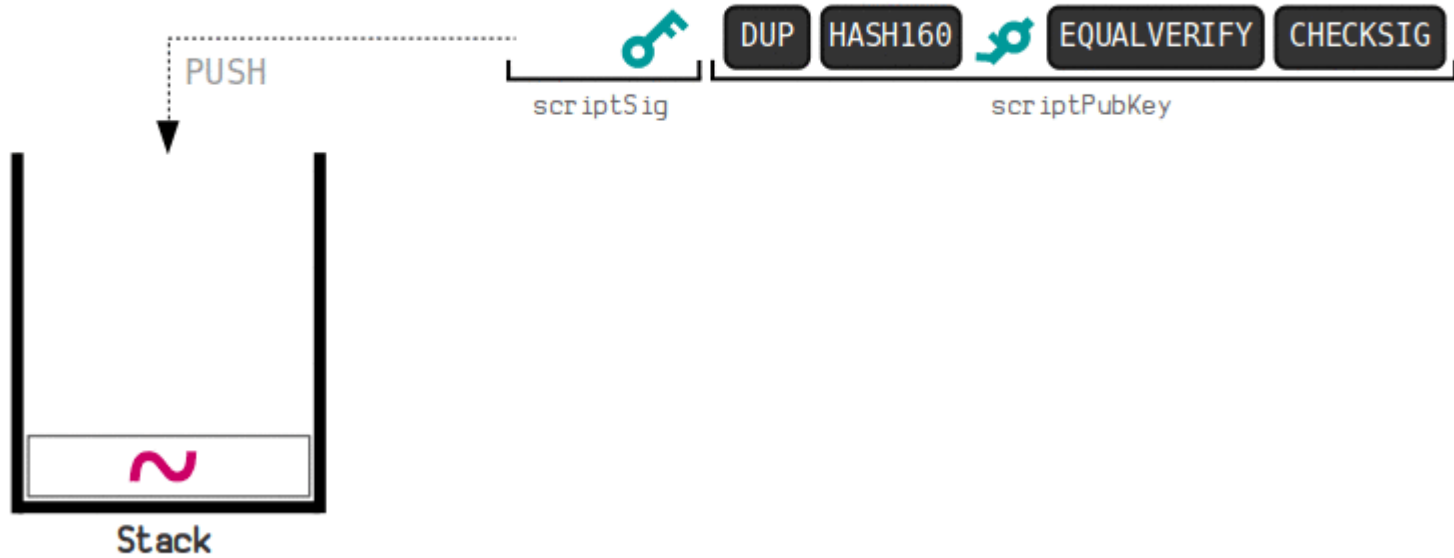
    OP_INVALIDOPCODE = 0xff,
};
```

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



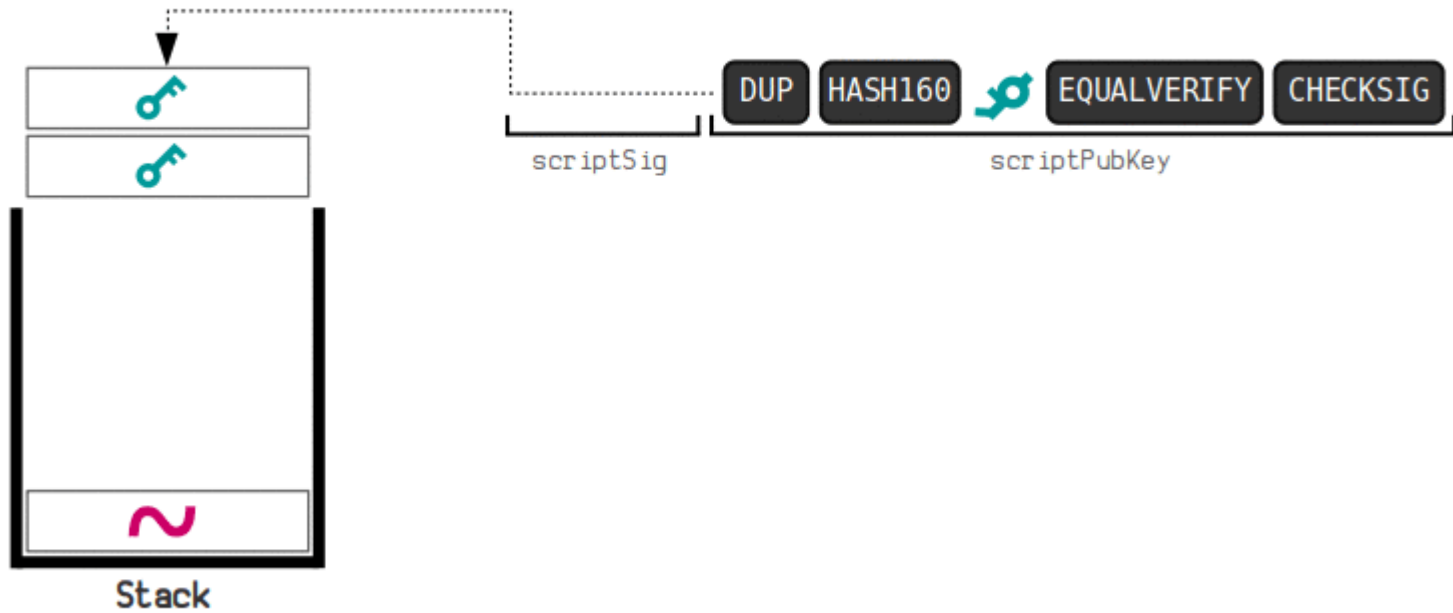
learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



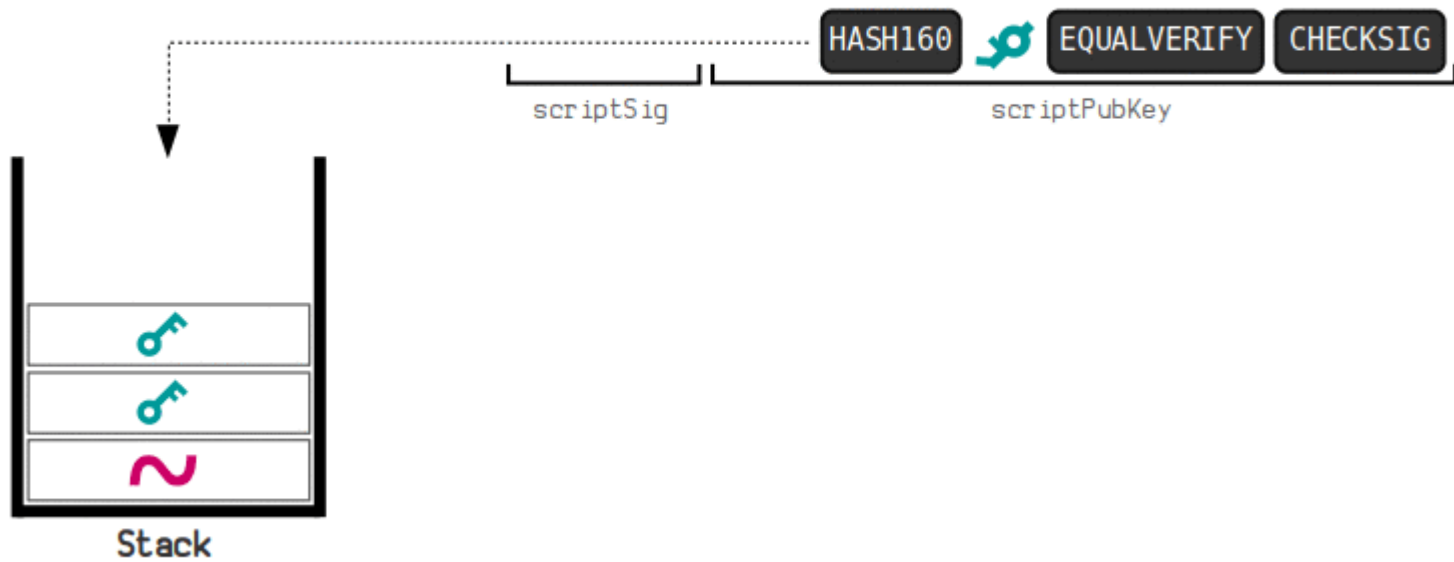
learnmeabitcoin.com

Standard Script: P2PKH



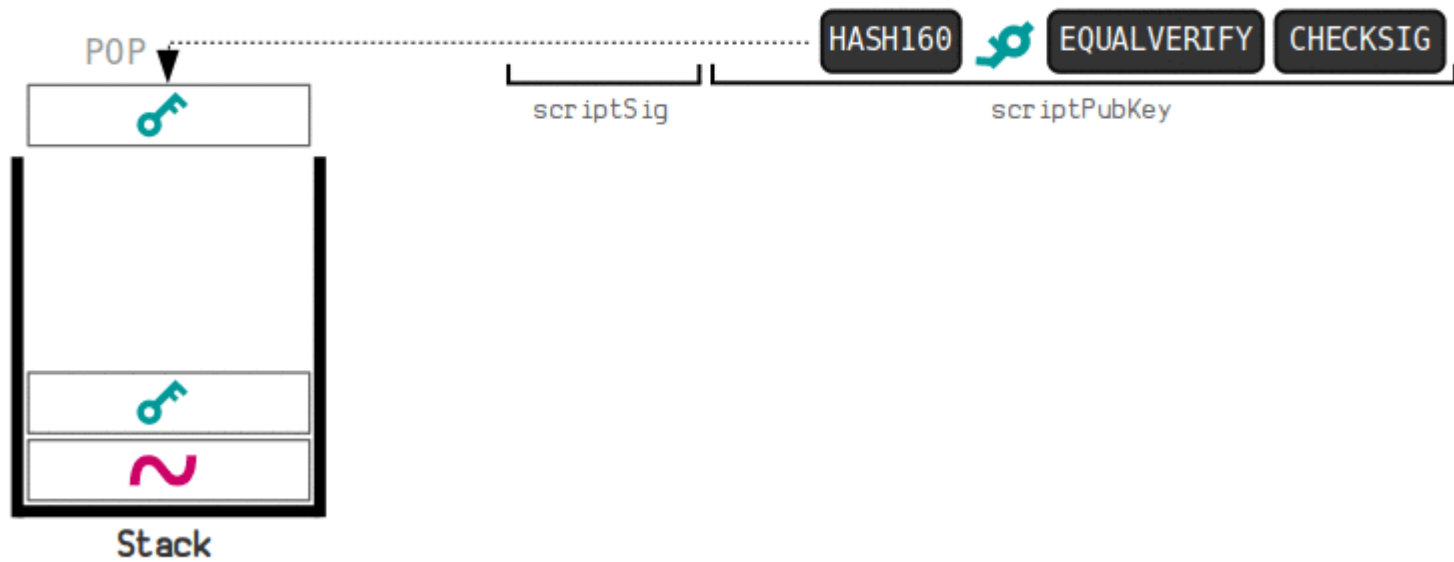
learnmeabitcoin.com

Standard Script: P2PKH



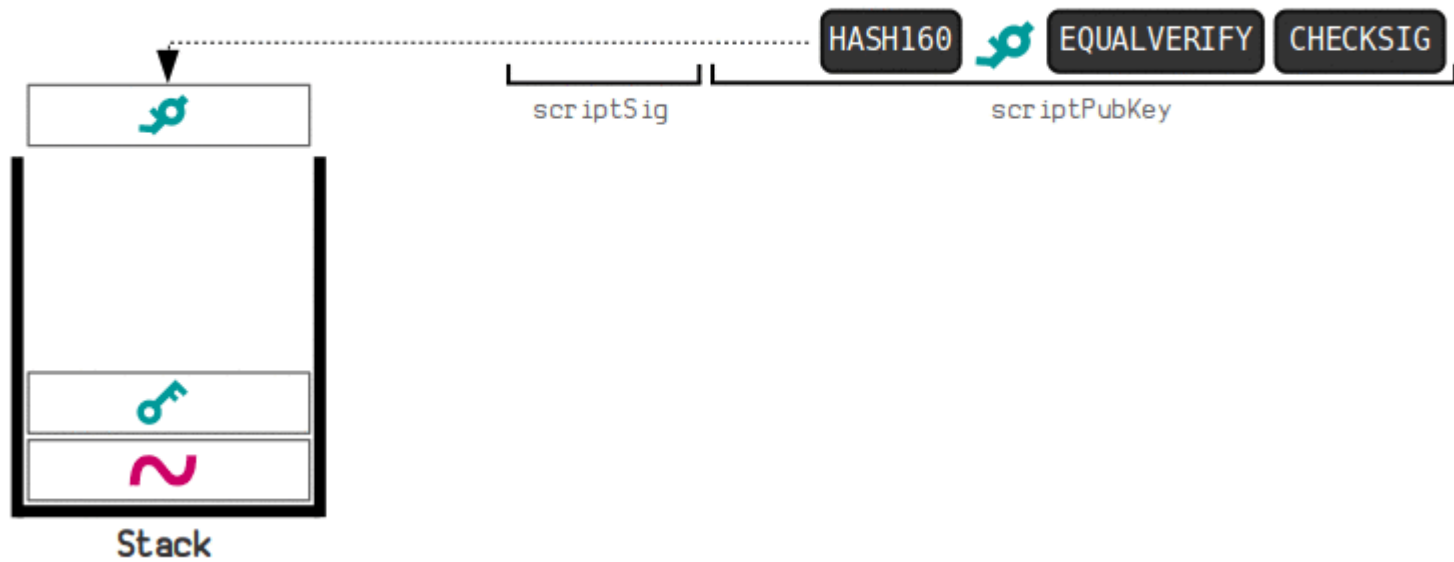
learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



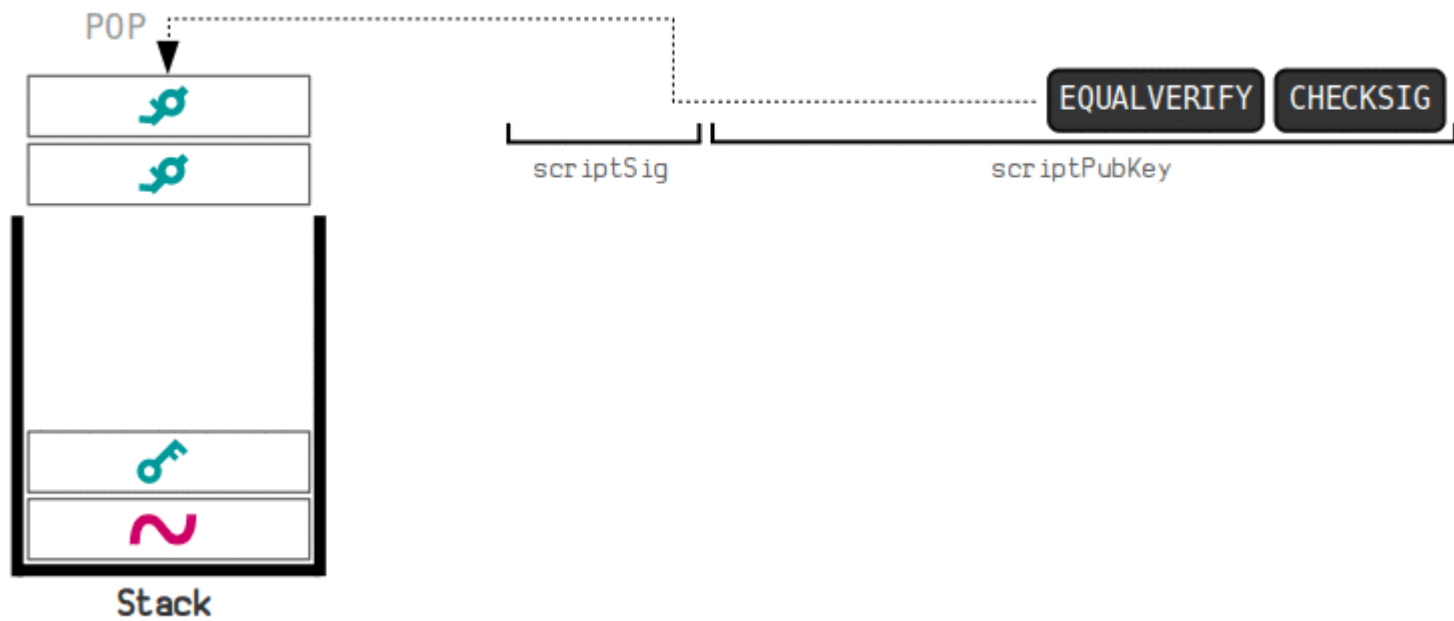
learnmeabitcoin.com

Standard Script: P2PKH



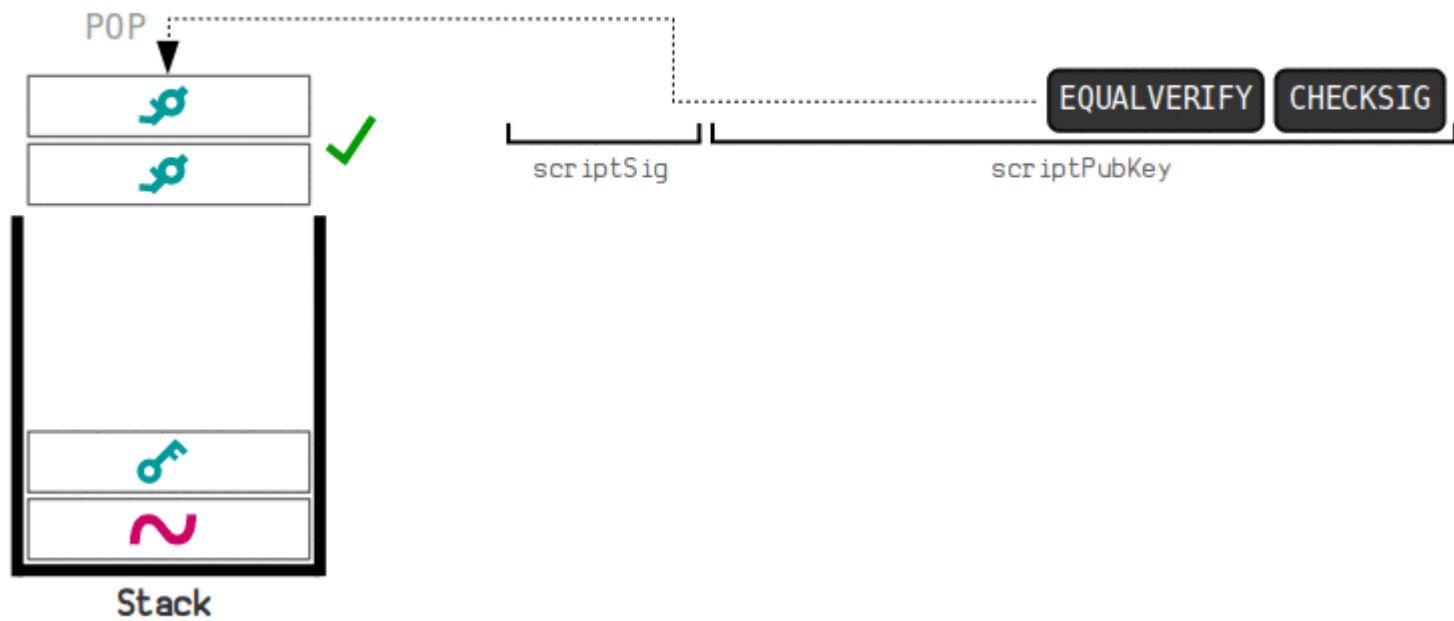
learnmeabitcoin.com

Standard Script: P2PKH



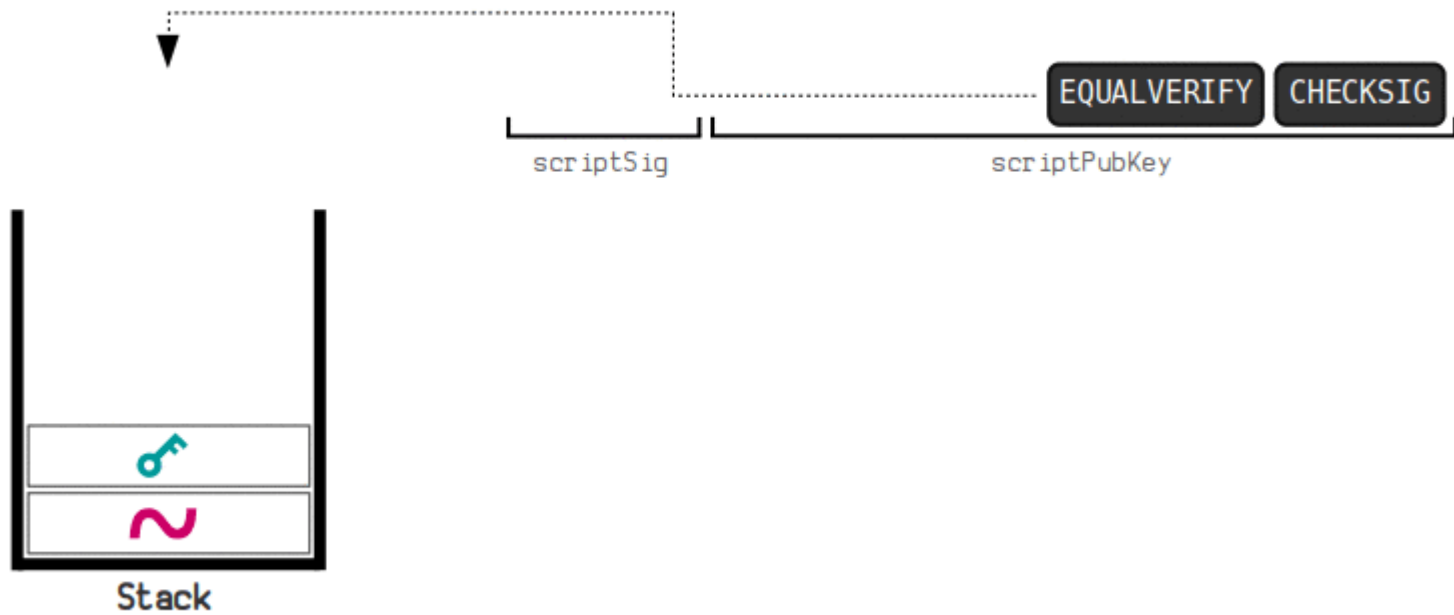
learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



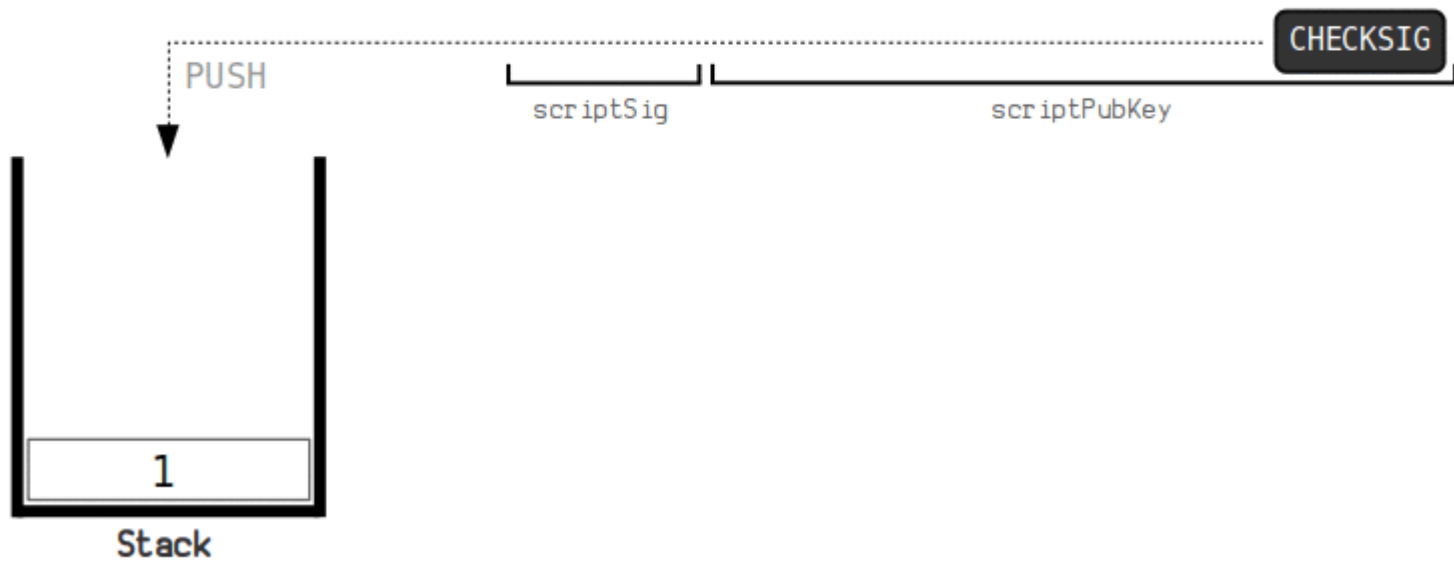
learnmeabitcoin.com

Standard Script: P2PKH



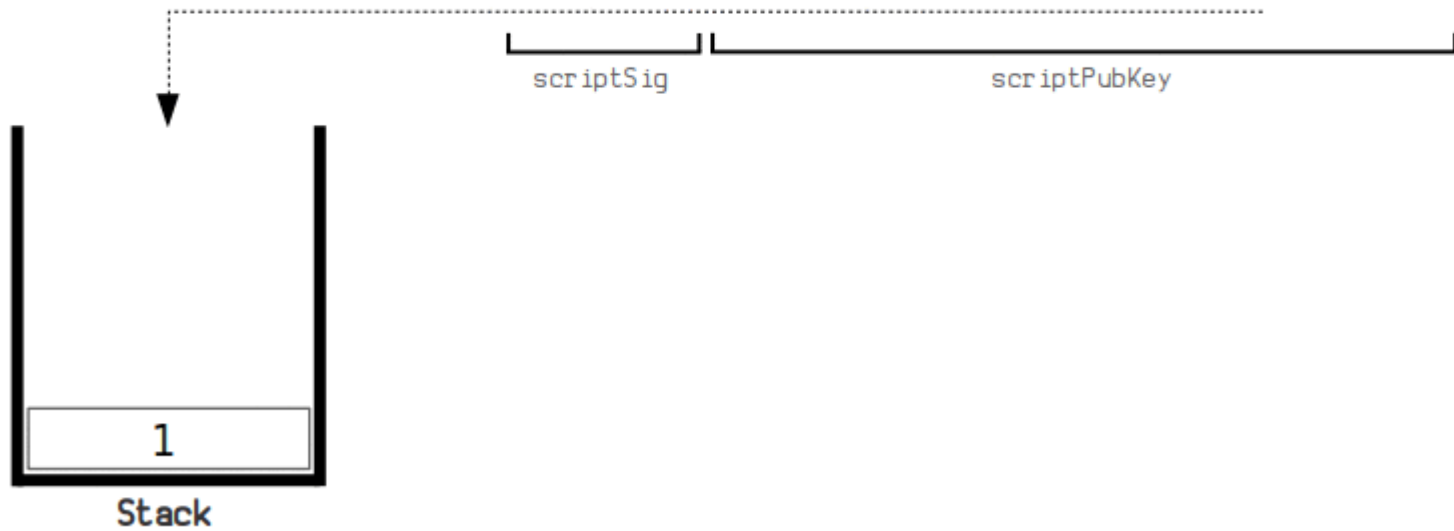
learnmeabitcoin.com

Standard Script: P2PKH



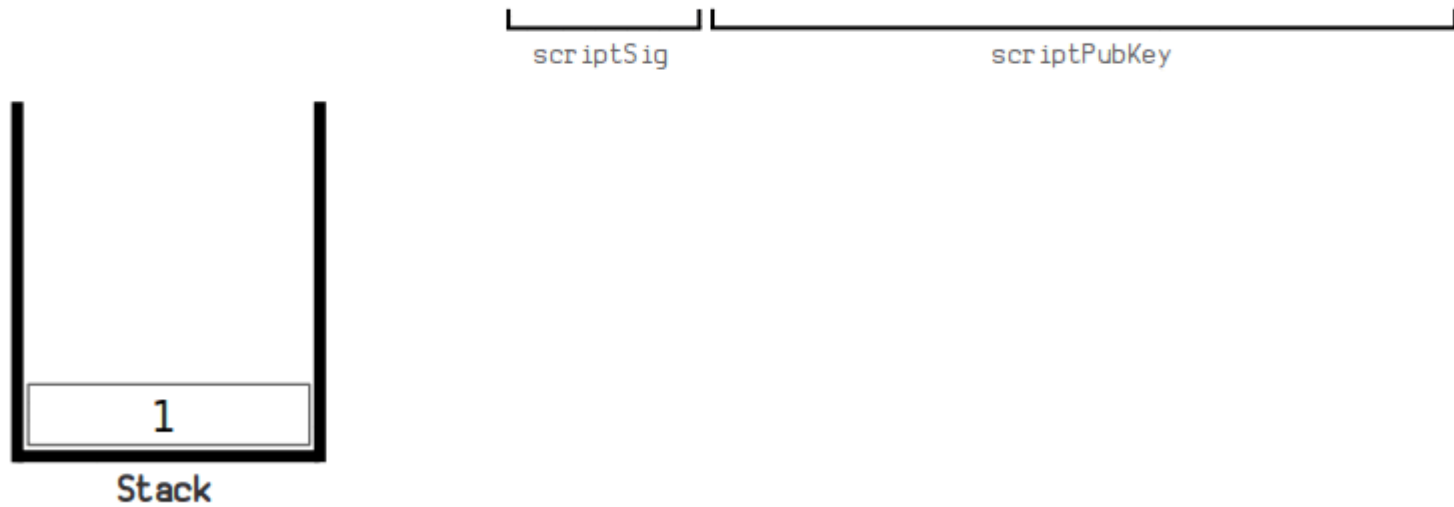
learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

Standard Script: P2PKH



learnmeabitcoin.com

- **2009**
 - Pay-to-public-key (**P2PK**)
 - Pay-to-public-key-hash (**P2PKH**) [ADDRESS]
- **2012**
 - Pay-to-multisig (**P2MS**)
 - Pay-to-script-hash (**P2SH**) [ADDRESS]
- **2017 - SEGWIT**
 - Pay-to-witness-public-key-hash (**P2WPKH**) [ADDRESS]
 - Pay-to-witness-script-hash (**P2WSH**) [ADDRESS]
 - Pay-to-script-hash wrapped segwit (**P2SH-P2WPKH, P2SH-P2WSH**) [ADDRESS]
- **2021 - TAPROOT**
 - Pay-to-taproot (**P2TR**) [ADDRESS]

PEER-TO-PEER NETWORK

- Codified by Bitcoin Foundation
 - Binary
 - https://developer.bitcoin.org/devguide/p2p_network.html
- Operates over TCP on well-known port 8333
 - 18333 testbed network
 - NAT traversal

Message	Description	Message	Description
version	When a node creates an outgoing connection, it will immediately advertise its version . The remote node will respond with its version.	verack	The verack message is sent in reply to version .
addr	Provide information on known nodes of the network.	getaddr	The getaddr sends a request to a node asking for information about known active peers to help with finding potential nodes in the network. The response to receiving this message is to transmit one or more addr .
inv	Allows a node to advertise its knowledge of one or more objects. It can be received unsolicited, or in reply to getblocks .	getdata	getdata is used to retrieve the content of a specific object, and is usually sent after receiving an inv packet, after filtering known elements.
mempool	The mempool sends a request to a node asking for information about transactions it has verified but which have not yet confirmed.	tx	tx describes a bitcoin transaction, in reply to getdata .
notfound	notfound is a response to a getdata , sent if any requested data items could not be relayed.	reject	The reject is sent when messages are rejected.
getblocks	Return an inv containing the list of blocks starting right after the last known hash in the block locator object.	block	The block is sent in response to a getdata which requests transaction information from a block hash.
getheaders	Return a headers containing the headers of blocks starting right after the last known hash in the block locator object.	headers	The headers returns block headers in response to a getheaders .
ping	The ping is sent primarily to confirm that the TCP/IP connection is still valid.	pong	The pong is sent in response to a ping message.
alert	An alert is sent between nodes to send a general notification message throughout the network.		
filterload, filteradd, filterclear, merkleblock	These messages are related to Bloom filtering of connections	checkorder, submitorder, replyorder	Deprecated

- *At the beginning, client does not know any other bitcoin peers*
- Specify them manually
 - `bitcoin-cli addnode "192.168.0.6:8333" "onetry"`
 - `curl --user myusername --data-binary '{"jsonrpc": "1.0", "id": "curltest", "method": "addnode", "params": ["192.168.0.6:8333", "onetry"]}' -H 'content-type: text/plain;' http://127.0.0.1:8332/`

```
# Use as many addnode= settings as you like to connect to specific peers
#addnode=69.164.218.197
#addnode=10.0.0.2:8333
```

- Dynamic resolution via **DNS seeds**
 - dynamic vs. static maintenance
 - <https://github.com/bitcoin/bitcoin/blob/master/src/chainparams.cpp#L121>

No.	Time	Source	Destination	Protocol	Length	Info
8238	68.807389000	192.168.159.130	192.168.159.2	DNS	80	Standard query 0x8043 seed.bitcoin.sipa.be
8241	68.883293000	192.168.159.2	192.168.159.130	DNS	528	Standard query response 0x8043 A 83.160.76.197 A 88.
8242	68.886230000	192.168.159.130	192.168.159.2	DNS	75	Standard query 0x30ff bitcoin.sipa.be
8243	69.010352000	192.168.159.2	192.168.159.130	DNS	109	Standard query response 0x30ff CNAME vps.sipa.be A 1
8244	69.013018000	192.168.159.130	192.168.159.2	DNS	79	Standard query 0x80a1 dnsseed.bluematt.me
8245	69.077661000	192.168.159.2	192.168.159.130	DNS	415	Standard query response 0x80a1 A 109.163.235.239 A 1
8246	69.080160000	192.168.159.130	192.168.159.2	DNS	71	Standard query 0xd933 bluematt.me
8247	69.132002000	192.168.159.2	192.168.159.130	DNS	119	Standard query response 0xd933
8266	74.453394000	192.168.159.130	192.168.159.2	DNS	86	Standard query 0xbb80 A dnsseed.bitcoin.dashjr.org

Frame 8241: 528 bytes on wire (4224 bits), 528 bytes captured (4224 bits) on interface 0
Ethernet II, Src: Vmware_e0:ce:52 (00:50:56:e0:ce:52), Dst: Vmware_3e:4a:25 (00:0c:29:3e:4a:25)
Internet Protocol Version 4, Src: 192.168.159.2 (192.168.159.2), Dst: 192.168.159.130 (192.168.159.130)
User Datagram Protocol, Src Port: domain (53), Dst Port: 51551 (51551)
Domain Name System (response)
[Request In: 8238]
[Time: 0.075904000 seconds]
Transaction ID: 0x8043
Flags: 0x8180 Standard query response, No error
Questions: 1
Answer RRs: 28
Authority RRs: 0
Additional RRs: 0
Queries
Answers
seed.bitcoin.sipa.be: type A, class IN, addr 83.160.76.197
seed.bitcoin.sipa.be: type A, class IN, addr 88.179.155.125
seed.bitcoin.sipa.be: type A, class IN, addr 90.54.44.37
seed.bitcoin.sipa.be: type A, class IN, addr 98.117.72.130
seed.bitcoin.sipa.be: type A, class IN, addr 104.236.86.251
seed.bitcoin.sipa.be: type A, class IN, addr 107.179.244.59
seed.bitcoin.sipa.be: type A, class IN, addr 109.219.141.49
seed.bitcoin.sipa.be: type A, class IN, addr 173.174.43.26

- **version – verack** (connection initiation)

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	192.168.159.130	89.105.218.192	Bitcoin	180	version
2	0.061655000	89.105.218.192	192.168.159.130	Bitcoin	181	version
3	0.094644000	192.168.159.130	89.105.218.192	Bitcoin	78	verack
4	0.095042000	192.168.159.130	89.105.218.192	Bitcoin	78	getaddr
5	0.096198000	192.168.159.130	89.105.218.192	Bitcoin	86	ping
6	0.096501000	192.168.159.130	89.105.218.192	Bitcoin	1075	getheaders[Malform
7	0.116884000	89.105.218.192	192.168.159.130	Bitcoin	78	verack
8	0.180503000	89.105.218.192	192.168.159.130	Bitcoin	86	[unknown command]
9	0.262058000	89.105.218.192	192.168.159.130	Bitcoin	160	headers
10	0.311981000	192.168.159.130	89.105.218.192	Bitcoin	655	getdata

+ Frame 1: 180 bytes on wire (1440 bits), 180 bytes captured (1440 bits) on interface 0
 + Ethernet II, Src: Vmware_3e:4a:25 (00:0c:29:3e:4a:25), Dst: Vmware_e0:ce:52 (00:50:56:e0:ce:52)
 + Internet Protocol Version 4, Src: 192.168.159.130 (192.168.159.130), Dst: 89.105.218.192 (89.105.218.192)
 + Transmission Control Protocol, Src Port: 60561 (60561), Dst Port: 8333 (8333), Seq: 1, Ack: 1, Len: 180
 - Bitcoin protocol

- Packet magic: 0xf9beb4d9
- Command name: version
- Payload Length: 102
- Payload checksum: 0x69fec962
- Version message
 - Protocol version: 70002
 - + Node services: 0x000000000000000000000001
 - Node timestamp: Mar 14, 2015 15:05:29.000000000 w. Europe Standard Time
 - + Address as receiving node
 - + Address of emitting node
 - Random nonce: 0xc3f1232f2f9af772
 - User agent
 - Count: 16
 - String value: /Satoshi:0.10.0/
 - Block start height: 346096

- **getaddr – addr** (expand communication graph)
- **ping – pong** (keepalive mechanism every 30 minutes)

No.	Time	Source	Destination	Protocol	Length	Info
2510	27.3723930	178.32.219.48	192.168.159.130	Bitcoin	978	addr
2511	27.5620730	178.32.219.48	192.168.159.130	Bitcoin	115	inv
2512	27.6621600	178.32.219.48	192.168.159.130	Bitcoin	115	[TCP Retransmission]
2513	28.0705250	192.168.159.130	89.105.218.192	Bitcoin	115	[TCP Zerowindow], g

Frame 2510: 978 bytes on wire (7824 bits), 978 bytes captured (7824 bits) on interface 0

Ethernet II, Src: Vmware_e0:ce:52 (00:50:56:e0:ce:52), Dst: Vmware_3e:4a:25 (00:0c:29:3e:4a:25)

Internet Protocol Version 4, Src: 178.32.219.48 (178.32.219.48), Dst: 192.168.159.130 (192.168.159.130)

Transmission Control Protocol, Src Port: 8333 (8333), Dst Port: 52666 (52666), Seq: 29549

[21 Reassembled TCP Segments (30027 bytes): #2490(1460), #2491(1460), #2492(1460), #2493(1460), #2494(1460), #2495(1460), #2496(1460), #2497(1460), #2498(1460), #2499(1460), #2500(1460), #2501(1460), #2502(1460), #2503(1460), #2504(1460), #2505(1460), #2506(1460), #2507(1460), #2508(1460), #2509(1460), #2510(1460)]

Bitcoin protocol

Packet magic: 0xf9beb4d9
 Command name: addr
 Payload Length: 30003
 Payload checksum: 0x34f9ff9d

Address message
 Count: 1000

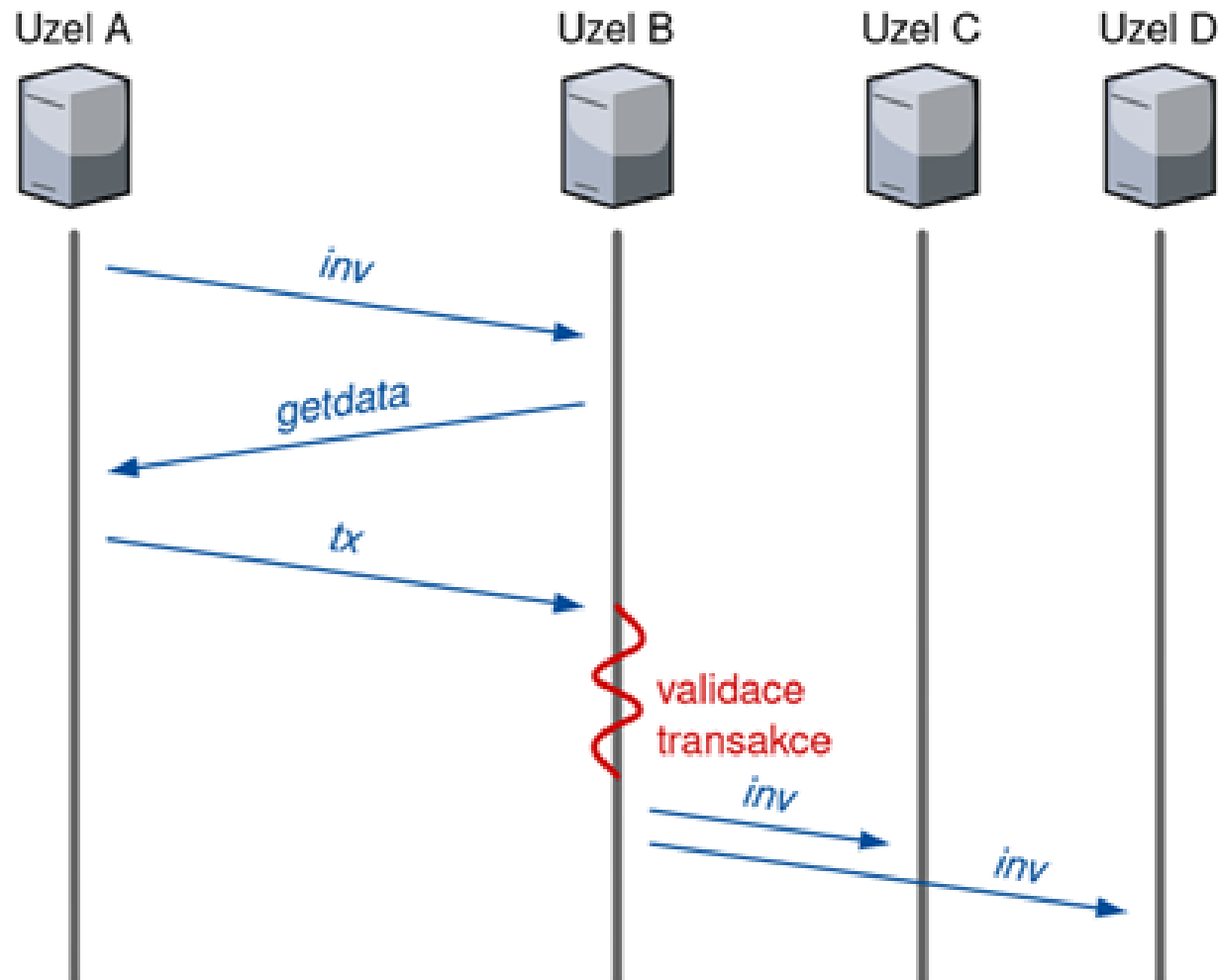
Address: f40e03550100000000000000000000000000000000000000ffff...

Node services: 0x0000000000000001
 Node address: ::ffff:185.15.42.30 (::ffff:185.15.42.30)
 Node port: 8333
 Address timestamp: Mar 13, 2015 17:23:16.000000000 Central Europe Standard Time

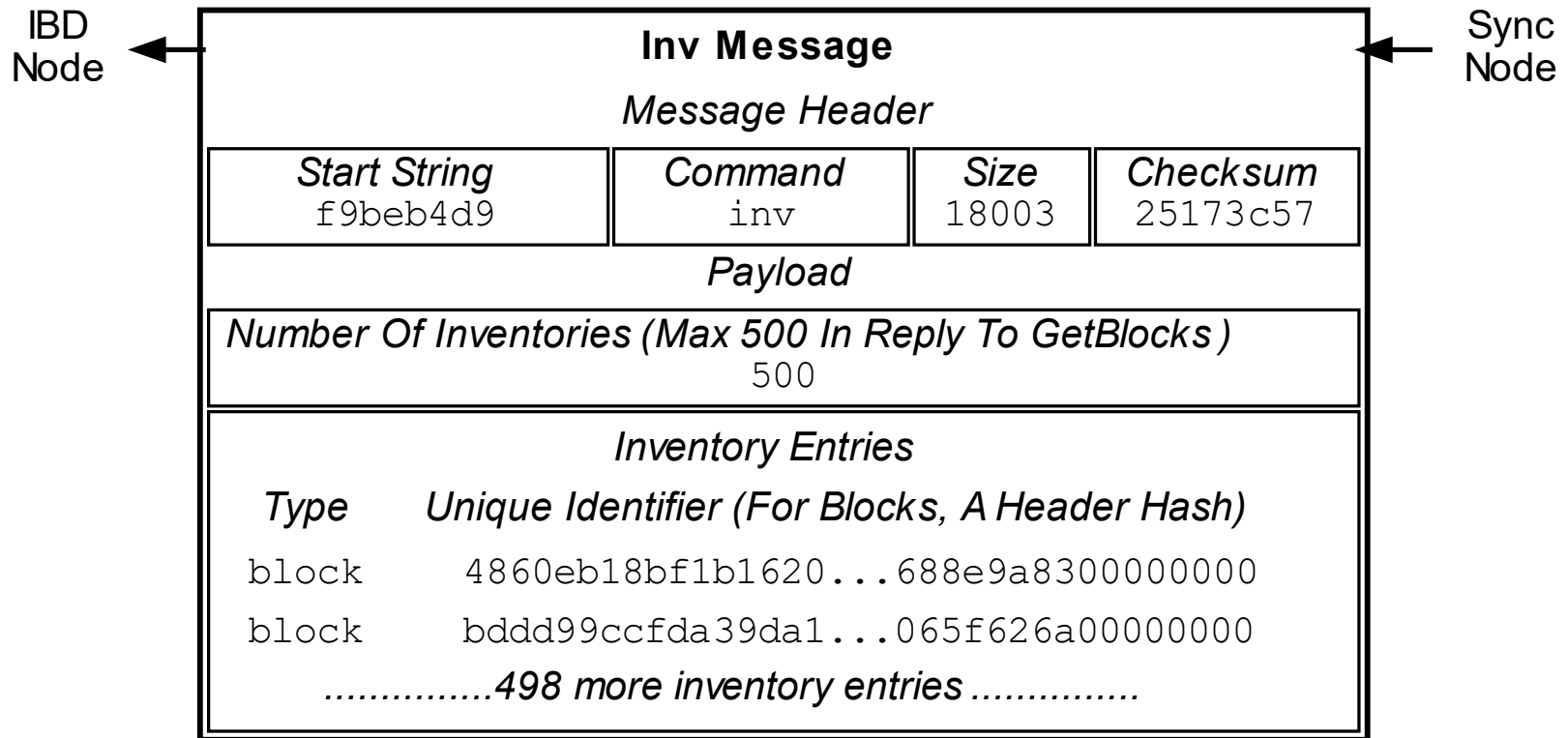
Address: 8ee200550100000000000000000000000000000000000000ffff...

Node services: 0x0000000000000001
 Node address: ::ffff:67.184.223.224 (::ffff:67.184.223.224)
 Node port: 8333
 Address timestamp: Mar 12, 2015 01:49:18.000000000 Central Europe Standard Time

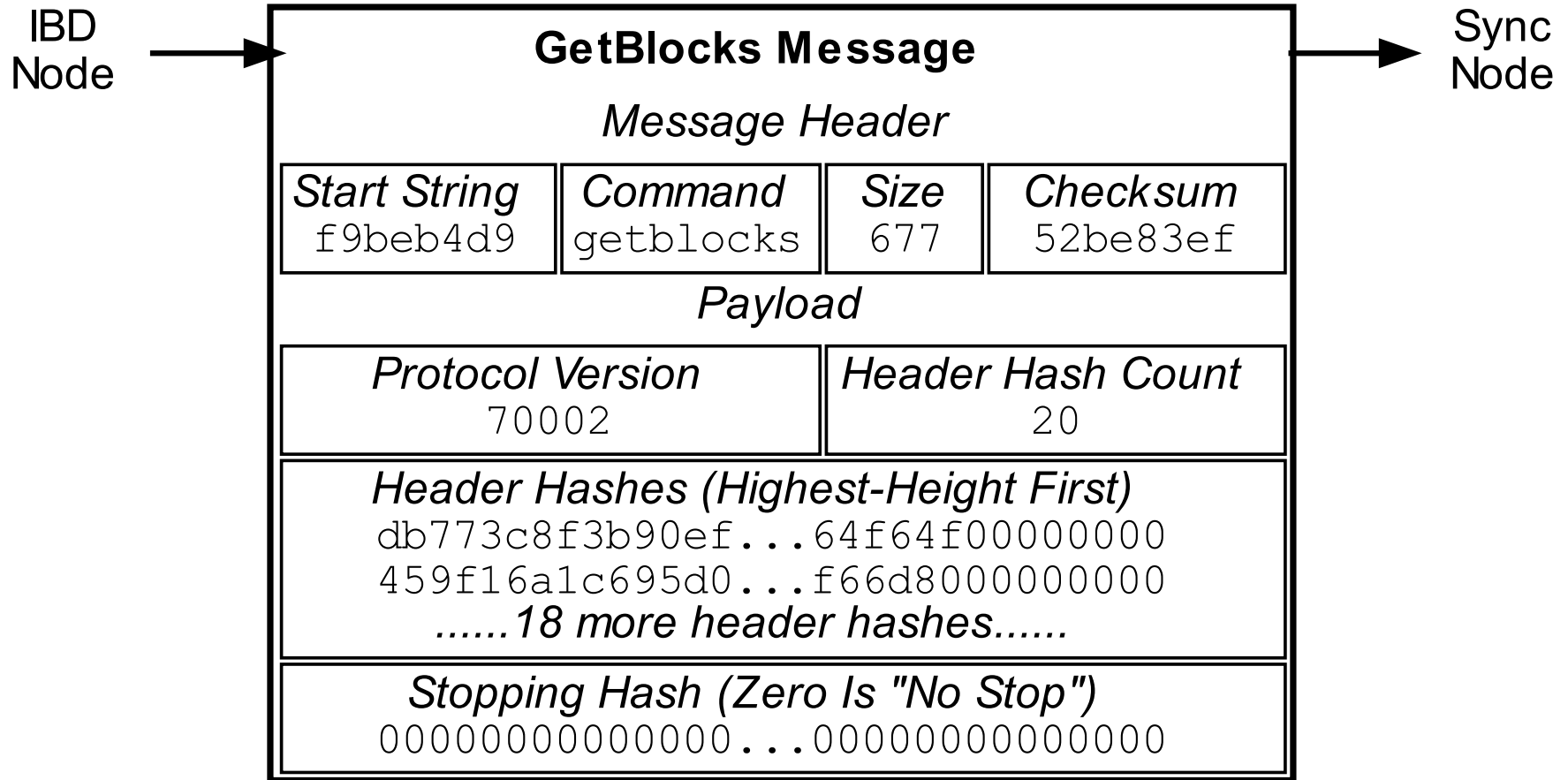
- inv** with id, followed by **getdata** as request



- Download and validation of all blocks up until blockchain tip



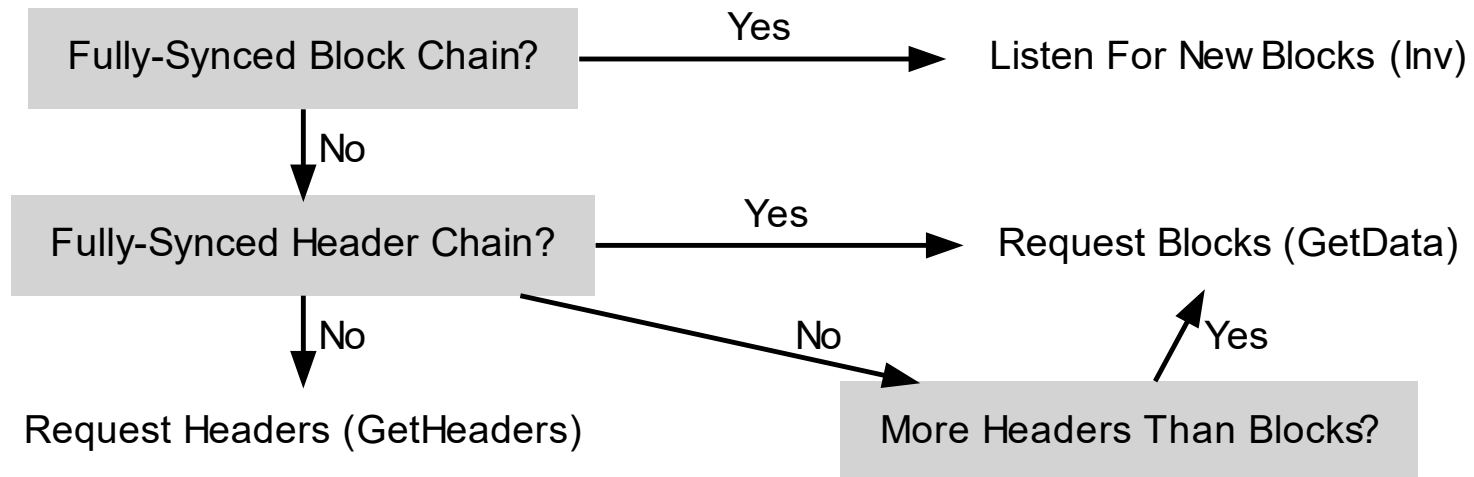
First inv message reply sent to Initial Blocks Download (IBD) node



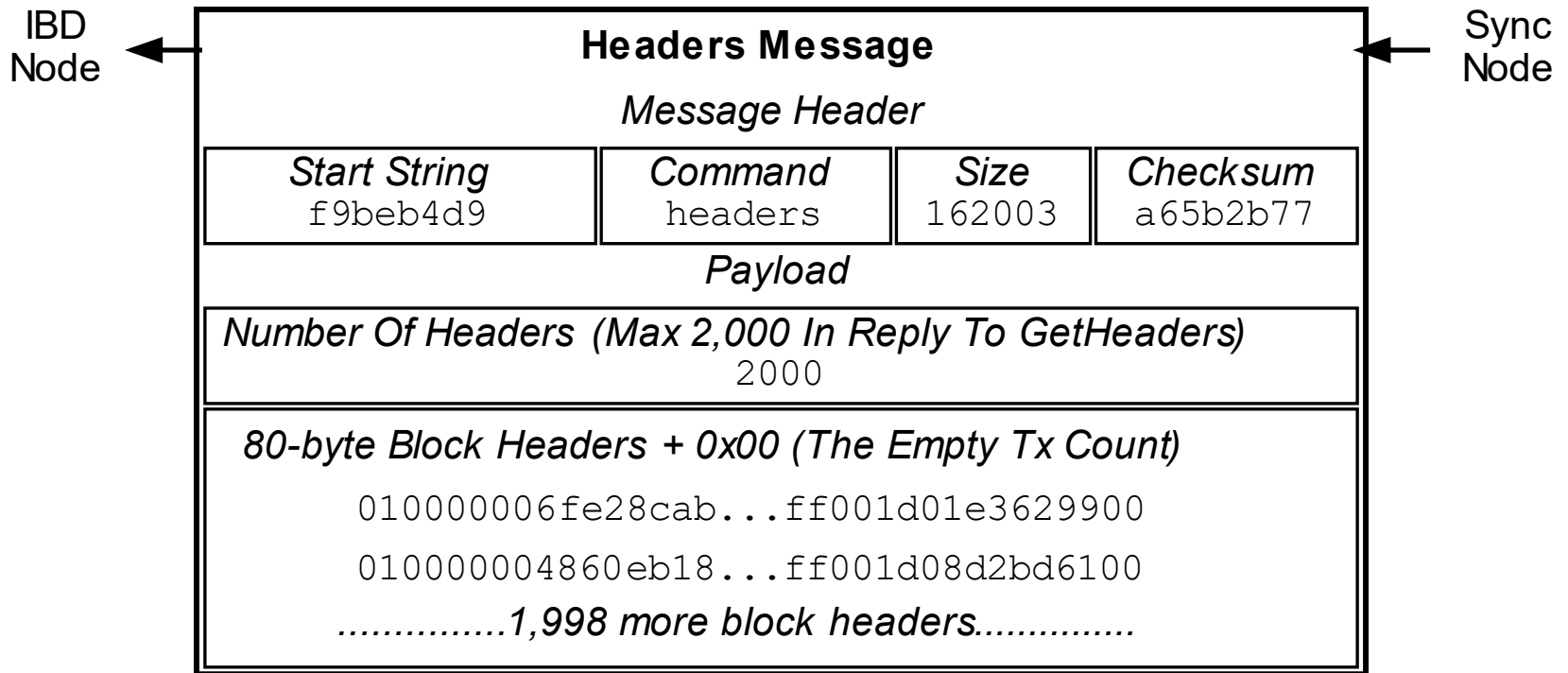
Second getblocks message sent from Initial Blocks Download (IBD) node

- **Speed Limits:** All requests are made to the sync node, so if the sync node has limited upload bandwidth, the IBD node will have slow download speeds. Note: if the sync node goes offline, Bitcoin Core will continue downloading from another node—but **it will still only download from a single sync node at a time.**
- **Download Restarts:** **The sync node can send a non-best (but otherwise valid) block chain to the IBD node.** The IBD node won't be able to identify it as non-best until the initial block download nears completion, forcing the IBD node to restart its block chain download over again from a different node. Bitcoin Core ships with several block chain checkpoints at various block heights selected by developers to help an IBD node detect that it is being fed an alternative block chain history—allowing the IBD node to restart its download earlier in the process.
- **Disk Fill Attacks:** Closely related to the download restarts, if the sync node sends a non-best (but otherwise valid) block chain, **the chain will be stored on disk, wasting space and possibly filling up the disk drive with useless data.**
- **High Memory Use:** Whether maliciously or by accident, the sync node can send blocks out of order, creating orphan blocks which can't be validated until their parents have been received and validated. **Orphan blocks are stored in memory while they await validation, which may lead to high memory use.**

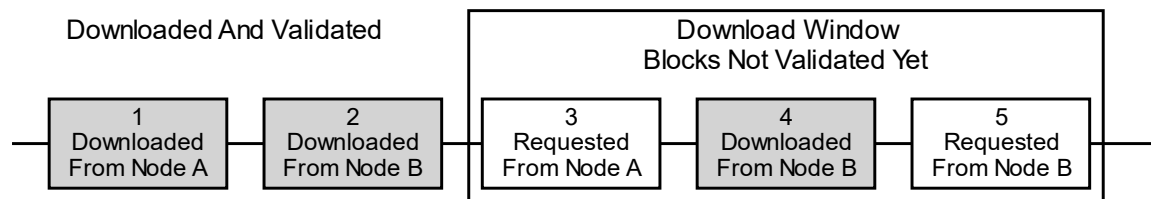
- Since Bitcoin Core 0.10.0
- The goal is to download the headers for the best headers chain, partially validate them as best as possible, and then download the corresponding blocks in parallel.



Overview Of Headers-First Initial Blocks Download (IBD)



First headers message reply sent to Initial Blocks Download (IBD) node



Simulated Headers-First Download Window (Real Window Is Much Larger)

- **Mainnet** – main, global production network (“real” bitcoins)
- **Testnet** – testing network (global, some mining happens...)
 - Restarted from time to time, contains many different types and versions of TXs
- **Regtest** – local instance of Bitcoin network
 - Used for local testing (integration, regression, debugging)
 - Blockchain started from block 0, you are the only miner
 - (mined bitcoins unusable on Mainnet)
 - You can insert own transactions, decide on mining new blocks, debug...
- **Signet** – testing network like Testnet, but with features not yet active on Mainnet
 - Initially for testing Taproot, now for future possible softforks
- **(Lightning** – second layer network of payment channels atop of mainnet)
 - Practically instant and very low fees independently from mainnet

- <https://bitnodes.io/>

BITNODES

Bitnodes estimates the relative size of the Bitcoin peer-to-peer network by finding all of its reachable nodes.

REACHABLE BITCOIN NODES

Updated: Thu Feb 27 08:58:48 2025 CET

21691 NODES

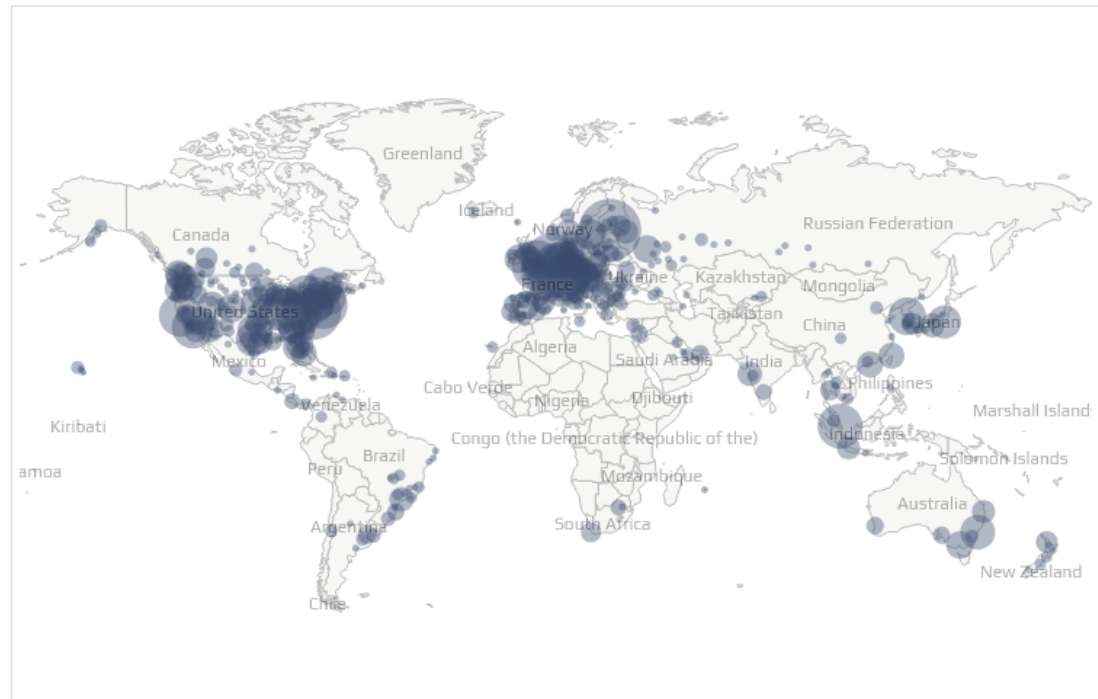
CHARTS

IPv4: +5.3% / IPv6: +6.6% / .onion: +2.7%

Top 10 countries with their respective number of reachable nodes are as follows.

RANK	COUNTRY	NODES
1	n/a	13809 (63.66%)
2	United States	2220 (10.23%)
3	Germany	1322 (6.09%)
4	France	567 (2.61%)
5	Finland	405 (1.87%)
6	Canada	366 (1.69%)
7	Netherlands	361 (1.66%)
8	United Kingdom	275 (1.27%)
9	Switzerland	202 (0.93%)
10	Russian Federation	184 (0.85%)

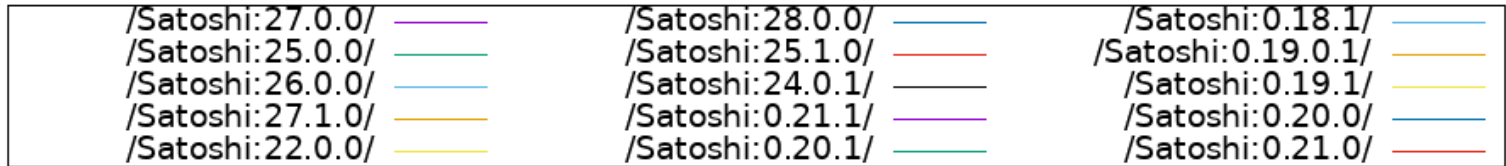
[All \(97\)](#) »



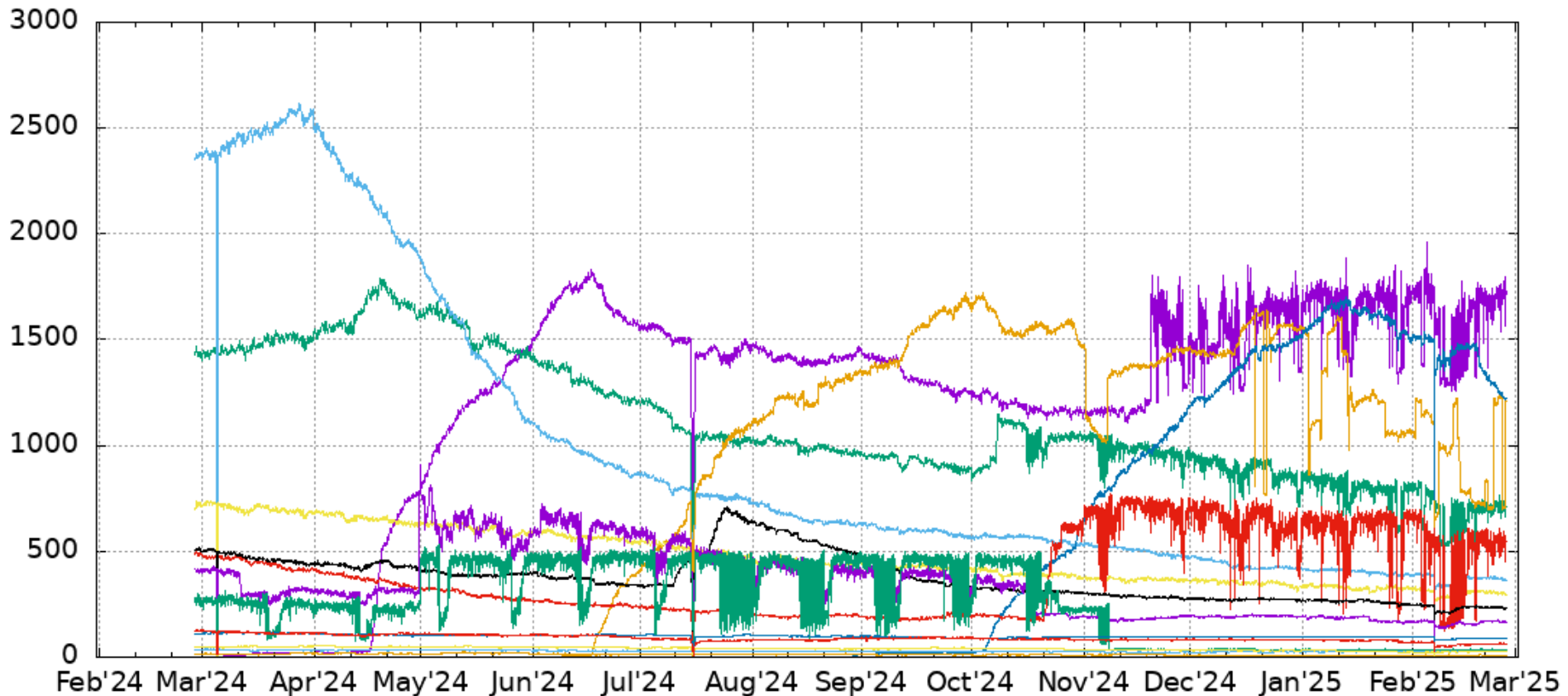
Map shows concentration of reachable Bitcoin nodes found in countries around the world.

LIVE MAP

- <https://www.dsn.kastel.kit.edu/bitcoin/index.html>



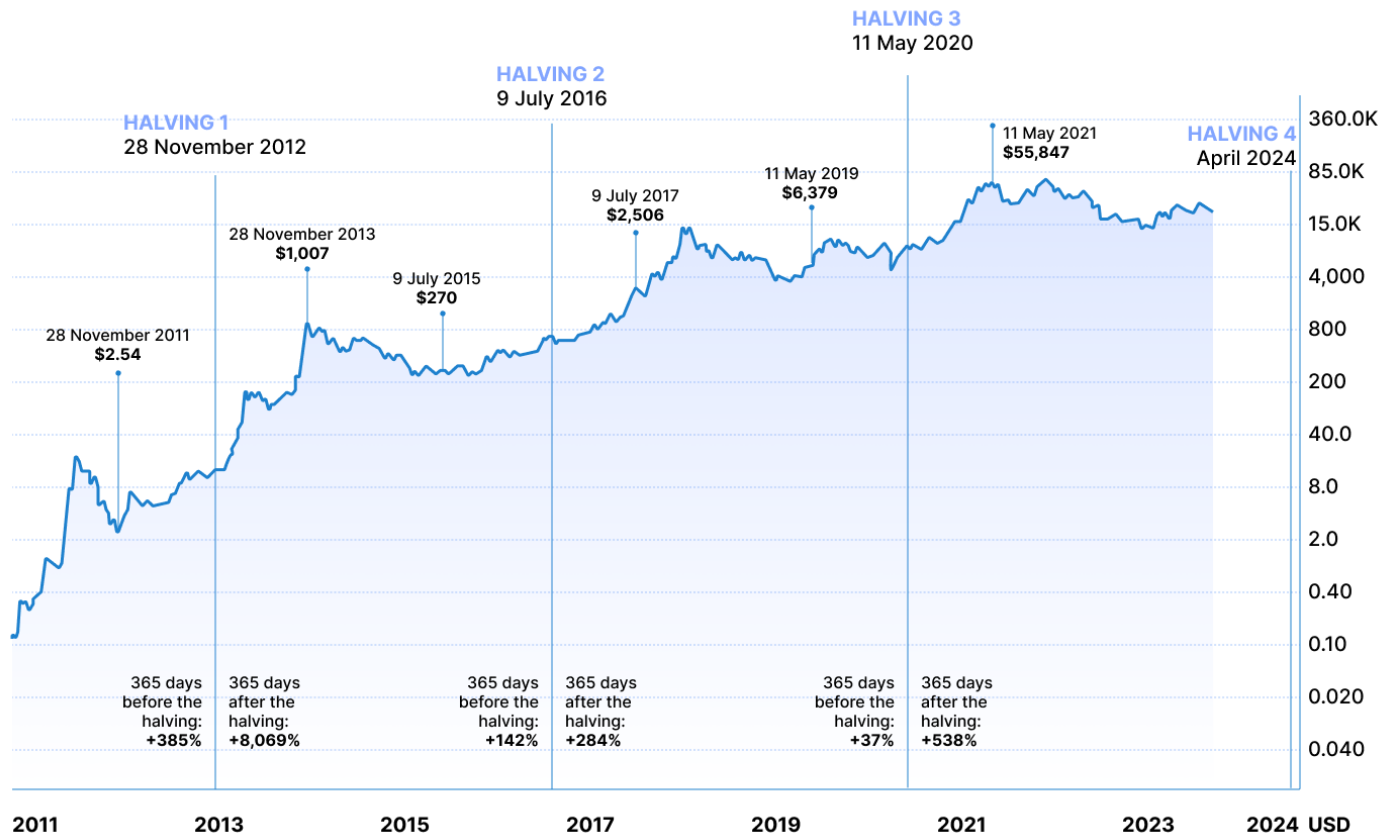
User Agent



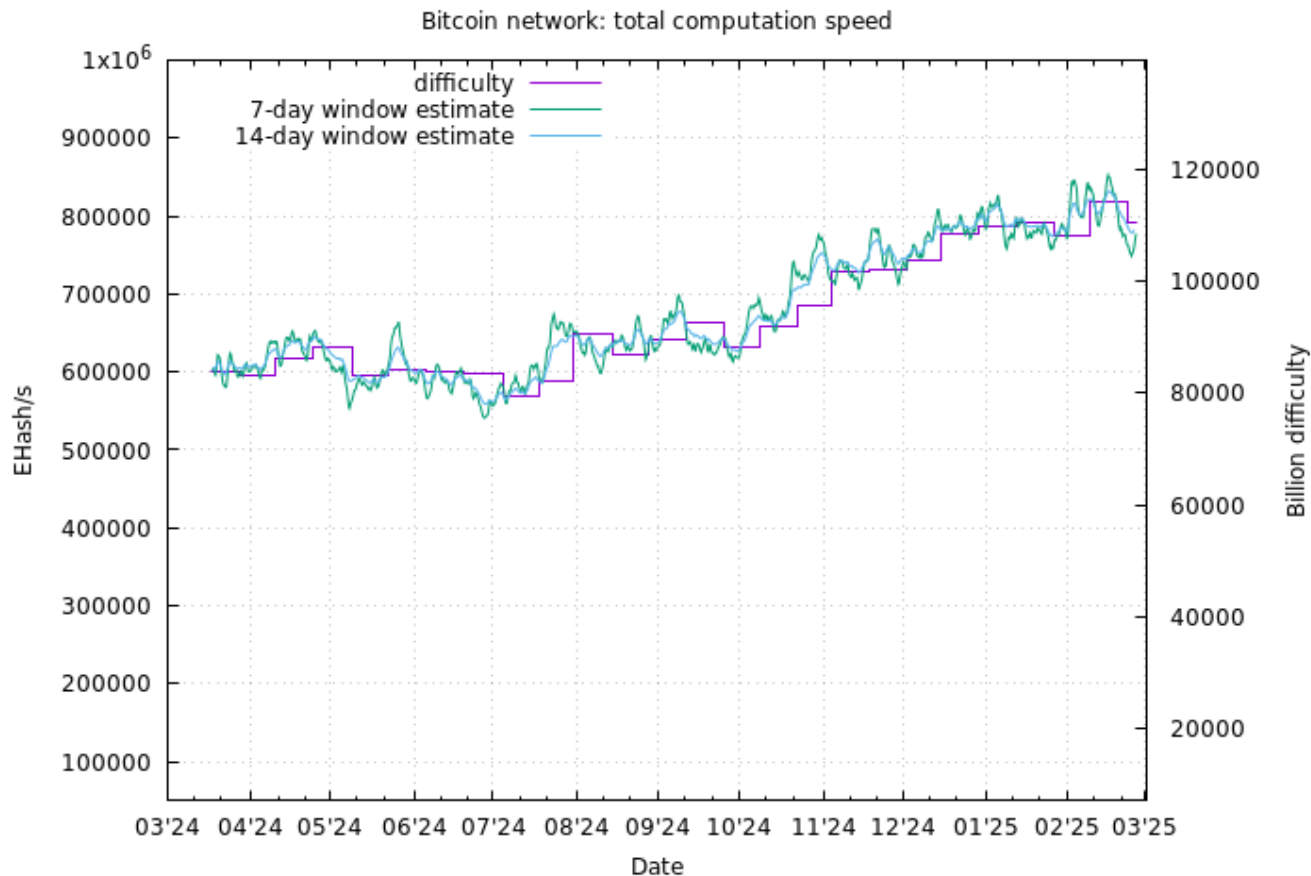
MINING

- Game theory
 - Strategy space: players and moves
 - Payoff function: gain for following a strategy
 - Nash equilibrium: strategies such that no player gains by deviating from them
- Game theory in Bitcoin
 - *Txs to include? Which block to mine on? What if forks have the same length? When to announce a new block? Honest vs malicious behavior? Mine or not?*
 - Payoff function: $reward + fees - cost$
 - Cost is difficult to model (equipment and electricity, prices in fiat, volatility, ...)

- Every 210 000 blocks reward drops $\frac{1}{2}$
- Last halving in the year 2140



- *“Proof-of-work is essentially one-CPU-one-vote. The majority decision is represented by the longest chain, which has the greatest PoW effort invested in it.”*
- <https://bitcoin.sipa.be/index.html>





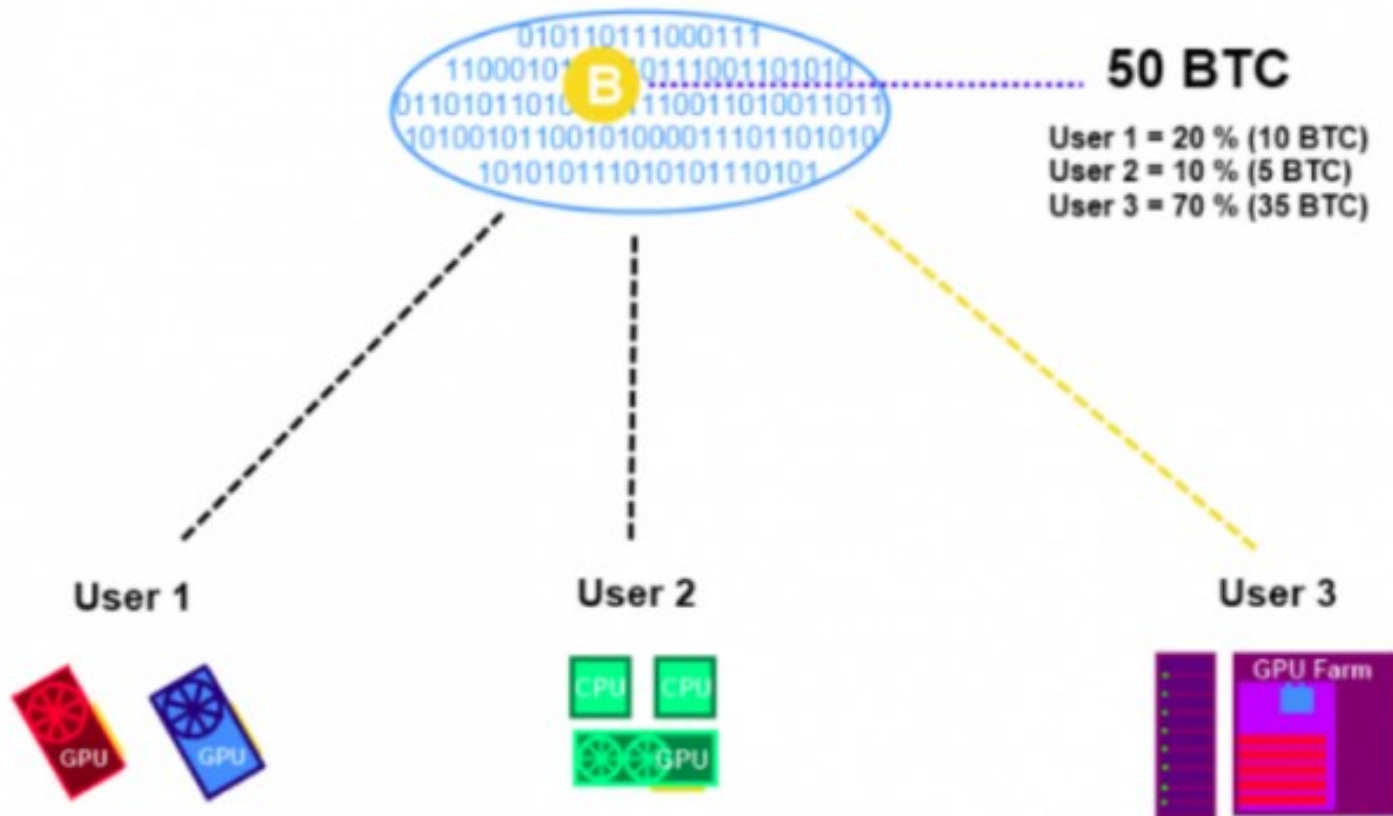
Hardware	Generation	Hashrate
Intel i7 Core 3930k	CPU	66 Mhash/s
AMD Radeon 7970	GPU	710 Mhash/s
nVidia GeForce GTX 590	GPU	190 Mhash/s
BitForce SHA256 Single	FPGA	830 Mhash/s
ModMiner Quad	FPGA	800 Mhash/s
ButterFly Labs Single SC	ASIC	30 Ghash/s
Avalon1 A3256 Miner	ASIC	66 Ghash/s

Table 1: A comparison of Bitcoin mining hardware relevant to 2013

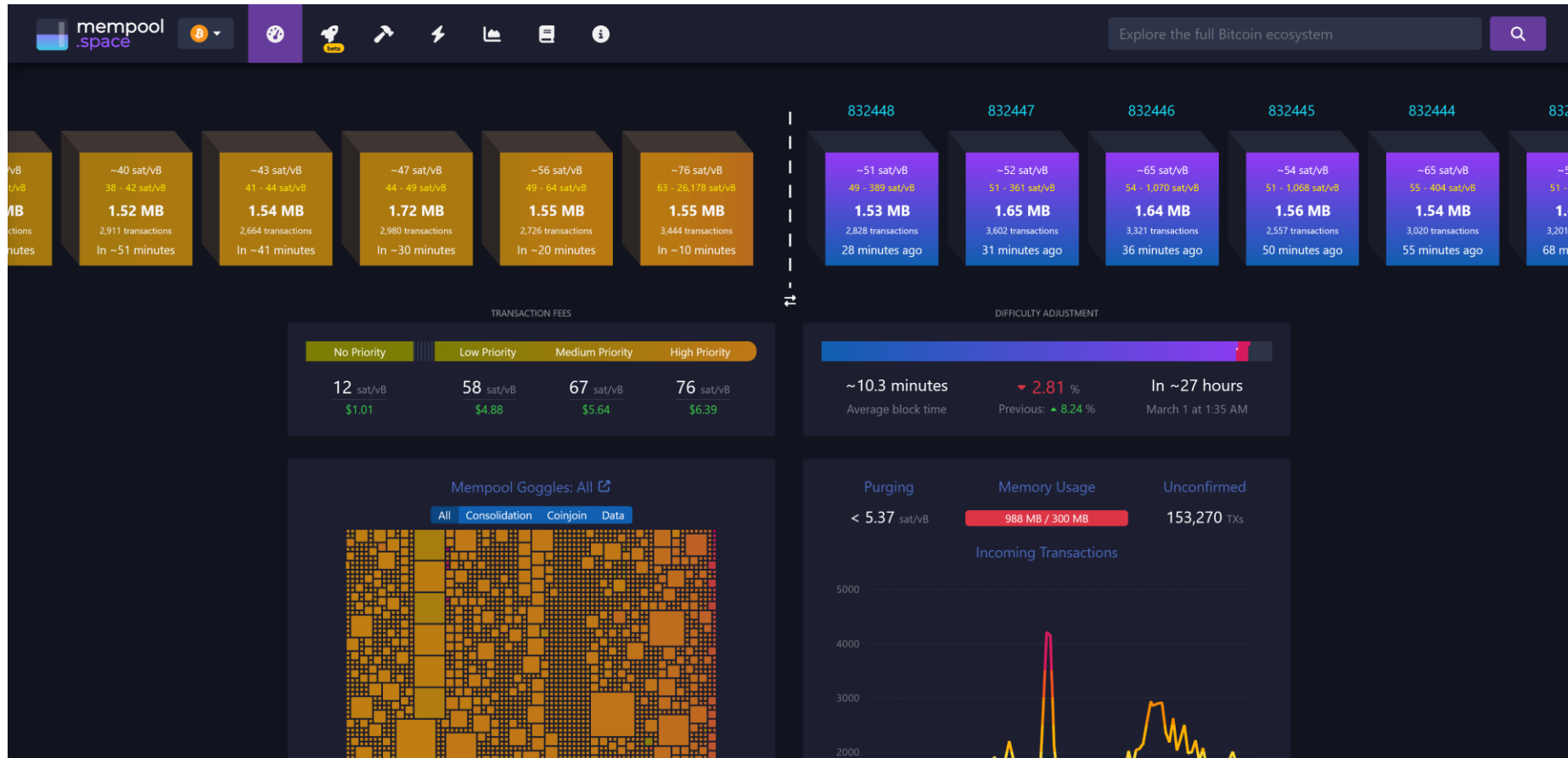
- <https://www.nicehash.com/profitability-calculator/nvidia-rtx-4090>
- https://whattomine.com/coins?aq_4090Ti=1

	1 DAY	1 WEEK	1 MONTH
 Income	0.00003076 BTC 1.82 USD	0.00022975 BTC 13.61 USD	0.00104731 BTC 62.04 USD
 El. costs	0.00000000 BTC 0.00 USD	0.00000000 BTC 0.00 USD	0.00000000 BTC 0.00 USD
 Profit	0.00003076 BTC 1.82 USD	0.00022975 BTC 13.61 USD	0.00104731 BTC 62.04 USD

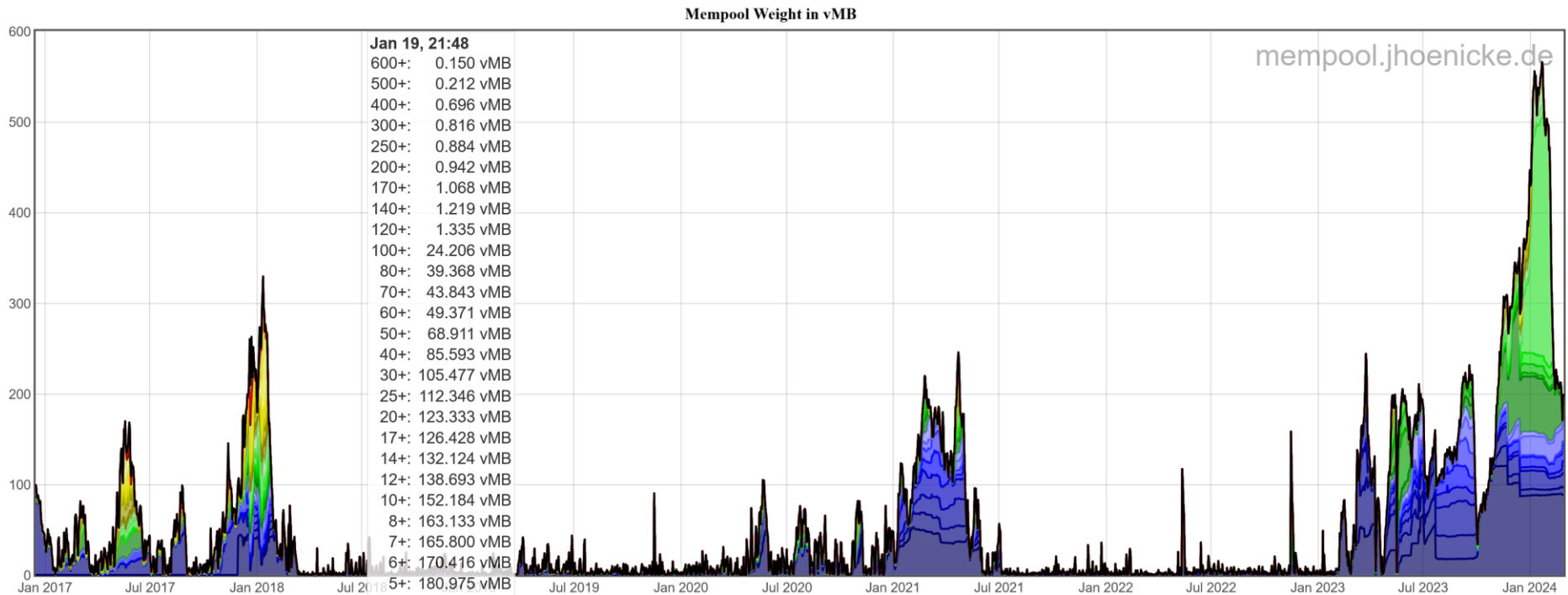
- Client-server communication model
- Distribute workload and collect results



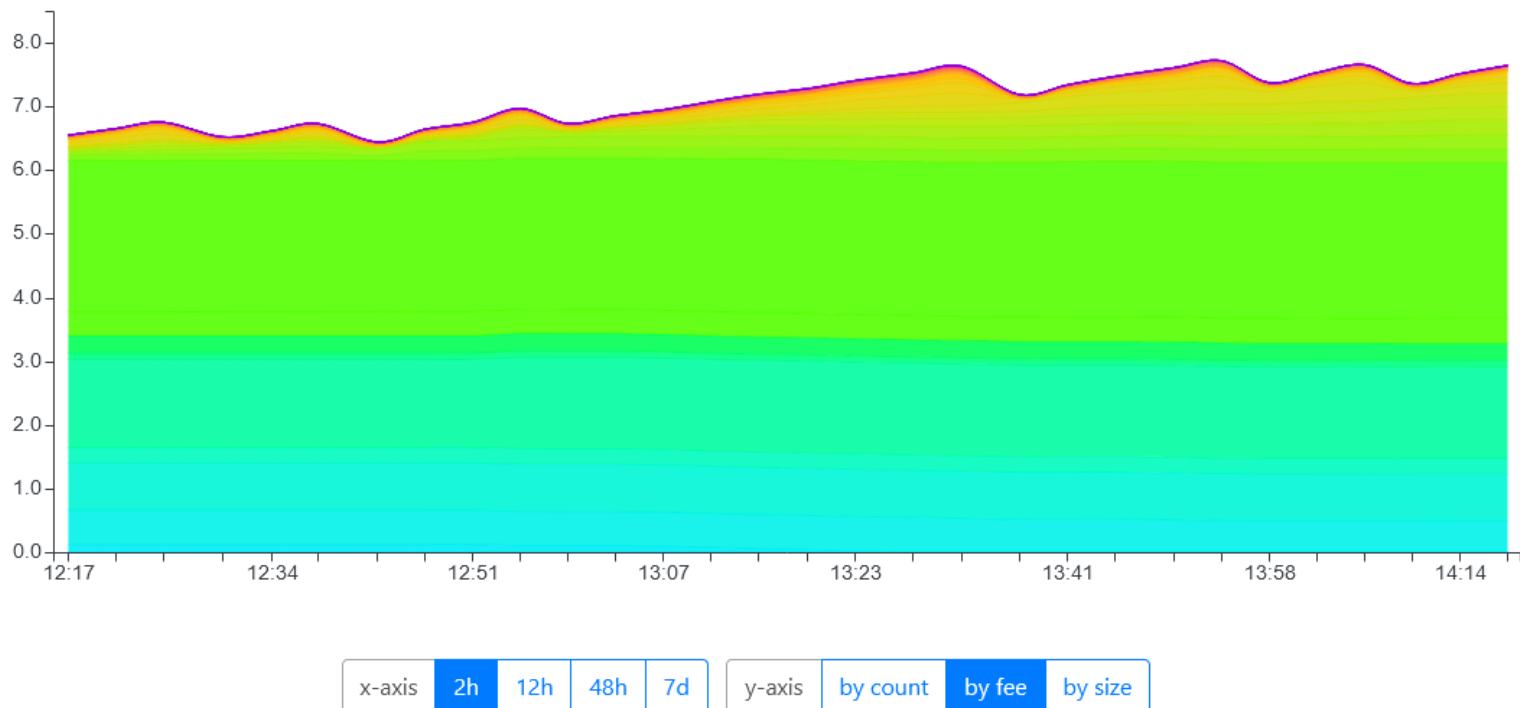
- <https://mempool.space/>



- <https://jochen-hoenicke.de/queue/#BTC,all,weight>



- A vByte is a unit of measure for the weight of blocks and transactions.
- vByte is equivalent to 4 weight units, and thus, a block is limited to being 1 vMegabyte large, or 4 million weight units.
- Wallets usually calculate and display transaction fees in terms of sats/vByte, meaning the fee is paid per vByte of data used.
- <https://mempool.observer/>



- registration-oriented
- registration-less

[Home](#) / [Getting started](#) / [Getting Started - Bitcoin](#)

How can we help?

[← Back](#)

**Mining for Beginners**
Want see the big picture?

**Getting Started - Bitcoin**
Mining for the first time?

**Getting Started - Zcash**
How to set everything up?

**Advanced Mining Setup**
Connect your ASIC based miner

**Stratum Mining Proxy**
Connect your legacy HW

Getting Started - Bitcoin

In order to start mining you basically need just two things, create an account with our pool and setup your miner.

1. **Sign-up** for a new account
 1. [Sign-up](#) and wait for a confirmation email.
 2. [Login](#) to your account.
2. **Configure Your Device**

Your miner has to be pointed to one of the stratum servers below and user credentials for your account have to be specified. We currently operate in the following regions: US east coast (us-east), Europe (eu), China mainland (cn), Singapore - South Asia (sg) and Japan - Pacific (jp).

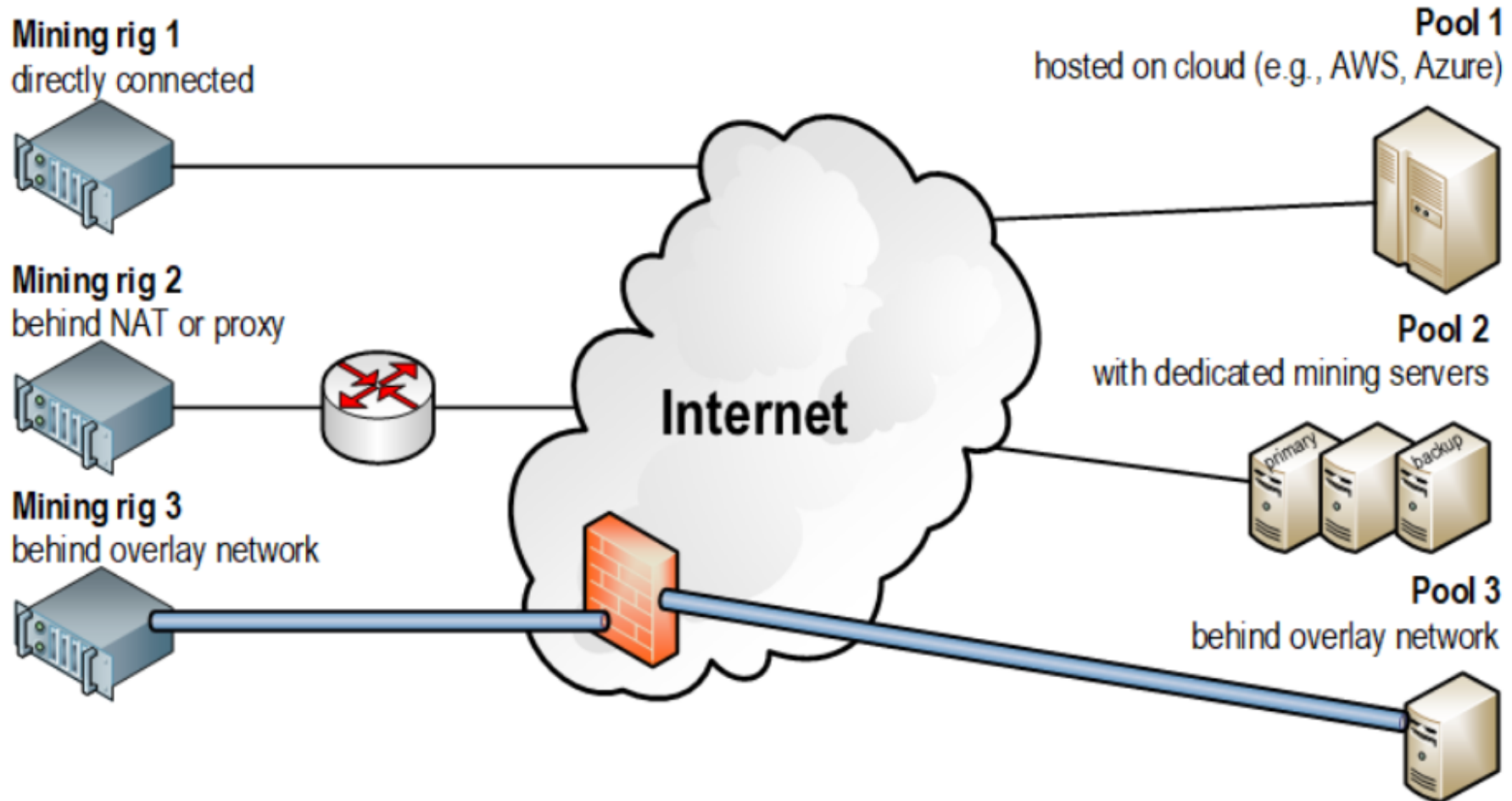
The login credentials needed for your miner look like this: (please, fill your **user ID** and **worker name**)

```
URL: stratum+tcp://stratum.slushpool.com:3333
userID: userName.workerName
password: anything
```

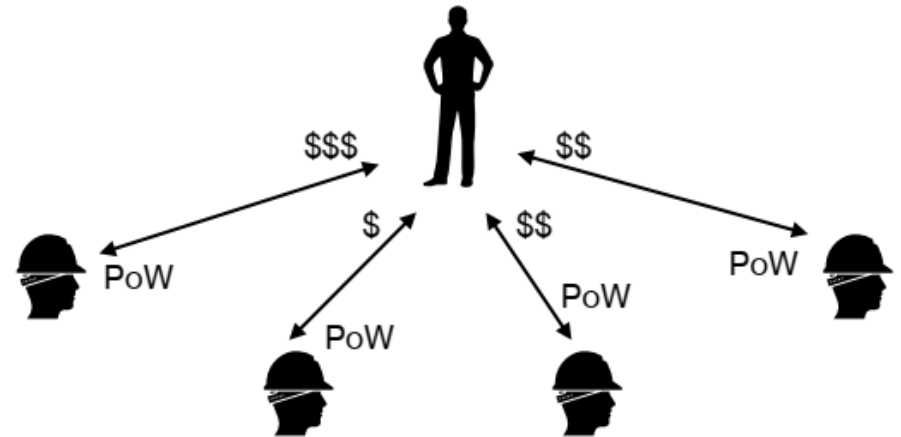
The password can be an arbitrary text since there is no security issue present here. If someone tried to connect to our servers with your credentials, he would be just mining for your benefit.

The servers can be chosen from the following list based on your geographical location:

Servers Location	Address
USA, east coast	stratum+tcp://us-east.stratum.slushpool.com:3333
Canada	stratum+tcp://ca.stratum.slushpool.com:3333
Europe	stratum+tcp://eu.stratum.slushpool.com:3333
China, mainland	stratum+tcp://cn.stratum.slushpool.com:3333 stratum+tcp://cn.stratum.slushpool.com:443
Singapore, South Asia	stratum+tcp://sg.stratum.slushpool.com:3333
Japan, Pacific	stratum+tcp://jp.stratum.slushpool.com:3333



- Pay-per-Share
- Proportional
- Bitcoin Pooled mining
- Pay-per-last-N-shares
- Solo Mining Pool
- Peer-to-Peer Mining Pool
- Geometric method
- Double Geometric method



- https://en.wikipedia.org/wiki/Mining_pool#Mining_pool_methods

- **GetWork** was the first mining protocol ever.
- Comparing to its descendants, GetWork is a simple request-response scheme protocol, where server assigns work package and miner blindly conducts mining task.
- Due to its simplicity, GetWork allows doublespent transactions in the case of corrupt pool operator. GetWork messages with JSON syntax are carried inside HTTP.
- GetWork supports a limited number of protocol extensions using additional HTTP header lines.

- **GetBlockTemplate** is official mining protocol developed in the frame of Bitcoin community but also adopted by other cryptocurrencies.
- GetBlockTemplate was codified in BIP22
- GetBlockTemplate is more decentralized by offloading block creation process onto miners instead of pools.
- GetBlockTemplate increases potential work package size and reduces mining protocol overhead to support performance delivered by ASIC miners.
- Moreover, BIP 23 standardizes extensions and ways how to flexible improve GetBlockTemplate without any major protocol redesign or non-conformant HTTP header hacks.

- **Stratum** protocol was prototyped by M. Palatinus, inventor of pooled mining and operator of the oldest Slush pool.
- Stratum development was motivated by a need to remedy design flaws of previous two protocols:
 - a) by removing HTTP as a carrier, which reduces unnecessary protocol overhead;
 - b) by removing long polling feature that posed scalability issue for load balancing of traffic between miner and server;
 - c) and by adding extranonce field that allows miner to generate more hashes locally without bothering a mining server for a new batch of work.
- Stratum is JSON-RPC 2.0 compatible protocol that operates directly above TCP

[illegible]

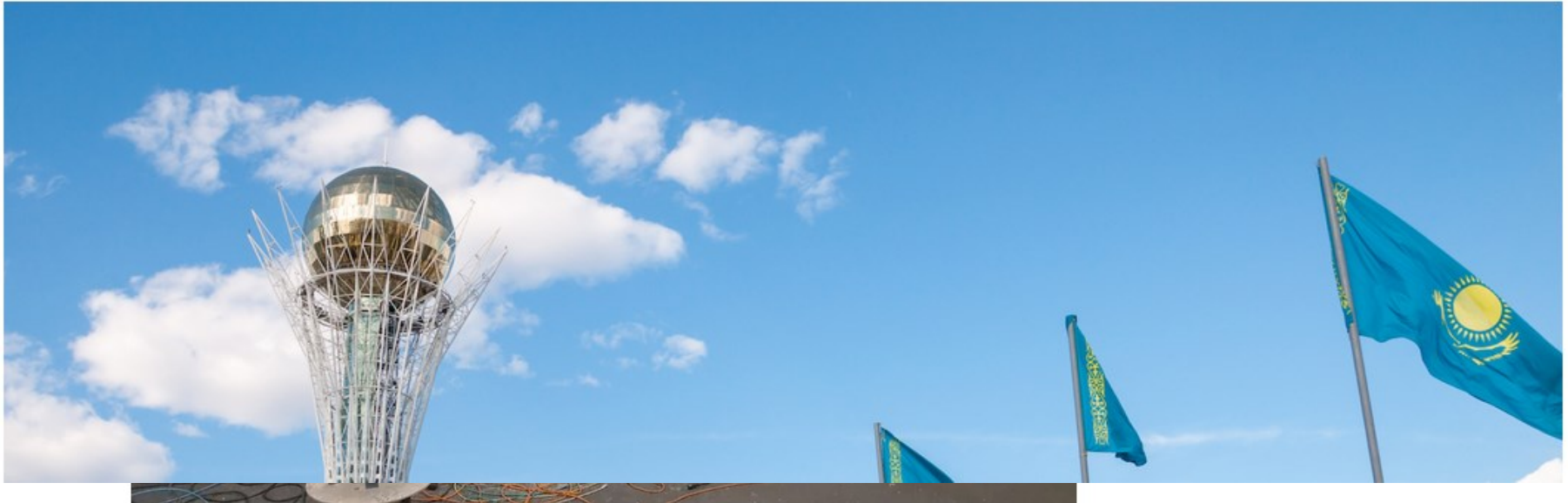
Figure 2: Example of Stratum protocol message exchange

UK police surprised to learn energy-intensive weed
Business

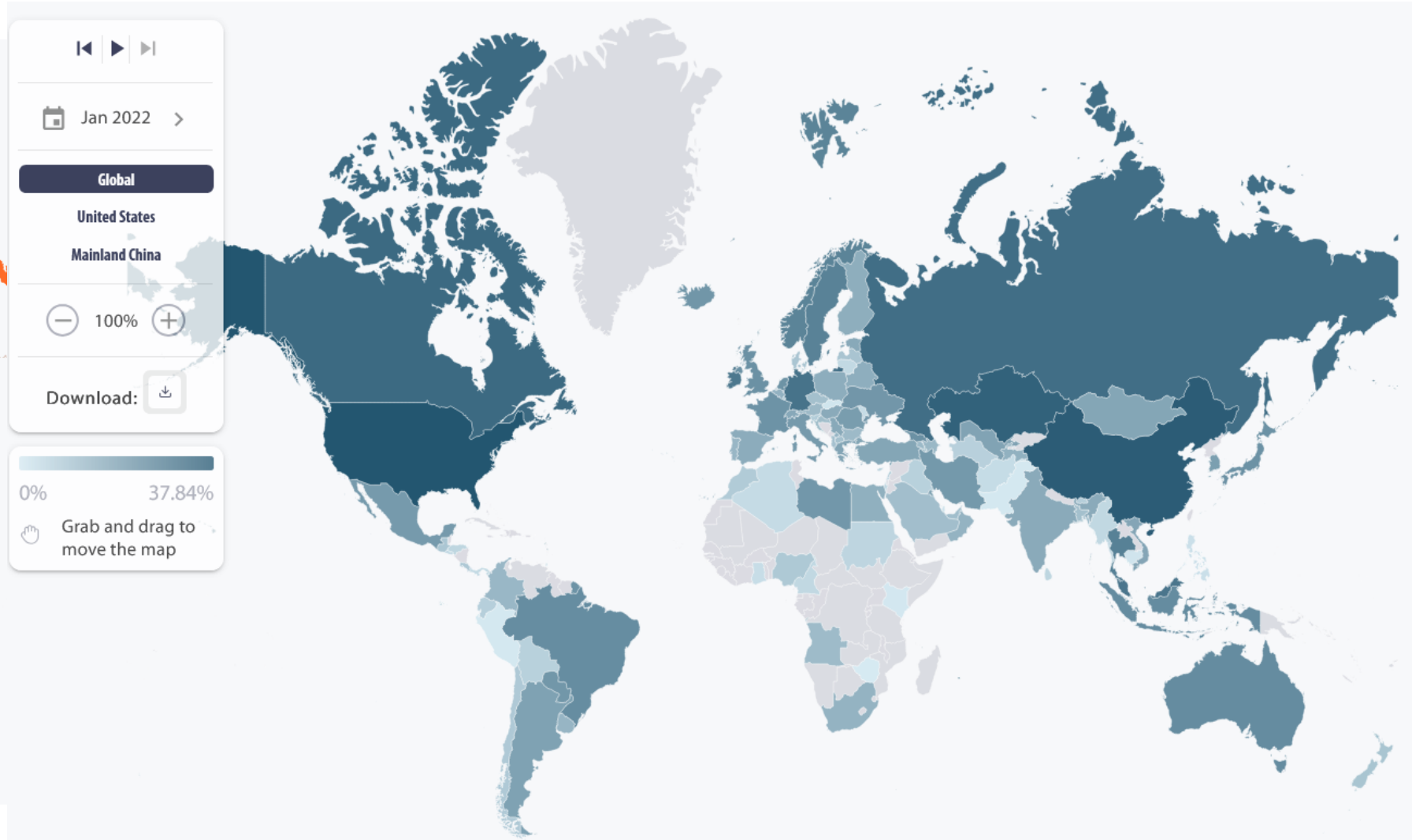
Kazakh Crypto Miners Cut Off From Electricity Supply Until End of January

Kazakhstan's energy problem worsened on Tuesday when a major transmission line was disconnected.

By Eliza Gkritsi · ⌚ Jan 25, 2022 at 2:32 p.m. UTC · Updated Jan 25, 2022 at 4:34 p.m. UTC



- https://ccaf.io/cbnsi/cbeci/mining_map



Q & A

- <https://bitcoin.org/bitcoin.pdf>
- <https://github.com/bitcoin/bips>
- <https://bitcoin.org/en/developer-guide>
- <https://queue.acm.org/detail.cfm?id=3136559>
- <https://medium.com/@ackhor>

How a Bitcoin transaction works

Bob, an online merchant, decides to begin accepting bitcoins as payment. Alice, a buyer, has bitcoins and wants to purchase merchandise from Bob.

WALLETS AND ADDRESSES



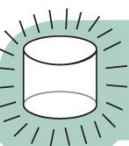
Bob and Alice both have Bitcoin "wallets" on their computers.



Wallets are files that provide access to multiple Bitcoin addresses.



An address is a string of letters and numbers, such as 1HULMwZEPkJEPeCh43BkKJlybLCWrDpN.



Bob creates a new Bitcoin address for Alice to send her payment to.

CREATING A NEW ADDRESS

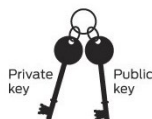


Each address has its own balance of bitcoins.

SUBMITTING A PAYMENT



Alice tells her Bitcoin client that she'd like to transfer the purchase amount to Bob's address.



Public Key Cryptography 101

When Bob creates a new address, what he's really doing is generating a "cryptographic key pair," composed of a private key and a public key. If you sign a message with a private key (which only you know), it can be verified by using the matching public key (which is known to anyone). Bob's new Bitcoin address represents a unique public key, and the corresponding private key is stored in his wallet. The public key allows anyone to verify that a message signed with the private key is valid.

Private key



Alice's wallet holds the private key for each of her addresses. The Bitcoin client signs her transaction request with the private key of the address she's transferring bitcoins from.



Public key

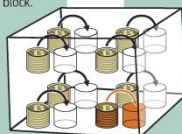
Anyone on the network can now use the public key to verify that the transaction request is actually coming from the legitimate account owner.

VERIFYING THE TRANSACTION

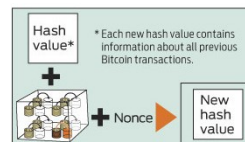
Gary, Garth, and Glenn are Bitcoin miners.



Their computers bundle the transactions of the past 10 minutes into a new "transaction block."



The miners' computers are set up to calculate cryptographic hash functions.



The mining computers calculate new hash values based on a combination of the previous hash value, the new transaction block, and a nonce.

Cryptographic Hashes

Cryptographic hash functions transform a collection of data into an alphanumeric string with a fixed length, called a hash value. Even tiny changes in the original data drastically change the resulting hash value. And it's essentially impossible to predict which initial data set will create a specific hash value.

The root of all evil	6d0a1899 086a... (56 more characters)
The root of all evil	486c 6be4 6dde...
The root of all evil	b8db 7ee9 8392...

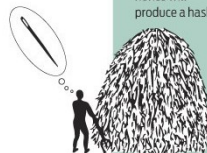
Nonces

To create different hash values from the same data, Bitcoin uses "nonces." A nonce is just a random number that's added to data prior to hashing. Changing the nonce results in a wildly different hash value.

The root of all evil ??? → 0000 0000 0000 ...

Creating hashes is computationally trivial, but the Bitcoin system requires that the new hash value have a particular form—specifically, it must start with a certain number of zeros.

The miners have no way to predict which nonce will produce a hash



value with the required number of leading zeros. So they're forced to generate many hashes with different nonces until they happen upon one that works.



Each block includes a "coinbase" transaction that pays out 50 bitcoins to the winning miner—in this case, Gary. A new address is created in Gary's wallet with a balance of newly minted bitcoins.

TRANSACTION VERIFIED

As time goes on, Alice's transfer to Bob gets buried beneath other, more recent transactions. For anyone to modify the details, he would have to redo the work that Gary did—because any changes require a completely different winning nonce—and then redo the work of all the subsequent miners. Such a feat is nearly impossible.



Vladimír Veselý veselyv@fit.vut.cz



- výzkumná skupina NES@FIT = specializace NNET
 - <https://www.fit.vut.cz/research/groups/nes@fit/>
 - <https://nesfit.github.io>
- repo s různými projekty
 - <https://git.fit.vutbr.cz/NESFIT>